

High-Performance Graph Processing on Multicore Systems

CHEN, YuAng

A Thesis Submitted in Partial Fulfillment
of the Requirements for the Degree of
Doctor of Philosophy
in
Computer and Information Engineering

The Chinese University of Hong Kong, Shenzhen
June 2022

Thesis Assessment Committee

Professor ZHANG, David (Chair)

Professor CHUNG, Yeh-Ching (Thesis Supervisor)

Professor FANG, Yixiang (Committee Member)

Professor CAI, Wei (Committee Member)

Professor LO, Chi Lik Eric (Examiner from CUHK)

Professor YU, Zhibin (External Examiner)

Abstract

Graph data structure is a non-linear organization composed by a set of vertices that are connected by edges. It depicts the relationships between objects. Hence, graphs are used to model complicated systems and processes in diverse domains, such as data mining, machine learning, etc. The growing size and complexity of graph data increase the need for high-performance graph systems.

This thesis explores the optimization space of graph processing on multicore systems. The irregular nature of graph data is analyzed in order to understand the root cause of performance bottleneck. Specifically, the correlations between graph algorithms and hardware activities are investigated in the aspects of cache efficiency, memory traffic, thread balance and multicore parallelism. Experimental evaluations and theoretical analyses demonstrate a trade-off amongst these factors is inevitable to achieve optimal performance.

In this paper, five-fold contributions, including algorithms, paradigms and frameworks, are presented to accelerate the computation of graphs. (1) Corder, a cache-aware reordering algorithm exploiting the memory hierarchy. Within the shared memory, Corder promotes balanced loads amongst cores. Inside the private cache, Corder enhances cache efficiency by refining local vertex order. (2) Syze, an unequal caching framework that unequally segments a graph into cache-able subgraphs of varying sizes. Bit-aware propagation mechanism is devised to facilitate the communication between subgraphs. (3) HiPa, a novel hierarchical partitioning paradigm utilizing the memory architecture of modern machines. HiPa firstly splits the graph based on NUMA memory to reduce remote memory. Next, the graph is segmented into cache-able partitions to promote in-core computing. (4) Mixen, a processing engine mixing matrix compressing formats and propagation flows. The adjacency matrix of a graph is reformatted in a new scheme CSMix. (5) Lebo, extracts the locality out of graph for blocking on the adjacency matrices of graphs. In every iteration of computation, a hybrid traversal scheme is deployed on different graph portions.

The ultimate goal of this thesis is to advance the high-performance computing for irregularly structured data.

摘要

图是一种由节点和边构成的非线性的数据结构，它描绘了物体之间的关系。因此，图结构被广泛地应用在各种的领域,用来模拟复杂的系统及其变化的过程。随着图数据规模的不断增长，对高性能的图处理系统的需求也在日益增加。

本论文探索了图计算在单机多核系统下的优化空间。首先，为了定位性能瓶颈，我们分析并总结了图数据在结构上的不规则性。通过实验，我们研究了图算法对于底层系统在缓存效率、内存访问、负载平衡和计算并行度等性能的影响。实验结果和理论分析表明，为了取得最快的运行速度，这些因素之间的相互妥协是不可避免的。针对图计算在多核架构上的不同挑战，我们提出了五个新的算法/范式/框架：

1) **Corder** – 基于缓存结构的图重排序算法。在计算机的共享内存中，**Corder** 实现了处理器核心之间的负载均衡。然后，在每个核心的私有缓存内，该算法进一步地优化了本地数据的访问策略。

2) **Syze** – 主动式非均匀缓存的图分析框架。**Syze** 把图分割成可以嵌入缓存的子图，每个子图会迭代分割直到所有子图的计算量大致相等。通过对图的节点的比特位进行编码和解码，**Syze** 有效地减少子图之间的通信开销。

3) **HiPa** – 内存-缓存架构下的分层式图分割方法。**HiPa** 首先根据多核系统的 NUMA（非一致内存访问）特征将图数据进行第一次分割，以减少远程内存访问。然后，在每个 NUMA 节点内部，子图被再次根据缓存的尺寸进行第二次分割。

4) **Mixen** – 图遍历混合引擎。**Mixen** 将图重新格式化，转换成一种混合了多种压缩编码的新格式 **CSMix**。在每次图遍历的过程中，不同的子格式采用不同遍历的方式。

5) **Lebo** – 图的邻接矩阵的局部区块化。**Lebo** 通过图的重排序，将图的邻接矩阵转化成一个分为稠密和稀疏部分的新矩阵。稠密部分被分成了小的区块，并且根据“推”的模式进行数据更新；稀疏矩阵则采用传统的“拉”模式。

本文的根本目标是提升面向不规则结构大数据的高性能计算。

Acknowledgments

Firstly, I want to express my sincere gratitude to my advisor, Prof Yeh-Ching Chung, who generously offers me enormous motivations, guidance, supports and encouragements during my Ph.D study. He truly cares about the building of research capacity, which allows me to develop sustainable skills to perform high-quality research. It is my fortune to be immersed in an ideal academic environment where the freedom and independence of scientific research are granted.

Besides, I am grateful to the rest of my thesis committee members: Prof. David Zhang, Prof. Yixiang Fang, Prof. Wei Cai, Prof. Eric Lo, and Prof. Zhibin Yu, for their insightful comments and valuable feedback on my thesis draft and oral defense. Also, I would like to thank Dr. Yonggang Li for his friendship and mentorship. His rigorous and positive attitude towards science remarkably inspired me, especially when I was confused by technical details or frustrated by experimental results.

In addition, many thanks go to my lab mates: Ao Shan, Shihao Hong, Weibin Chen, Yufan Yang, Wenyu Huang for the daily discussions and the assistance in coursework, and for the laughter in the lab. Further, I would like to thank Wenqi Zeng, who is not only my girlfriend, but also my best friend. I feel blessed to meet someone who shares the same sense of humor and similar life vision with me. She provides me with tremendous comforts and joy. I wish to enjoy her bakery for a lifetime.

Most of all, this thesis could not have been written without the selfless love of my parents, Liu Yang. and Shirong Jiang. They brought me into this world, and have been unconditionally supporting me during my struggling. I am deeply indebted to them.

Dedicated to my parents

Contents

1	Introduction	1
1.1	Graph Frameworks	1
1.2	Challenges of Shared-Memory Graph Processing	3
1.3	Research Contributions	4
2	Background and Related Work	7
2.1	Graph Basics	7
2.2	Graph Reordering	9
2.3	Graph Partitioning	11
2.4	SpMV on Graphs	12
3	Cache-Aware Reordering	15
3.1	Estimating Imbalance	15
3.2	Balancing Workloads	18
3.3	Improving Graph Locality	19
3.4	An Efficient Version - Corder	20
3.5	Evaluation	22
3.6	A Performance Model for Corder	22
3.7	Experiment Setup	24
3.8	Execution Time	26
3.9	Multicore Activities with Corder	29
3.9.1	Thread Behavior	29
3.9.2	Cache Utility	30
3.9.3	Memory Dynamics	31
3.10	The Derivation of Performance Model	32
3.11	Sensitivity Analysis of Partition Size	34
3.12	Scalability onto Vertex-Centric Paradigm	35
3.13	Reordering Overhead	37
3.14	Summary	39

4	Unequal-Sized Caching	40
4.1	Motivation	40
4.2	Design of Syze	43
4.2.1	Unequal Caching	44
4.2.2	Locating A Subgraph	45
4.2.3	Bit-aware Propagation	47
4.2.4	Program Execution	48
4.2.5	Benefits and Costs	50
4.3	Evaluation	50
4.3.1	Estimating Imbalance	51
4.3.2	Experiment Setup	52
4.3.3	Execution Time	53
4.3.4	Impact on Multicores	55
4.3.5	Decomposition of Syze	59
4.3.6	Initial Segmenting Size	60
4.4	Summary	61
5	Hierarchical Partitioning	62
5.1	Background	62
5.1.1	PageRank	62
5.1.2	NUMA Architecture	63
5.1.3	Partition-Centric Graph Processing	63
5.2	NUMA-Aware Partitioning	64
5.3	Cache-Aware Partitioning	66
5.4	Thread Management	67
5.4.1	Problem: Thread Contention	67
5.4.2	Solution: Thread-data Pinning	68
5.5	NUMA-Aware Partition-Based Data Layout	70
5.6	Evaluation	71
5.7	Experimental Setup	71
5.8	Execution Time	72
5.9	Memory Utility	74
5.10	Scalability	75
5.11	Sensitivity Analysis of Partition Size	76
5.12	Summary	79
6	Reformatting & Mixed Processing	80
6.1	Background and Motivation	80
6.1.1	Compressed Sparse Format	80

6.1.2	Motivation	82
6.1.3	Pull vs Block	82
6.1.4	Computational Redundancy	83
6.2	CSMix	84
6.2.1	Graph Filtering	84
6.2.2	Graph Formatting	85
6.3	Mixen	86
6.3.1	Graph Partitioning and Binning	86
6.3.2	Graph Scheduling and Propagating	88
6.3.3	Benefits and Generalization	89
6.4	Performance Analysis	90
6.5	Evaluation	91
6.5.1	Setup and Implementation Details	91
6.5.2	Execution Time	92
6.5.3	Memory Traffic	94
6.5.4	Block Size	96
6.5.5	Thread Behavior	98
6.6	Related Work	99
6.7	Summary	100
7	Locality Extraction & Blocking	101
7.1	Background	101
7.2	Design of Lebo	102
7.3	Locality Extraction by Reordering	103
7.4	Blocking on The Dense Submatrix	104
7.5	SpMV with Lebo	106
7.5.1	Generic Execution Model	106
7.5.2	Programmability of Lebo	108
7.6	Design Rationale for Lebo	109
7.7	Evaluation	109
7.7.1	Execution Time	110
7.7.2	Decomposition of Lebo	111
7.7.3	Preprocessing Overhead	112
7.7.4	Memory & Cache Utility	113
7.7.5	Size Tuning	114
7.7.6	Scalability	115
7.8	Summary	116

CONTENTS

x

8 Conclusion and Future Work	118
8.1 Conclusion	118
8.2 Future Work	119
References	121

List of Figures

1.1	Interrelations among the research topics in this thesis.	5
2.1	Graphs and their belonging adjacency matrices	8
2.2	Vertex placement by different reordering techniques	10
2.3	A graph and its CSR representation.	12
3.1	Conceptual design of Corder	16
3.2	Vertex placement after applying Corder	21
3.3	The coefficients w_1, w_2, w_3 of performance model in three different cases . .	23
3.4	The correlation between speedup and Locality-Skew ($\lambda = 20\%$).	29
3.5	The impact of Corder on thread behaviors and cache efficiency	30
3.6	The correlation between memory traffic and edge compression	32
3.7	The coefficients of performance models fitting the experimental data of PageRank (a) and all graph applications (b).	33
3.8	The impact of partition size on the execution time	34
3.9	Speedups in different graph processing frameworks	36
4.1	A directed graph and its CSR representation	41
4.2	Parallel efficiency of physical cores and logical cores during the running of PageRank on graph <i>twitter</i>	42
4.3	Abstraction of unequal-sized caching	43
4.4	An example of unequal-sized segmenting	44
4.5	Bit-aware propagation of Syze	47
4.6	The speedup of the graph processing, the longest thread and the average thread over the baseline	55
4.7	The speedups of Syze and baseline and the parallel efficiencies of physical cores and logical cores, by performing PageRank on <i>twitter</i>	57
4.8	The performance decline of the memory, the cache and the average thread compared with the baseline.	58

4.9	The speedups delivered by Syze and its variants in comparison with the baseline, by performing BFS on <i>twitter</i>	59
4.10	The normalized execution time of Syze with different initial segmenting size, by performing BFS.	60
5.1	Conceptual Design of HiPa	64
5.2	Partitioning result of HiPa	67
5.3	2-level hierarchical lookup Table	70
5.4	Scatter via inter-edges and intra-edges	71
5.5	Normalized memory accesses of PageRank in different implementing methodology	74
5.6	Normalized execution time with varied numbers of threads on <i>journal</i>	76
5.7	LLC cache hits and execution time with different partition sizes on graph <i>journal</i>	77
6.1	Directed graph in the compressed formats of CSR, DCSR and TCSR	81
6.2	Compressing and filtering on the adjacency matrix of a graph.	85
6.3	Local DCSRs and bins	87
6.4	Normalized memory traffic. The lower shadowed bars are the reads and the upper bars are the writes.	95
6.5	The execution time with varied block size. The results are normalized by the longest execution time.	96
6.6	LLC hits and memory traffic with different block size on graph <i>pld</i>	97
6.7	Sustained memory bandwidth and speedup (over the 2-threaded result) with different number of threads on graph <i>pld</i>	98
7.1	Concept of Lebo	102
7.2	The reordering method of Lebo	103
7.3	The refinement of columns by Lebo	105
7.4	The contributions of <i>#nnz</i> and of the execution time by the dense and sparse submatrices	112
7.5	Memory traffic volume and L2 cache requests, normalized by the number of edges.	114
7.6	The execution time, L2 cache hits and LLC cache hits on graph <i>wiki</i> , normalized by the smallest results respectively.	115
7.7	The speedup of Lebo with different number of threads over the 1-threaded version on graph <i>wiki</i> , and the corresponding memory bandwidth.	116

List of Tables

1.1	The characteristics of different graph processing systems.	2
2.1	Graph properties in terms of hot vertices percentages (V), edge coverage of hot vertices (E), and average degree (D).	7
3.1	Locality-Skew of graphs with varied λ	17
3.2	Locality-Skew of graphs with varied λ after applying Swap Reorder	19
3.3	Statistic summary of graphs (K: Thousand, M: Million, B: Billion). Inter stands for the inter-edges per subgraph.	25
3.4	Graph processing time of graph applications in seconds after applying different graph reordering methods	26
3.5	Locality-Skew ($\lambda = 20\%$) of graphs after graph reordering	27
3.6	L2 cache misses (in Billions) during 20 iterations of Pagerank after applying different reordering methods	28
3.7	Reordering Overhead in seconds	38
4.1	Statistical summary of GPOP-based PageRank	41
4.2	Graph features in terms of the hot vertices percentages (V), the edge coverage of hot vertices (E), the hot subgraphs percentage (H), and the maximum n_i	51
4.3	Graph Information	53
4.4	Graph processing time for graph algorithms under different graph frameworks	54
4.5	The numbers of subgraphs and inter-edges (in Million), and the time for locating the subgraphs of all vertices (in Milliseconds) by Syze and the baseline	58
5.1	Graph Descriptions	73
5.2	Execution time (in seconds) of PageRank with various implementations.	73
5.3	Normalized execution time using different partition sizes on Haswell and Skylake micro-architectures	78
6.1	Graph Decomposition and the maximum in- and out-degree.	83
6.2	Space requirement for different formats.	86

6.3	Characteristics of machines.	91
6.4	Graph datasets and their attributes (M: Million).	92
6.5	Execution time of PageRank with various implementations	94
7.1	Graph attributes	110
7.2	PageRank execution time per round	110
7.3	Size ratio (%) of dense submatrix to the adjacency matrices	111
7.4	PageRank execution time per round	113

Chapter 1

Introduction

Graphs play a vital role in formulating the research of relationships. Nevertheless, the irregularity of graph-structured workloads impose severe constraints on achieving high-performance graph analytics on multicore systems. Thereby, this thesis is aimed to improve the performance of parallel graph processing by understanding and utilizing the structural property of graphs.

Graph is an abstract data structure that models pairwise relations between entities. It is widely adopted in diverse real-world applications, such as social networks analysis [1], financial fraud detection [2] and transportation planning [3]. With the fast development of the graph-related domains, graph data also continuously grow in size and complexity. Therefore, an urgent need is called for the development of high-performance computing systems for large graphs.

1.1 Graph Frameworks

Based on memory utilization, large graph processing can be categorized into three major classes:

Shared-memory (on single-machine multicore systems) [4, 5, 6, 7, 8, 9, 10]: a graph can suitably fit into the main memory of a powerful multicore machine and globally shared by all cores. It is showcased that even a single server equipped with sufficiently large memory (e.g., 1 Terabyte RAM) is able to process the largest publicly available graph [7]. The performance of shared-memory graph depends mostly on the cache efficiency, memory traffic, and synchronization primitives.

External memory (on out-of-core or disk-based system) [11, 12, 13, 14, 15, 16]: a graph is processed on a resource-limited personal computer (PC), for instance, a 4-core PC with only 16GB memory. The graph datasets cannot be accommodated by the memory; that is, graphs are running out of the cores. Consequently, the disk has to store graph data and

intermediate results during the processing of a graph. Hence, a main design concern for such system is the efficient utilization of disk (e.g., improving disk I/O bandwidth) for data streaming.

Distributed-memory (on multiple-machine distributed systems) [17, 18, 19, 20, 21, 22]: when the graph size is too large (e.g., hundreds of Terabytes) to fit into a single machine, the graph processing scales out to a cluster of distributed machines. Each machine processes a subgraph locally and then synchronize for a global update. Distributed graph processing incurs expensive communication overheads across the network. Hence, it may deliver suboptimal performance and cost efficiency compared with the shared-memory counterpart [19]. Further, a study demonstrates that even an optimized single-thread implementation can outperform many distributed graph frameworks [23].

In general, the characteristics of above systems can be summarized as in Table 1.1:

Table 1.1: The characteristics of different graph processing systems.

Systems	Cost	Speed	Scale
External-memory	Cheap	Slow	Small
Shared-memory	Moderate	Fast	Moderate
Distributed-memory	Expensive	Moderate	Large

All of the systems are designed with a trade-off among performance, efficiency, and scale. Thereby, it is hard to tell which system is the best. The choice of graph systems for deployment should be considered as a good fit for the problem at hand. In addition, there are a variety of graph systems running on heterogeneous architectures (e.g., FPGAs [24, 25, 26] and GPUs [27, 28]), and in "tiny" scale (e.g., domain specific language [29, 30] and cache management [31]) for the purpose of conciseness.

Although dealing with different hardware limits, the design philosophies of these systems are not mutually exclusive. Instead, they often inspire each other. A typical example is the graph partitioning. It is conventionally used for the task allocation of distributed graph processing [19, 20, 18, 32]. In recent years, many shared-memory graph systems also adopts this idea to co-locate graph data and computation for the improvement of cache-memory utilization [5, 8, 10]. Nevertheless, there are also cases where certain graph optimizations are (so far) only applicable or tested for specific hardware platforms, such as the graph reordering for shared-memory systems.

1.2 Challenges of Shared-Memory Graph Processing

The research scope of this thesis is focused on the shared-memory graph processing on the single-machine multicore system due to its advantages in performance and cost efficiency. Without the necessity of offloading the workloads to external memory (i.e., disks) [11, 13] nor distributed memory (i.e., clusters) [33, 19], the shared-memory graph processing avoids the overheads relating to disk I/O and network communication. Thus, it often delivers better performance compared with its external- and distributed-memory counterparts [19, 14].

However, the irregular data structure of graphs still imposes non-trivial challenges for the efficient utilization of multicore systems. The computation of graphs invokes a large volume of communication among the vertices along their edges. This leads to highly randomized memory access pattern [8]. Also, a race condition occurs when multiple edges share a common source/destination vertex. Hence, many frameworks require heavy usage of atomics and mutexes for synchronization [6].

Moreover, real-world graph datasets are typically featured by power-law degree distribution [34]. In a graph, a minority of vertices constitute a large portion of edges. By contrast, a majority of vertices are sparsely connected or even completely isolated with degrees equal to zero. The "hot" vertices (i.e., densely connected ones) raise frequent message exchanges and create substantial workloads. During parallelized graph processing, the "cold" vertices (i.e., sparsely connected ones) have to wait idly for the hot vertices to finish executions. The skewed distribution of vertex degrees not only causes imbalanced workloads for parallel graph processing, but also burdens the synchronization overhead.

Hot vertices randomly spread throughout the whole graph but not evenly. Naturally, they tend to aggregate under high concentration, which we refer to as *graph locality*. The impact of graph locality on cache behaviors is interpreted as cache locality [35, 36, 37, 38]. Characterized by the reference pattern, cache locality can be further classified into two types: *temporal* and *spatial*. A group of vertices shows high temporal locality if frequently accessed. Also, they exhibit high spatial locality if stored in nearby memory allowing for continuous reading and writing.

Besides the cache efficiency, the irregularity of graphs also affects the effectiveness of parallel computing. Some local subgraphs (i.e., subsets of a graph that is partitioned according to certain criteria or algorithms) inside the graph accommodate more vertices or richer connections than others. Therefore, on the one hand, the gathering of hot vertices improves cache locality as frequently accessed data are allocated in consecutive address space; on the other hand, it further exacerbates the workload imbalance amongst computing units [36].

1.3 Research Contributions

To address aforementioned problems, this thesis undertakes an in-depth investigation into the interaction between graph algorithm software and multicore hardware. The graph workloads are characterized not only by graph structural irregularity, but also by the graph traversing pattern, graph encoding scheme. Their impacts on the computing platform are assessed by measuring the utilizations of real hardware in the aspects of cache efficiency, memory traffic, thread balance, multicore parallelism, etc. Based on the understanding of graph workload performance, a locality-oriented methodology is formed for achieving load-balancing, memory- and cache-efficient parallel graph computations.

The main contributions of this thesis are outlined as follows:

Firstly, a novel cache-aware reordering algorithm (Corder) is proposed for workload balances and cache efficiency [39, 40]. At the shared memory level, Corder promotes even distribution of computation-demanding vertices amongst multicores. At the private cache level, Corder facilitates cache efficiency by further concentrating these vertices for the enhancement of cache locality.

Secondly, a unequal caching strategy (Syze) is presented [41]. Syze unequally segments graph components into cache-able subsets of varying sizes. Firstly, the computational loads of cache-sized subgraphs are estimated. Then, the demanding subgraphs are further subdivided until certain threshold is met. Moreover, during the propagation of updates, delicate bitwise operations are performed on vertex ID to facilitate the communication between subgraphs.

Thirdly, a hierarchical partitioning methodology (HiPa) is designed to accelerate PageRank and SpMV [42]. HiPa subdivides the graph based on the NUMA characteristics to reduce remote memory access while ensuring workload balance. For the private cache, HiPa again splits the graph into cache-able partitions to promote in-core computing and cache locality.

Fourthly, a coalescing pair of optimizations is composed: a mixed matrix format CSMix and a graph processing engine Mixen [43]. CSMix perceives a graph from the angle of matrix. The adjacency matrix of a graph is filtered and divided, such that different portions of the matrix are represented in varied formats. Then, the refined matrix is feed to Mixen to eliminated redundant computation. Also, submatrices of different types are processed in respective paradigms.

Lastly, an integration between graph reordering and matrix is achieved, which implements locality extraction & blocking (Lebo) [44]. Lebo exercises lightweight reordering that reorganizes the adjacency matrix of a graph into dense submatrix and sparse one for locality extraction. In every iteration of computation, a hybrid traversal scheme is deployed. Numerous blocks with varying sizes are constructed in the dense portion, and propagate

in pushing flow. Meanwhile, the sparse portion is processed in pulling flow for efficient computation.

The five topics inspire the creation of each other and exhibit many commonalities. Fig. 1.1 illustrates the interrelations among these topics. For instance, Corder and Syze are motivated by the same reason: to balance the workloads, but provide different solutions. HiPa, Mixen and Lebo explore graph access pattern. Moreover, Mixen and Lebo both leverages the duality between graphs and matrices to facilitate graph traversal. In particular, Lebo can be considered as an aggregation of Corder, Syze and Mixen, as it incorporatse the optimization techniques of graph reordering, unequal-sized caching and matrix manipulation.

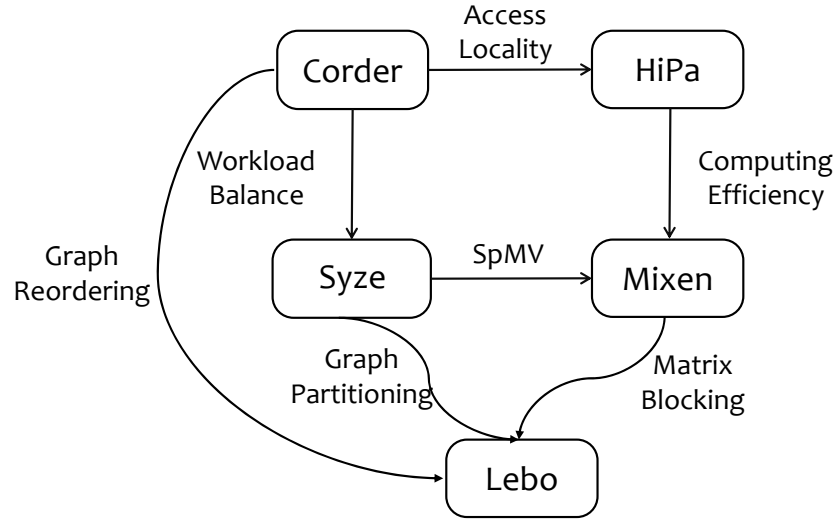


Figure 1.1: Interrelations among the research topics in this thesis.

Two key themes guiding the proposal of the five topics: *access locality* and *workload balance*. The former demonstrates the access efficiency of the graph data loaded in the memory and cache; the later indicates the effective utilization of parallel computing units. Prior works have conducted comprehensive studies (e.g., NUMA- and cache-awareness) on improving the locality of graph access. By contrast, the problem of workload imbalance is oftentimes ignored in many shared-memory graph processing, thereby bottlenecking the parallel computing speed. In this thesis, the proposed works are designed under the principle of bracing high-locality accesses and balanced workloads.

The goal of this thesis is to provide in-depth understanding towards the irregular graph workloads processed on modern multicore systems. The gained insights into the structural property of graphs and the architectural feature of multicores are utilized to conceptualize and develop optimization strategies for accelerating graph computations.

Thesis Organization. Following the introduction, **Chapter 2** discusses preliminary knowledge about graphs and state-of-the-art research related to graph processing. **Chapter**

3 presents Corder, which serves as a preprocessing step to alleviate workload imbalance for graph analytics. **Chapter 4** describes Syze, a shared-memory framework aggressively caching graphs to achieve high performance. **Chapter 5** offers HiPa that leverages memory hierarchy for the optimization of SpMV-based graph computations. **Chapter 6** introduces Mixen and CSMix, a pair of optimizations that adopt mixed formats for graph representation and mixed traversal schemes. **Chapter 7** shows Lebo, which promotes efficient SpMVs on graph adjacency matrices. The final **Chapter 8** summarizes important facts and results throughout this thesis. In the end, future work is proposed.

Chapter 2

Background and Related Work

2.1 Graph Basics

Skewed Degree Distribution. It is rare to see that a real-world graph that is not skewed with respect to its degree distribution[45, 34]. The skewed degree distribution, following the power law, means that a small fraction of vertices are responsible for a major fraction of edges. Mathematically, the portion of vertices have k connections to other vertices can be formulated as $p(k) \sim k^{-\lambda}$, where λ is a parameter empirically ranging in $[2, 3]$. Such inequality can be differently interpreted in various contexts. For instance, a celebrity has enormous social influence in social network graphs, or a search engine website sends billions of requests to other websites on web graphs.

Table 2.1: Graph properties in terms of hot vertices percentages (V), edge coverage of hot vertices (E), and average degree (D).

Graphs	Descriptions	V (%)	E (%)	D
<i>urand</i>	Synthetic Graph[46]	51	59	32
<i>kron</i>	Synthetic Graph[46]	8	93	31
<i>pld</i>	Pay-Level-Domain[47]	12	88	14
<i>live</i>	Live Journal[48]	24	80	14
<i>wiki</i>	Wiki Links[49]	19	94	9
<i>twitter</i>	Twitter Follower[50]	9	79	35
<i>mpi</i>	Twitter Influence[51]	11	81	38

Table 2.1 lists the percentage of hot vertices and their edge coverage across various graphs. Here, the out-degree is used as the default measurement for the degree of a vertex. Vertices with degrees higher than the average degree of the graph is considered as the hot vertices, while the rest of vertices in the graph are the cold ones. In skewed graphs, 8-24

percent hot vertices compose 79-94 percent edges. Graph *urand* is not skewed, since over half of vertices in it are classified as hot.

Graph Locality. The placement of graph vertices and the relationship among them pose non-negligible impact on graph locality. Consider the examples in Fig. 2.1 and assume the vertices are stored in an array in the order of vertex ID. We suppose that a graph algorithm examines the relationship between every vertex and its neighbors in sequence. Our goal is to observe the access pattern relating to vertices 0 and 1. In the random graph of Fig. 2.1a, vertex 0 and its neighboring vertices 2 and 5 are firstly evaluated, then followed by the vertex 1 and its adjacent vertices 3 and 4. Therefore, the access pattern is 0-2-5-1-3-4, where each vertex is visited only once (i.e., poor temporal locality), and vertices are not co-located in the array (i.e., poor spatial locality).

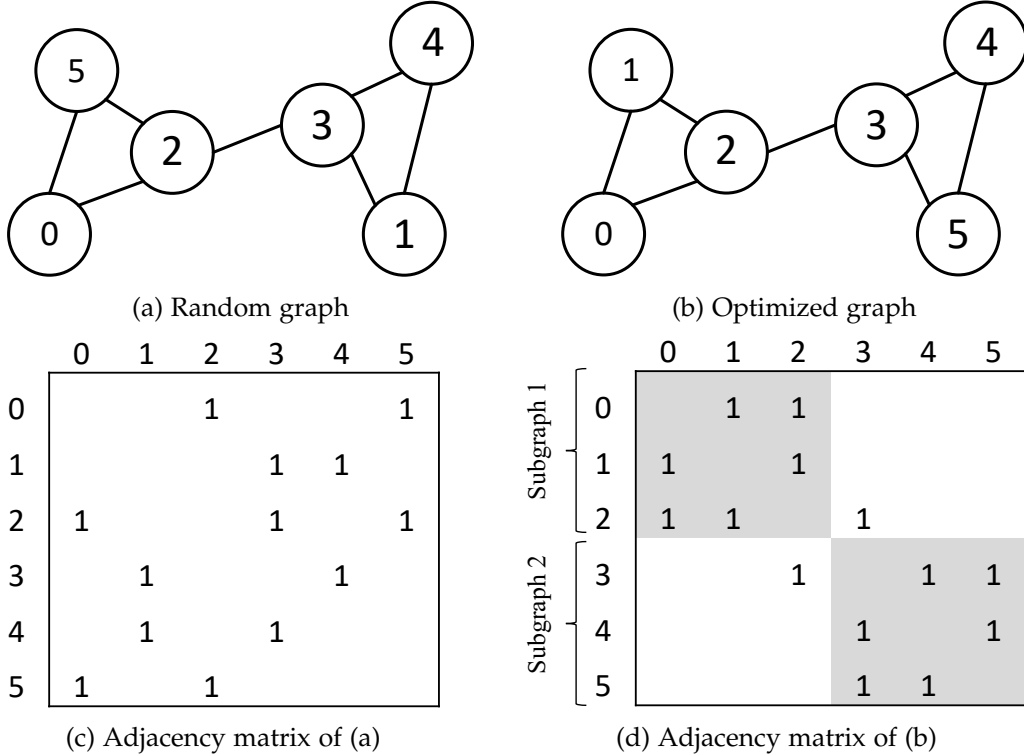


Figure 2.1: Graphs and their belonging adjacency matrices. Graph (b) is optimized from graph (a) by graph reordering. In graph (b), vertices in each subgraph are assigned with continuous ID numbers, which are depicted as shadow-colored diagonal blocks in matrix (d).

The graph in Fig. 2.1b is optimized on the vertex order, such that nearby vertices are labeled with close ID number. As a result, the access pattern corresponding to vertices 0 and 1 becomes 0-1-2-1-0-2 in the reordered graph. This pattern achieves good temporal-spatial

locality, because vertices 0, 1, 2 are *repeatedly* accessed in *continuous* memory space.

The graph locality can also be directly observed from the adjacency matrix. Fig. 2.1c presents the adjacency matrix of the random graph. The elements of the matrix, representing the connections of vertices, are sparsely distributed. By contrast, the reordered adjacency matrix in Fig. 2.1d, containing two dense diagonal blocks, exhibits better graph locality.

2.2 Graph Reordering

The random and sparse distribution of hot vertices in memory incurs severe underutilization of caches. To improving cache efficiency, prior works have proposed various graph reordering methods [38, 52, 35, 53]. These aim to search for an (approximately) optimal permutation of vertex order to improve graph locality, such that frequently accessed vertices are placed together. The examples of state-of-the-art graph reordering methods are described as follows:

Gorder [38] offers significant performance boost to graph applications by leveraging the internal structure of graphs. It undertakes a sophisticated analysis of the connectivity of vertices. Consecutive IDs are relabeled to vertices that share common neighbors. Hence, these neighboring data are reused in cache when vertices are consecutively processed. Nevertheless, *Gorder* tends to cost *hours* to accomplish the reordering procedure, while the graph applications are typically executed within minutes or even seconds. For this reason, it is impractical to deploy *Gorder*, especially when the graph is constantly changing.

Gorder can be considered as a representative example of heavyweight reordering methods that require prohibitively high overhead [38, 54, 55, 56]. Calling for pragmatism, lightweight degree-based reordering methods with low computation complexity are proposed [52, 35, 53]:

Sorting places all vertices in descending order of degree. It is the most straightforward method for degree-based reordering. However, as illustrated in Fig. 2.2, the original graph structure is entirely reorganized, even including the relative order of the vertices. Moreover, sorting leads to extremely imbalanced workloads, imposing a considerable adverse effect on the overall performance of parallel graph processing.

Frequency Based Clustering (FBC) [52], also referred to as Hub Sorting, only applies sorting to hot vertices, while keeping remaining cold vertices unsorted. FBC improves the graph locality at the cost of fully rearranging the structure of hot vertices. Nonetheless, it still incurs high imbalance due to the sorting of hot vertices

Hub Clustering (HC) [35] is a variation of FBC. It separates hot and cold vertices into two memory spaces. HC does not employ sorting to any vertices. Therefore, the relative orders of hot vertices and cold vertices are maintained in their respective segments. HC reaches an optimal compromise between workload balance and graph locality.

Degree Based Grouping (DBG) [53] employs coarse-grain sorting to the graph. Vertices are allocated into different groups with exclusive degree ranges: $[0, \frac{\bar{D}}{2})$ and $[2^{n-1}\bar{D}, 2^n\bar{D})$, where $n = 0, 1, 2, \dots$ and $\bar{D}$ is the average degree. Then, these groups are sorted in descending order. To preserve graph structure, the relative order of vertices inside each group remains intact. DBG can be viewed as an intermediate between Sorting and HC.

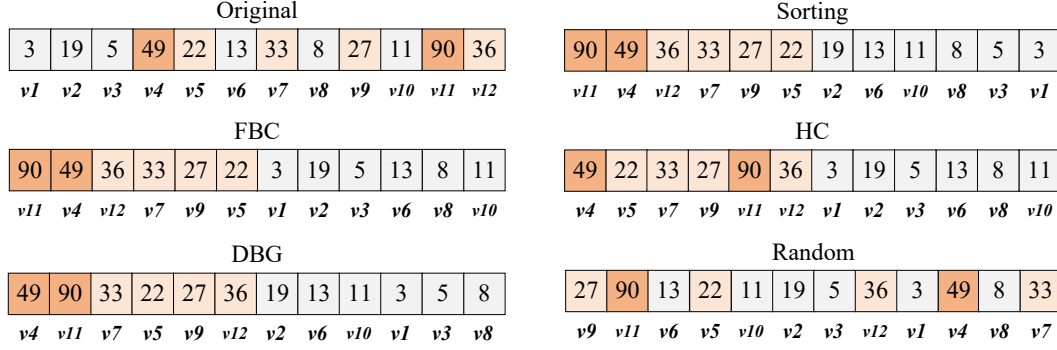


Figure 2.2: Vertex placement by different reordering techniques. The degrees of vertices are filled inside the box. Cold vertices (degree ≤ 20) are painted with grey, hot vertices are colored, and very hot vertices (degree > 40) are depicted with darker shade. Indexes below the box are the original vertex IDs.

Sorting places all vertices in descending order of degree. It is the most straightforward method for degree-based reordering. However, as illustrated in Fig. 2.2, the original graph structure is entirely reorganized, even including the relative order of the vertices. Moreover, sorting leads to extremely imbalanced workloads, imposing a considerable adverse effect on the overall performance of parallel graph processing.

Frequency Based Clustering (FBC) [52], also referred to as Hub Sorting, only applies sorting to hot vertices, while keeping remaining cold vertices unsorted. FBC improves the graph locality at the cost of fully rearranging the structure of hot vertices. Nonetheless, it still incurs high imbalance due to the sorting of hot vertices.

Hub Clustering (HC) [35] is a variation of FBC. It separates hot and cold vertices into two memory spaces. HC does not employ sorting to any vertices. Therefore, the relative orders of hot vertices and cold vertices are maintained in their respective segments. HC reaches an optimal compromise between workload balance and graph locality.

Degree Based Grouping (DBG) [53] employs coarse-grain sorting to the graph. Vertices are allocated into different groups with exclusive degree ranges: $[0, \frac{\bar{D}}{2})$ and $[2^{n-1}\bar{D}, 2^n\bar{D})$, where $n = 0, 1, 2, \dots$ and $\bar{D}$ is the average degree. Then, these groups are sorted in descending order. To preserve graph structure, the relative order of vertices inside each group remains intact. DBG can be viewed as an intermediate between Sorting and HC, leading to relatively high workload imbalance and also graph locality.

Fig. 2.2 visualizes the degree-based vertex placement policy. In addition, the random reordering (RND) is used as a benchmark for later analysis. RND shuffles a graph, therefore completely eliminating the graph locality and structure. On the other hand, it generates a uniformly distributed random permutation of vertices, which yields a uniform distribution of workloads.

However, graph reordering does not necessarily guarantee performance boost. In de facto, it often leads to a slowdown if an improper strategy is deployed [35]. The gathering of hot vertices aggravates the imbalanced distribution of computation load [52], forcing the main process to wait for the longest thread to synchronize. Moreover, reordering techniques inevitably disrupt the internal structure of graphs [53], such as communities of common friends in social graphs. The finer-grained the reordering is, the more structural information it loses.

2.3 Graph Partitioning

The partitioning of a graph can be considered equivalent to the task allocation of a graph parallel computing system. There exist two essential types of lightweight partitioning schemes: vertex-based and edge-based. In the former case, vertices are assigned to disjoint segments. It is an intuitive method of partitioning and adopted in early graph frameworks such as GraphLab [57] and Pregel [17]. As for the latter, edges of a vertices are detached to different segments, and the vertex is accordingly replicated. This scheme gains its popularity in edge-centric frameworks, such as GridGraph [11] and X-stream [12]. For better performance, hybrid partitioning approaches are proposed to take advantages of the two, for example, PowerLyr [58] and CUBE [20]. The partitioning methodology proposed in our work falls into the hybrid category.

Moreover, numerous sophisticated partitioning algorithms are developed in order to obtain high-quality results, for example METIS [59, 60], KaHIP [61] and PULP [62]. Those algorithms extensively analyze the internal structure of the graphs. High-quality graph partitioning facilitates workload balance for parallel computation and reduces the communication volume between working units. As a result, it leads to a substantial performance boost for downstream applications [63].

Nevertheless, compared with the lightweight counterpart, sophisticated partitioning methods incur high overhead and complicated implementation procedure. To minimize the preprocessing time, high-performance partitioning algorithms are designed, such as ParMETIS [64], ParHip [65] and XtraPULP [66]. In these partitioners, various optimization techniques are exploited, for instance, hierarchical partitioning [65] and distributed parallelism [66].

2.4 SpMV on Graphs

SpMV has been drawing enormous research attention over the past decades for diverse computing architectures, including CPUs [67, 68, 69, 70] and GPUs [71, 72, 73, 74]. The essential challenge of SpMV arises from the massive number of zeros in the sparse matrix, which are not engaged in the computation. Hence, the matrices are commonly represented in compressed formats that contain only the non-zeros [75, 76].

However, there is no single format universally suitable for all SpMV algorithms on all architectures. The selection of matrix representation depends on the specific tasks. For instance, when the non-zeros are well structured and form a few matrix diagonals, Diagonal (DIA) is a favored choice [77]. Ellpack (ELL), allowing a general pattern of matrices, suits for regular SIMD programming [78]. For the irregular matrices, Coordinate List (COO) copes with the problem of load imbalance [79]. Furthermore, Hybrid (HYB) approach is devised by decomposing the matrix into two parts of regular ELL and irregular COO [80].

A graph can be mathematically represented as an adjacency matrix $\mathbf{A}$, where $a_{i,j}$ stands for an edge from vertex v_i to v_j . If $a_{i,j} = 0$, there is no edge between the two vertices. The i^{th} row stores all outgoing-edges of v_i , whereas the j^{th} column records all incoming-edges of v_j . Leveraging the duality between graphs and matrices, graph algorithms are oftentimes recast as Sparse Matrix-Vector multiplication (SpMV) for optimizations [81, 82, 83]. In reverse, graph algorithms, such as PageRank and BFS, oftentimes serve as the representative SpMV applications in SpMV papers [84].

In a large-scale power-law graph, there could be more than millions of vertices, but most of them are connected with only a few neighbors. In other words, the adjacency matrix $\mathbf{A}$ of the graph is highly sparse, since a massive number of elements are zero. For efficient storage, compact representation of $\mathbf{A}$ is needed. The popular choices are Compressed Sparse Rows (CSR) and its transpose Compressed Sparse Columns (CSC) [81].

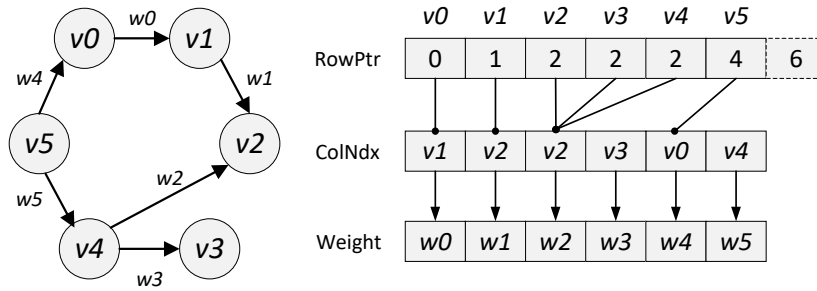


Figure 2.3: A graph and its CSR representation.

Fig. 2.3 illustrates the adjacency matrix of a graph and its CSR representation. CSR consists of two essential arrays: rowPtr and colNdx. For every row of matrix $\mathbf{A}$, the rowPtr

array points to the position of its first non-zero element in the `colNdx` array. The `colNdx` array stores the column indices of all non-zeros. If the graph is weighted, an optional array `val` records the values of all non-zeros (i.e., edge weights); otherwise, the $\mathbf{A}$ is a binary matrix. Due to the limited space, the explanation regarding CSC is omitted.

CSR consecutively stores the non-zero elements of a matrix row by row. Thus, it is well suited to perform the row-major matrix traversal (i.e., graph pushing). Likewise, CSC places the non-zeros of matrix columns in contiguous memory space, thus fitting for column-major matrix traversal (i.e., graphs pulling).

Consider a naive case of graph traversal, where each vertex iteratively updates its new data by accumulating all old data from its in-neighbors. The traversal can be converted to a SpMV problem: $\mathbf{y} = \mathbf{A}^T \mathbf{x}$, where $\mathbf{x}$ and $\mathbf{y}$ are dense input (i.e., old data) and output (i.e., new data) vectors of size n , and T denotes matrix transpose. The computation for every element of $\mathbf{y}$ (with the j^{th} column of $\mathbf{A}$) is: $y_j = \sum_{i=0}^{n-1} a_{i,j} \cdot x_i$.

Algorithm 1 SpMV in pulling flow based on CSC

```

1: for all  $i=0$  to  $n-1$  do in parallel
2:    $y[i]=0$ 
3:   for  $j=\text{colPtr}[i]$  to  $\text{colPtr}[i+1]-1$  do
4:      $y[i] += \text{val}[j] \cdot x[\text{rowNdx}[j]]$ 

```

The SpMV per iteration can be expressed by Algorithm 1 with the matrix $\mathbf{A}$ encoded in CSC. The matrix $\mathbf{A}$ is executed in column-major order (and $\mathbf{A}^T$ in row-major order); that is, the graph propagates in pulling direction. As at line 3, the random memory accesses results from the random reads to $\mathbf{x}$ caused by `rowNdx[j]`.

Algorithm 2 SpMV in pushing flow based on CSR

```

1:  $y[:] = 0$ 
2: for all  $i=0$  to  $n-1$  do in parallel
3:   for  $j=\text{rowPtr}[i]$  to  $\text{rowPtr}[i+1]-1$  do
4:      $\text{atomAdd}(y[\text{colNdx}[j]], \text{val}[j] \cdot x[i])$ 

```

The SpMV can also iterates over the matrix $\mathbf{A}$ row by row as outlined in Algorithm 2. In this scenario, graph is processed in pushing flow. As depicted at line 3, `colNdx[j]` invokes random writes to the output array $\mathbf{y}$. Unlike in the pulling flow, the element of $\mathbf{y}$ for threads to write is not unique. Hence, the access to $\mathbf{y}$ must be safeguarded by atomic instruction `atomAdd(x1, x2)` to prevent multiple threads from writing to the same destination.

Without the overhead of write protection, pulling flow is the preferred option for link-intensive graph processing, where a large fraction of vertices and edges are visited in every

iteration. It backbones a diversity of graph analytics algorithms, such as PageRank [85], Belief Propagation [86], Community Detection [87], and graph machine learning, including Graph Convolutional Network [88] and its variants [89].

Chapter 3

Cache-Aware Reordering

To utilize graph locality and improve cache efficiency, a variety of lightweight graph reordering methods have been proposed [35, 53, 52]. These approaches, in principle, reorder vertices such that frequently accessed vertices are allocated in contiguous memory space. As a result, they have higher chances to be retained within the cache lines, which allows for better cache utilization. In other words, prior reordering methods mainly focus on enhancing the graph locality for performance boost by concentrating hot/neighbor vertices.

However, the applicability of these methods is seriously limited. They are effective only in a particular framework. In most cases, the performance of downstream graph applications is deteriorated due to imbalanced workloads caused by the concentration of hot vertices. To overcome aforementioned limitations, we propose *Cache-aware Reorder*, namely *Corder*, a lightweight reordering method based on the characteristics of cache [39, 40]. For the design of *Corder*, we firstly dig deeper into the property of graphs in the following section.

3.1 Estimating Imbalance

Despite the improvement in graph locality, the aggregation of hot vertices aggravates the uneven placement of hot vertices among subgraphs, and thus exacerbates load imbalance in a multicore system. To quantify the skewed distribution of *graph locality* incurred by concentration of vertices, we define the term "Locality-Skew".

To the best of our knowledge, there is no means to quantitatively measure the graph locality [36, 35]. The closest analogy is provided by *Gorder*, where the locality of two vertices is calculated for co-placement [38]. The extensive calculation of vertex pairs throughout the entire graph causes an excessively high computational cost (see Table 3.7). Hence, our primary task is to provide an efficient measurement approach for the graph locality. Given that the graph locality is mainly determined by the *connectivity* among the vertices in local *subgraphs*, the measurement task could be decomposed into two steps: (1) partition the

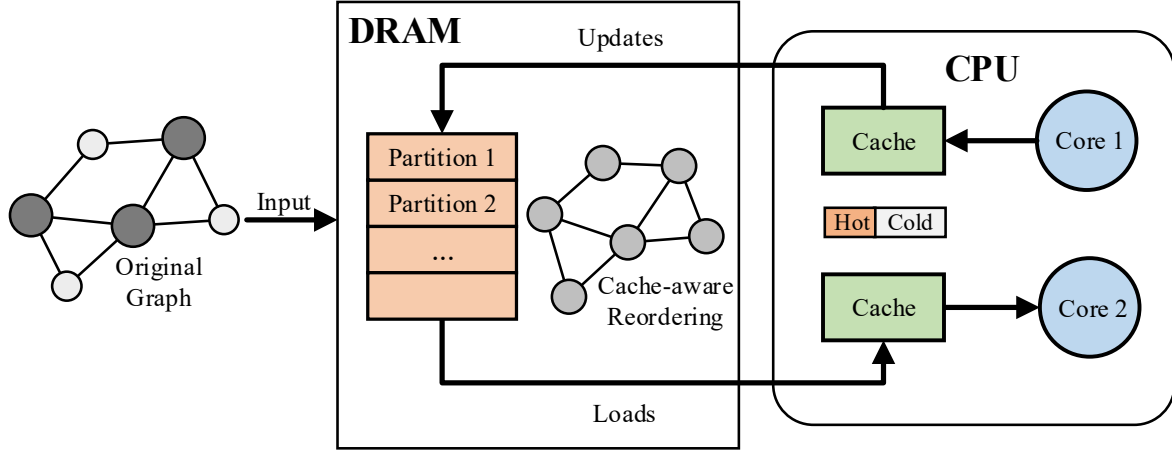


Figure 3.1: Conceptual design of Corder. Grey cycles represent subgraphs. Different grey colors and sizes indicate imbalanced workloads. Corder reorders the graph such that the vertex subset of each subgraph can fit into the private caches of multicores with fairly balanced loads. Within the cache, hot vertices and cold ones are segmented to improve cache locality.

graph, and (2) approximate the connectivity.

We perform the partitioning based on the characteristic of cache hierarchy. For a modern multicore machine, its cache is typically comprised of three level hierarchy according to the speed/storage. Level-1 (L1) cache operates at the highest accessing speed but with the smallest storage capacity. Level-2 (L2) cache holds the medium-size storage by sacrificing the speed. Last Level cache (LLC) offers the lowest speed but the largest capacity. Every core is assigned with its own private L1 and L2 caches, whilst the LLC is shared between cores. According to these features, we split the vertex set of a graph into numerous partitions, the size of which is fixed to L2 cache size. Hence, each subgraph along its vertex array can be fitted in the private L2 cache of one core. The design choice of L2 cache achieves a balance between speed and storage.

Since the size of a vertex subset is predetermined, the graph locality now depends on the connectivity of vertices inside the subgraph. Here, we simply approximate the connectivity of a subgraph by accumulating the degrees of all vertices inside it. This is because, on one hand, the degree represents the number of edges one vertex is connected to. Vertices with higher degrees are likely to involve heavier computation. Thus, subgraphs with larger sum of degrees tend to associate with larger workloads. On the other hand, the summation of degrees is a simple arithmetic operation that requires no in-depth analysis, e.g., leveraging the internal structure of graphs.

In concise words, we evaluate the graph locality by *the cumulative total of vertex degrees in a subgraph with a fixed size of vertex array equal to L2 cache*. Furthermore, the localities (i.e.,

Table 3.1: Locality-Skew of graphs with varied λ . The larger a value, the more imbalanced a graph

λ	1%	10%	20%	30%	40%	50%
<i>urand</i>	1.00	1.00	1.00	1.00	1.00	1.00
<i>kron</i>	1.20	1.13	1.10	1.08	1.07	1.06
<i>pld</i>	2.83	1.86	1.61	1.46	1.36	1.29
<i>live</i>	25.89	21.24	13.19	9.34	6.54	4.48
<i>wiki</i>	39.12	17.60	12.04	9.03	7.17	5.69
<i>twitter</i>	192.12	30.61	15.56	10.47	7.90	6.35
<i>mpi</i>	432.43	56.12	22.69	12.82	8.21	5.93

total sum of degrees) of different subgraphs vary with the skewed distribution of degrees. To measure the variation, we thereby introduce "Locality-Skew".

The calculation of Locality-Skew consists of three steps. Firstly, the locality of each subgraph is obtained. Next, subgraphs are sorted by their localities in descending order. Therefore, the subgraphs at the top are loaded with much higher localities than these at the end. Last, the total locality of top λ percent subgraphs is divided by the counterpart of end λ percent to compute Locality-Skew:

$$\text{Locality-Skew} = \frac{\sum_{s \in \text{top}_\lambda} L(s)}{\sum_{s \in \text{end}_\lambda} L(s)} \quad (3.1)$$

In Eq. 3.1, s is the sorted subgraphs, $L(\cdot)$ stands for the locality of a subgraph (i.e., the sum of degrees). Table 3.1 presents the measurement of Locality-Skew with varied λ (denoted as $L\text{-}S_\lambda$) across different graphs. *urand* is a perfectly unskewed and balanced graph. *kron* and *pld* are slightly imbalanced. The imbalances of *live* and *wiki* become severer as their L-S grow higher. As for *twitter* and *mpi*, hot vertices are concentrated in the 1 percent subgraphs, exhibiting extremely unequal distribution.

By absorbing the knowledge of graph locality and skewed degree distribution together, Locality-Skew offers us a new perspective to observe the features of a graph. For instance, the conventional measurement in Table 2.1 indicates *kron* is highly skewed, because 8 percent vertices compose 93 percent edges in this graph. Nevertheless, as listed in Table 3.1, its $L\text{-}S_{1\%} = 1.20$ implies that it is, in fact, fairly balanced.

It should be emphasized that we aim to provide an efficient estimation regarding the graph property rather than a precise solution. Researchers have devoted much effort to designing fine-grained algorithms for the exploration of graph features, including graph reordering [38], graph partitioning [90], subgraph finding [91] and community detection

[92]. However, the complexity of these algorithms prohibits their deployments in the context of "lightweight". Although our approaches, such as partitioning a graph by the L2 cache and sorting subgraphs instead of vertices, are rather coarse-grained, they considerably simplify implementation process and reduce computation complexity. Moreover, The estimation results clearly reveal the hidden property of graphs, and inspires a new direction in optimizing graph analytics: workload balance.

3.2 Balancing Workloads

The summation of vertex degrees allows us to approximate the locality of L2-cache-sized subgraphs. More importantly, we are able to estimate the imbalance of workloads among these subgraphs based on Locality-Skew. To address the issue of imbalance, we develop a reordering strategy that swaps the hot vertices out of the overloaded subgraph.

Algorithm 3 Swap Reorder

Input: graph

Output: subgraphs

```

1: subgraphs[Nsub] ← split(graph, Nsub)
2: for all sub ∈ subgraphs do in parallel
3:   /* is current sub overflowed with hot vertices? */
4:   if isOverflow(sub) then
5:     /* swap hot vertices for cold ones with next subgraph */
6:     swap(sub.hotVtx, sub.next.coldVtx)
7:   else
8:     swap(sub.coldVtx, sub.next.hotVtx)
```

Algorithm 3 generalizes the procedure of Swap Reorder, a swap-based reordering method. The initial step in line 1 is to split the graph into partitions in accordance with the L2 cache. The number of subgraphs (i.e., partitions) is determined by the number of vertices N_v in the graph and the size of L2 cache: $N_{sub} = N_v / \text{Size}(L2)$. Then, based on the percentages of hot vertices P_h , we can calculate the total number of hot vertices in a graph dataset: $N_h = P_h \cdot N_v$. To equate the workloads, hot vertices are reallocated, such that each subgraph contains $n = N_h / N_{sub}$ hot vertices. If the number of hot vertices in a subgraph exceeds the threshold n , this subgraph is considered as overflowed (e.g., line 4). Once a subgraph is overflowed, its extra hot vertices will be swapped for cold ones with the next subgraph (e.g., line 6), and vice versa. The vertices to be modified are decided on a first-come-first-served basis, so their relative order is preserved.

Table 3.2 lists the results of Locality-Skew after swapping. The L-S of *urand* remains unchanged as 1.00, since it is already well balanced. The imbalance of *kron* is slightly enlarged as its L-S_{1%} increases from 1.20 to 1.23. The rest of graphs achieve a substantial

Table 3.2: Locality-Skew of graphs with varied λ after applying Swap Reorder. The larger a value, the more imbalanced a graph

λ	1%	10%	20%	30%	40%	50%
<i>urand</i>	1.00	1.00	1.00	1.00	1.00	1.00
<i>kron</i>	1.23	1.14	1.11	1.09	1.07	1.06
<i>pld</i>	2.54	1.67	1.48	1.37	1.30	1.25
<i>live</i>	2.98	2.69	2.40	2.13	1.92	1.69
<i>wiki</i>	6.27	3.00	2.37	2.01	1.78	1.61
<i>twitter</i>	63.79	14.04	7.81	5.56	4.34	3.57
<i>mpi</i>	21.53	6.85	4.43	3.38	2.79	2.40

reduction in L-S, such as the L-S_{1%} of *mpi* declining from 432.43 to 21.53.

3.3 Improving Graph Locality

Enlightened by aforementioned degree-based reordering techniques, we spot an opportunity for further optimization in graph locality. We adapt HC, which globally manipulates the vertex order at the level of main memory, to the level of local L2 cache. More specifically, after the deployment of Swap Reorder, the vertex order inside each subgraph within the L2 cache is further refined by classifying hot and cold vertices into two segments. As presented in Algorithm 4, it is simply just to append `hubCluster` to `swapReorder`. From the memory-cache hierarchy point of view, the workload is balanced in the main memory, and then the graph locality is improved in the private L2 cache.

Algorithm 4 Swap Reorder + Local HC

Input: graph

Output: subgraphs

```

1: subgraphs = swapReorder(graph)
2: /* local vertex reordering */
3: for all do in parallel sub ∈ subgraphs
4:   hubCluster(sub)

```

However, a drawback of Swap Reorder is the interdependence of subgraphs, which leads to high computational complexity. To preserve the relative order, the swapping of vertices is incurred by every subgraph with the next one in a sequential order. This prohibits the swapping process from being parallelized. Considering the best case for Algorithm 3, the vertex distribution of a graph is so balanced that every subgraph contains exactly n hot vertices. No subgraph is overflowed nor underflowed; hence, vertices are counted but never swapped. As a result, the time complexity of Algorithm 3 is $O(N_v)$. Then, in Algorithm 4,

the local HC that classifies vertices into hot or cold vertices allows for parallelism. Its time complexity is $O(N_v/N_t)$, where N_t is the number of threads. Adding these together, the time complexity of Algorithm 4 is $O(N_v + N_v/N_t)$ in the best case.

3.4 An Efficient Version - Corder

The sequential execution of Swap Reorder underutilizes the capacity of the multicore system. It induces high overhead costs and challenges the practical deployment of Algorithm 4. To parallelize the reordering procedure, an efficient version is thereby needed for pragmatism. Based on previous discussions, we organize the essential ideas of Swap Reorder plus local HC as follows:

1. The vertex set of a graph is subdivided into cache-able disjoint partitions of size equivalent to the L2 cache.
2. Each subgraph contains the same ratio of hot/cold vertices as the original graph.
3. Hot and cold vertices are segregated into two segments inside the subgraph.
4. The relative order of vertices in the hot segment as well as the cold one is preserved.

Accordingly, we propose *Cache-aware Reorder (Corder)*, an efficient and effective algorithm fulfilling these ideas. The programming steps of Corder are outlined in Algorithm 5. It consists of two phases: *collecting* and *distributing*. In the collecting phase of lines 2-6, hot vertices and cold vertices are respectively collected into two distinct vectors. In the distributing phase of lines 10-12, n hot vertices and $\bar{n}$ cold vertices are distributed from the vectors to all subgraphs, where $n = P_h \cdot N/N_{sub}$ and $\bar{n} = (1 - P_h) \cdot N/N_{sub}$.

Algorithm 5 Corder

Input: graph

Output: subgraphs

```

1: /* collect phase */
2: for all vertex  $\in$  graph.vertices do in parallel
3:   if isHot(vertex) then
4:     hotVertices.add(vertex)
5:   else
6:     coldVertices.add(vertex)
7: /* distribute phase */
8: for all sub  $\in$  subgraphs do in parallel
9:   move  $n$  vertices from hotVertices to sub
10:  move  $\bar{n}$  vertices from coldVertices to sub

```

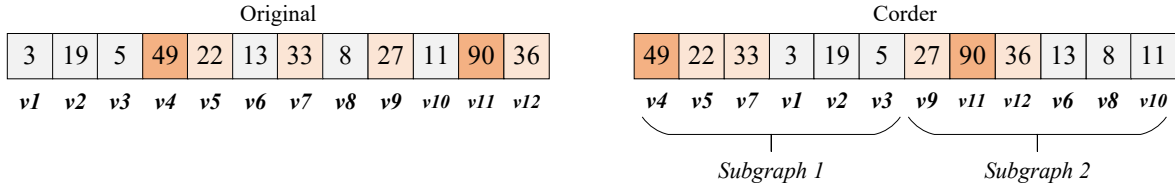


Figure 3.2: Vertex placement after applying Corder. Assuming a private cache can hold six vertices, the graph of twelve vertices is partitioned into two subgraphs. In each subgraph, hot vertices (colored) are placed in the front of the cold ones (grey)

Compared with Algorithm 3 and 4, Corder exhibits its advantages in four aspects. First of all, the conditional statement `isHot(vertex)` of line 3 in Algorithm 5 is significantly simpler than the `isOverflow(sub)` of line 4 in Algorithm 3. The former function verifies whether a single vertex is hot by simply comparing its degree with the average degree. By contrast, the latter examines whether a subgraph is overflowed by checking all included vertices. This demands additional memory to record overflowing vertices. Secondly, distributing hot/cold vertices over all subgraphs (e.g., lines 11 and 12 in Algorithm 5) is easier than swapping them between subgraphs (e.g., lines 6 and 8 in Algorithm 3). The distributing operation is nothing but to concatenate two subvectors, which is independent for every subgraph. Contrariwise, the swapping operation requires current subgraph to interact with the next subgraph(s) for vertex exchanging. Thirdly, the concatenation of hot and cold vertices (e.g., lines 11 and 12 in Algorithm 5) coincides with the completion of local HC (e.g., lines 3 and 4 in Algorithm 4). Therefore, local vertex reordering is not needed anymore. Last but not the least, the simplicity of Corder allows both *collect* and *distribute* phases to be parallelized, thereby significantly reducing overall reordering time.

The time complexity of the collecting phase and the distributing phase in Algorithm 5 are both $O(N_v/N_t)$. Therefore, the time complexity of Corder is $O(N_v/N_t + N_v/N_t) = O(N_v/N_t)$. It performs significantly faster than Algorithm 4, the best-case result of which is $O(N_v + N_v/N_t)$.

The vertex replacement result of Corder is exemplified in Fig. 3.2. A graph is partitioned into two subgraphs. Each subgraph contains the same number of hot vertices. Hence, the workload amongst two subgraphs (and therefore two processing cores) are roughly comparable. Inside the subgraph, hot vertices and cold vertices are segregated into two different memory locations. The grouping of hot vertices facilitates cache locality. At the meantime, the relative orders of hot vertices and cold ones are respectively maintained as original, so that the graph structure is preserved.

We expect that Corder can boost the performance of skewed graphs, e.g., all graphs except *urand*. In particular, the more "imbalanced" (i.e., higher Locality-Skew) a graph is, the better improvement Corder will lead to.

3.5 Evaluation

The performance of Corder is evaluated based on both theoretical and experimental approaches. First, a performance model for Corder is constructed, which incorporates multicore activities in diverse aspects. Then, we compare Corder with with state-of-the-art lightweight reordering methods. The behaviors of the multicore system are analyzed under the guideline of the proposed model. Besides, extensive experiments are undertaken to investigate the optimal partition size of Corder as well as its scalability onto vertex-centric paradigm.

3.6 A Performance Model for Corder

In this section, a theoretical evaluation is conducted to profile the correlation between Corder and various hardware activities in multicore systems. As previously described, Corder directly introduces two factors into the performance variability of graph processing on multicore systems: workload balance and cache utility. Besides, it might also raise memory traffic. This is because Corder balances the workload by evenly distributing hot vertices among partitions in different cores, thus elevating the inter-partition communication through the memory (verified in Section 3.9.3).

In the partition-centric paradigm, the number of inter-edges after compression poses substantial impact on the memory traffic (i.e., read + store) volume [93]. Since the access to memory is slower than the access to cache and CPU computation by orders of magnitude, it is considered as one of the most important factor in deciding the execution time of partition-centric graph processing [94]. Therefore, for a comprehensive analysis of underlying multicore systems, the memory dynamics is also taken into account as an indirect factor brought by Corder.

Thereby, the multicore activities are examined in three aspects: workload imbalance, cache utility, and memory dynamics. Particularly, we characterize the workload imbalance among multicores by the *the longest thread execution time* T , which signifies the most imbalanced load in a multi-threaded task and bottlenecks the parallel computing. The cache utility is represented by the *L2 cache misses* C , as the cache efficiency is improved by Corder within the L2 caches of each processor core. Also, following prior works [93, 94], the memory dynamics is featured by *memory accesses volume* M . A simple linear model, which quantitatively portrays the contributions from the three factors, is proposed as follows:

$$speedup \propto w_1 \cdot r(T) + w_2 \cdot r(C) + w_3 \cdot r(M) \quad (3.2)$$

Function $r(\cdot)$ stands for the ratio of original result to the reordered result; that is, it

accounts for the speedup of threads $r(T)$, the efficiency of cache $r(C)$ or the efficiency of memory $r(M)$ respectively. For example, $r(T) = \{\text{original thread execution time}\} / \{\text{thread execution time after reordering}\}$. Thus, the model in Eq. 3.2 defines that the speedup of overall graph processing is proportional to the sum of improvements in thread performance, cache and memory efficiencies. Coefficients w_1 , w_2 and w_3 express the weights of their corresponding factors in this model.

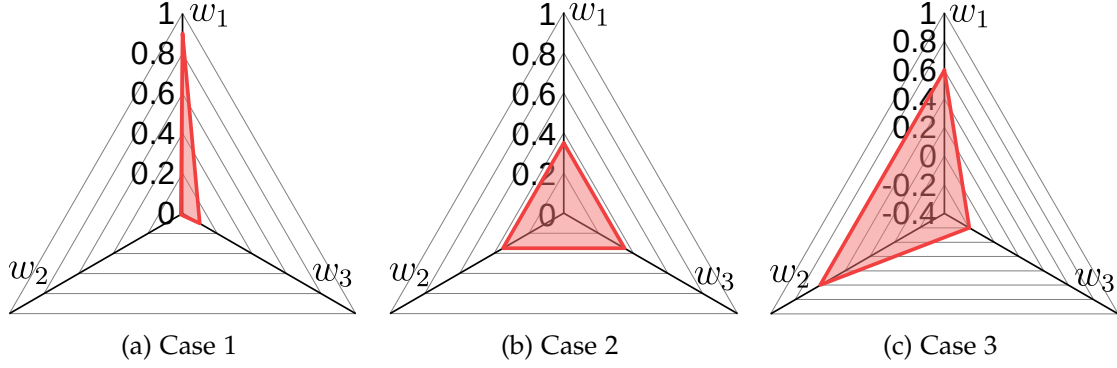


Figure 3.3: The coefficients w_1 , w_2 , w_3 of performance model in three different cases. Case 1: one factor (with w_1) dominates others. Case 2: all factors contribute equally. Case 3: one factor (with w_3) is negatively correlated.

Based on the variations of their weights, the relationship of T, C, M can be classified into three typical cases:

Case 1: One factor plays a dominant role in deciding the graph processing time: $w_i \gg w_{j \neq i} > 0$ ($i, j \in \{1, 2, 3\}$).

Case 2: More than one factors offer comparable and non-trivial contributions to the performance: $w_i \approx w_{j \neq i} > 0$.

Case 3: At least one factor counteracts against others with a negative weight: $w_i < 0$.

It should be emphasized that the above cases do not list out all possible situations, nor occur in mutual exclusiveness. They are enumerated due to their representativeness in explaining the behavior of the model. Many other cases can be described by mixing these three.

Fig. 3.3 exemplifies the three cases in Kiviat charts. Cases 1 and 2 are straightforward: the performance gain delivered by Corder attributes to the shortened thread time, reduced cache misses and/or decreased memory traffic. However, in Case 3 of Fig. 3.3c, the negative weight $w_3 < 0$, belonging to $r(M)$, indicates a performance loss. The memory accesses are raised, while both thread time and cache misses are lowered. When the loss from

memory outweighs the gains from thread and cache, the graph processing is expected to be decelerated, and vice versa.

The performance model of Eq. 3.2 depicts the relative importance of different multicore activities in building up the effectiveness of Corder. The coefficients in this model allow us to locate the root cause of the speedup offered by Corder; that is, the larger the weight value, the more important the corresponding factor. To obtain the values of those weights, comprehensive experiments are to be performed, which are detailed in following sections.

3.7 Experiment Setup

In our development environment, a dual-processor server is used. Each processor consists of 10 cores and each core has 2 threads. To eliminate contention on private caches, only 1 thread is enabled per core. In the memory hierarchy, the sizes of L1 cache, L2 cache, LLC and DRAM are 64 KB, 1 MB, 13.75 MB, 256 GB respectively.

Corder is developed in C++ on the basis of GPOP [8], a novel cache-aware graph processing framework designed for multicore systems. GPOP stores graphs in CSR for memory efficiency. It partitions a graph into subgraphs by equally splitting the vertex array into cache-able subsets. Each subgraph is exclusively processed by one core within its own cache. Following the authors' instruction, GPOP is parameterized with 20 threads and 1 MB partition size. A vertex occupies an unsigned 32-bit integer, so one subgraph contains 256K vertices. For reordering methods HC, FBC and DBG, we use the open-sourced code from the authors. All programs are compiled using G++ 9.3.0 with optimization level O3. The parallelization is realized with OpenMP. Our code is publicly accessible on GitHub¹.

The performance of reordering techniques are evaluated on every pair of graph applications and graph datasets. The experimental results are obtained by averaging 5 rounds of executions. The activities of cache and memory are measured using the `perf` [95] and `likwid` [96] tools respectively on Linux. Also, the hottest vertex with the richest connections in every graph dataset is sorted out. They are used as the input for root-dependent applications that require a root vertex as the starting point for traversing, such as BFS and SSSP.

Graph Datasets. We select seven large graphs for evaluation. The statistic summary in Table 3.3 presents their numbers of vertices, numbers of edges, maximum degrees and references. In addition, the inter-edges per subgraph after compression is presented.

Graphs *live*, *twitter* and *mpi* are collected from social networks to depict user relationships [51, 50, 48]. Extracted by web crawlers, *pld* and *wiki* model the hyperlinks among the web pages [49, 47]. *kron* is a synthetic graph generated by Graph500 Kronecker generator with

¹<https://github.com/yuang-chen/Corder-TPDS-21>

Table 3.3: Statistic summary of graphs (K: Thousand, M: Million, B: Billion). Inter stands for the inter-edges per subgraph.

Graphs	Vertices	Edges	Max Deg.	Refer.	Inter
<i>urand</i>	67.1M	2.1B	66	[46]	7.9M
<i>kron</i>	67.1M	2.1B	1.0M	[46]	2.8M
<i>pld</i>	42.9M	0.6B	3.9M	[47]	1.6M
<i>live</i>	4.8M	68.5M	2.0M	[48]	0.8M
<i>wiki</i>	18.3M	0.2B	9300	[49]	0.5M
<i>twitter</i>	41.7M	1.5B	3.0M	[50]	2.3M
<i>mpi</i>	52.6M	2.0B	0.8M	[51]	1.6M

the scale of 23 [46]. Using the same tool, *urand* is generated, but follows uniform random distribution [46]. Here, *urand* serves as an example of "not skewed and well balanced" graphs, though our work focuses on the optimization of skewed graphs. For this reason, the performance of *urand* is not taken into account when we calculate the average result over the graphs.

Graph Applications. Performance experiments are applied on six representative applications:

- **Pagerank (PR):** a algorithm that iteratively computes the ranks of vertices based on their own attributes and edges. It is a fundamental algorithm widely applied in numerous academic and industrial scenarios [85].
- **Pagerank Delta (PR- δ):** a variant of pagerank where vertices will be deactivated when their rank updates are smaller than a threshold δ . Compared to PR, it allows faster converging rate and requires less memory traffic.
- **Connected Component (CC):** find subgraphs, or namely components, where any two vertices can be connected by a path. Components are isolated from others, in each of which vertices share the same label.
- **Single Source Shortest Path (SSSP):** from a given root vertex, compute the shortest distance to all vertices in a weighted graph by using Bellman Ford algorithm.
- **Breadth-First Search (BFS):** traverse the graph starting from a given root vertex layer-wise.
- **Parallel Nibble (PN):** near a given vertex, discover a local cluster, the internal connections of which are enormously richer than its external connections [97]. The implementation is parallelized for optimization [98].

Table 3.4: Graph processing time of graph applications in seconds after applying different graph reordering methods. The original processing time (Orig.) without reordering is provided as the baseline for comparison.

Graph	Orig.	Corder	HC	DBG	FBC	Sort	RND		Orig.	Corder	HC	DBG	FBC	Sort	RND
Pagerank (1 iteration)									Pagerank Delta						
<i>urand</i>	1.23	1.19	1.13	1.16	1.14	1.21	1.15		7.24	7.92	7.92	7.59	8.58	10.54	8.17
<i>kron</i>	0.75	0.56	0.82	3.98	4.15	4.43	0.99		5.17	4.17	7.80	34.28	34.94	33.38	35.63
<i>pld</i>	0.27	0.24	0.26	0.66	0.62	0.68	0.55		2.32	2.29	3.01	5.08	4.87	5.21	2.79
<i>live</i>	0.06	0.04	0.06	0.10	0.12	0.12	0.05		0.35	0.26	0.47	0.66	0.73	0.82	0.41
<i>wiki</i>	0.20	0.12	0.22	0.23	0.23	0.23	0.09		1.33	0.76	1.38	1.32	1.40	1.39	0.63
<i>twitter</i>	0.76	0.45	0.95	1.40	1.64	1.81	0.72		7.81	3.58	10.70	13.36	16.88	18.89	7.42
<i>mpi</i>	1.74	1.09	1.66	1.70	1.82	2.05	0.97		16.47	7.95	13.40	11.30	13.16	13.75	6.34
Connected Component									Single Source Shortest Path						
<i>urand</i>	3.54	3.34	3.55	3.17	3.10	3.00	3.00		8.13	7.58	8.00	8.12	8.75	7.34	8.07
<i>kron</i>	2.09	1.14	2.21	6.87	7.94	7.45	2.34		1.81	1.53	3.94	8.08	9.65	9.72	2.25
<i>pld</i>	0.91	0.79	1.11	1.55	1.45	1.66	0.84		1.23	1.11	1.67	2.23	2.15	2.63	1.27
<i>live</i>	0.48	0.29	0.30	0.28	0.29	0.36	0.38		0.53	0.49	0.97	0.98	1.10	1.19	0.68
<i>wiki</i>	0.97	0.60	1.08	0.60	0.65	0.69	0.57		0.96	0.75	1.75	1.45	1.54	1.48	0.76
<i>twitter</i>	1.81	1.36	3.31	3.31	3.95	4.19	1.89		2.52	1.54	5.89	6.02	6.29	6.73	2.35
<i>mpi</i>	4.87	1.88	4.18	3.75	3.85	4.00	2.18		6.16	3.54	9.79	5.42	5.77	6.29	3.14
Breadth-First Search									Parallel Nibble						
<i>urand</i>	2.23	2.44	2.25	2.19	2.23	2.20	2.11		2.88	2.92	2.91	2.80	2.74	2.47	2.39
<i>kron</i>	1.35	1.26	1.08	3.25	3.71	3.55	2.63		0.76	0.68	0.69	1.46	1.43	1.40	0.77
<i>pld</i>	0.84	0.72	0.67	1.38	1.29	1.48	1.04		6.55	6.25	3.07	7.38	7.44	6.54	6.57
<i>live</i>	0.21	0.19	0.28	0.35	0.38	0.49	0.52		8.17	5.16	12.09	13.27	14.87	15.21	10.19
<i>wiki</i>	0.40	0.29	0.43	0.48	0.52	0.55	0.36		5.52	3.38	5.81	6.22	5.75	5.64	2.91
<i>twitter</i>	1.29	1.09	1.49	2.43	2.56	3.51	1.42		3.12	2.59	2.97	3.51	3.30	3.54	3.22
<i>mpi</i>	2.20	1.70	2.07	2.75	3.14	3.22	2.19		3.50	2.21	3.58	2.81	2.43	2.84	2.93

In particular, PR is the most sensitive application to the vertex order among the six. In every iteration of PR, all vertices stay active; that is, the property data of vertices are updated and propagated. By contrast, in other applications, only a subset of vertices participate in the computation. Since a vertex is deactivated and omitted from execution, its order becomes irrelevant. To this end, we select the hottest vertex as the root vertex as the input for those applications for the maximum activation.

3.8 Execution Time

Table 3.4 presents the graph processing time after applying different graph reordering methods. The processing time of graphs with original order (Orig.) is provided as the baseline for comparison. We can obtain the speedup by: speedup = {original process-

ing time}/\{reordered processing time\} = 1/slowdown. Therefore, speedup > 1 indicates performance improvement, while speedup < 1 (or slowdown > 1) means performance deterioration.

Corder outperforms state-of-the-art reordering methods in two aspects. First, it achieves the *highest* acceleration on the skewed graphs, which is up to $2.59\times$ and on average $1.45\times$. Second, Corder consistently boosts the performance of *every* graph application on *every* skewed graph. Slight slowdown occurs on graph *urand*, as it is an artificial graph with completely uniform distribution. Moreover, by analyzing the characteristics of graph datasets and graph applications, we acquire following findings:

Finding 1: The deployment of contemporary lightweight reordering techniques, including HC, FBC, DBG and Sort, often leads to considerable slowdowns. The average slowdowns of them are $1.09\times$, $1.37\times$, $1.43\times$ and $1.54\times$ respectively. The observation contradicts with the statement reported by [35, 53, 38] that speedup is expected. This contradiction ascribes to the "processing paradigm".

The *vertex-centric* paradigm (e.g., Ligra), as originally implemented underneath HC, FBC and DBG, addresses the graph at the granularity of vertex per thread. As a result, the issue of workload imbalance is relatively trivial, even though the degrees and corresponding workloads of the vertices could vary from 0 to millions as listed in Table 3.3.

The *partition-centric* paradigm adopted in our work, however, treats the graph at a much coarser granularity. A partition containing over hundred thousand vertices (e.g., 262144 vertices in our experiment) acts as the basic unit for single thread to process. Hence, with the use of HC, FBC, DBG and Sort, hot vertices are concentrated into a small fraction of subgraphs. As listed in Table 3.5, the $L\text{-}S_{20\%}$ of graphs are significantly enlarged by those reordering methods. The gathering of hot vertices heavily exacerbates the imbalance issue and consequently becomes the primary reason for performance degradation. Contrariwise, Corder and RND lower the L-S of graphs and effectively alleviate the problem of workload

Table 3.5: Locality-Skew ($\lambda = 20\%$) of graphs after graph reordering. ∞ indicates that the sum of degrees over ending subgraphs, i.e., $\sum_{s \in \text{end}_\lambda} L(s)$ in Eq. 3.1, is zero.

	Orig.	Corder	HC	FBC	DBG	Sort	RND
<i>urand</i>	1.00	1.00	1.34	1.47	1.33	1.632	1.00
<i>kron</i>	1.10	1.11	57.71	57.71	95.01	∞	1.11
<i>pld</i>	1.61	1.48	32.30	32.30	52.48	∞	1.22
<i>live</i>	13.19	2.40	25.07	26.50	30.94	84.81	1.01
<i>wiki</i>	12.04	2.37	70.90	70.90	156.45	∞	1.02
<i>twitter</i>	15.56	7.81	21.27	21.28	28.30	101.47	1.42
<i>mpi</i>	22.69	4.43	42.55	42.55	98.01	746.39	1.05

imbalance. Thus, the execution time is shortened (though not always for RND).

Finding 2: Random reordering is occasionally beneficial to the performance. Sometimes it provides performance improvement even higher than Corder, such as PR on graph *wiki*. From the perspective of statistics, random reordering can be interpreted as uniformly distributing hot vertices across all subgraphs. After applying RND, subgraphs not only accommodate the same number of hot vertices, but also incur almost identical workloads. As presented the RND column in Table 3.5, RND achieves the smallest L-S_{20%} in comparison with other reordering techniques. The problem of workload imbalance is solved by randomization.

However, the random placement of vertices completely destroys natural graph structure and eliminates graph locality. As the graph locality vanishes, cache misses are dramatically increased. For RND, there is an inevitable conflict between workload balance and graph locality. The overall performance is undermined when the loss from graph locality outweighs the gain from workload balance, and vice versa.

As shown in Table 3.5, graph *pld* is fairly balanced as its L-S_{20%} in the original order is low: 1.61. Applying RND on the graph, the L-S_{20%} is further reduced to 1.22, whereas the cache misses (e.g., for PR) are increased from 6.28 Billions to 9.87 Billions as in Table 3.6. In such scenario, the cache miss dominates the workload balance. Consequently, the execution time of PR per iteration on graph *pld* is extended from 0.27s to 0.55s due to the increased cache misses by RND. On the contrary, in the case of graph *mpi* that suffers from severe imbalance, RND effectively reduces the inequality of workloads, lowering L-S_{20%} from 22.64 to 1.05. Cache misses, though increased from 10.97 Billions to 32.51 Billions, are compensated by balanced workloads. As a result, the overall processing time of PR is shortened from 1.74s to 0.97s by the use of RND.

Finding 3: The higher Locality-Skew a graph is associated with, the better result Corder delivers. This meets our expectation proposed in the end of Section 3.4. Fig. 3.4 visualizes

Table 3.6: L2 cache misses (in Billions) during 20 iterations of Pagerank after applying different reordering methods

	Orig.	Corder	HC	DBG	FCB	SRT	RND
<i>urand</i>	41.95	40.78	41.90	42.56	42.70	42.75	44.41
<i>kron</i>	15.36	11.75	15.22	21.14	19.82	20.86	29.86
<i>pld</i>	6.28	5.12	6.90	6.72	7.72	7.42	9.87
<i>live</i>	0.49	0.56	0.52	0.61	0.67	0.70	1.05
<i>wiki</i>	1.11	1.08	1.18	1.35	1.48	1.46	2.62
<i>twitter</i>	9.61	9.82	9.96	13.72	13.05	16.08	23.72
<i>mpi</i>	10.97	11.88	13.22	16.36	17.16	18.76	32.51

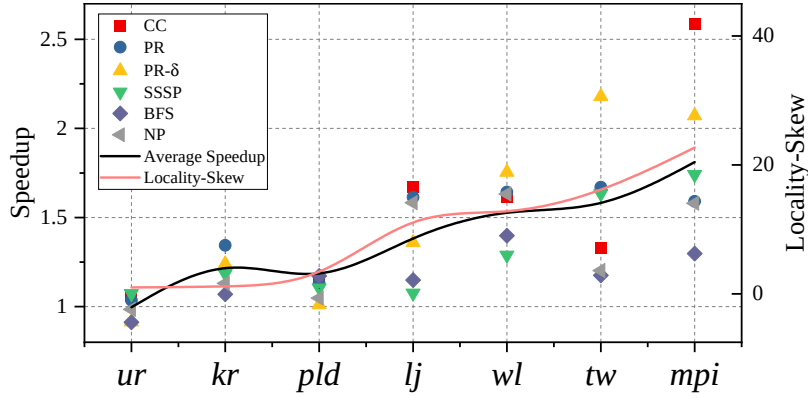


Figure 3.4: The correlation between speedup and Locality-Skew ($\lambda = 20\%$).

the correlation between the $L\text{-}S_{20\%}$ of graphs in original order and the speedups offered by Corder. Graph applications are increasingly accelerated as the Locality-Skew of graphs rises. For instance, graph *mpi* with $L\text{-}S_{20\%} = 22.69$ achieves $1.81\times$ speedup on average of all applications. By contrast, the average gain of graph *pld* with $L\text{-}S_{20\%} = 1.61$ is significantly lower, which is $1.10\times$ speedup. The ascending trend of speedup towards the growing $L\text{-}S_{20\%}$ validates the effectiveness of Corder in balancing workloads.

3.9 Multicore Activities with Corder

3.9.1 Thread Behavior

In a multicore system where synchronization is required, the processing time of a parallel region mainly depends on the thread with the longest execution time. We define such thread as the *critical thread*, which is positioned in the most imbalanced point in a multi-threaded task. Meanwhile, we define the *average thread* as the representative of all threads. The performance of the average thread is nothing but the arithmetic mean of all threads within the parallel region. Fig. 3.5 illustrates the speedups of graph processing, the belonging critical threads and the average threads.

It can be observed that the performance of the process is tightly related with the critical thread. As the Locality-Skew grows from graph *urand* to *mpi*, the process and the critical thread are simultaneously accelerated. Corder addresses the imbalance issue by shortening the critical thread, and consequently accelerates the overall graph processing. Moreover, the correlation between the critical thread and the overall performance indicates that the performance gain mainly results from the balanced workloads (i.e., shortened critical threads). In Section 3.10, we will present detailed verification.

Although the overall process is boosted by Corder, the average thread completion speed

may be actually slowed down. For instance, for PR on graph *live*, the overall processing time reduces from 63ms to 39ms per iteration (i.e., $1.61\times$ speedup), as its critical thread time decreases from 44ms to 31ms (i.e., $1.42\times$ speedup). Meanwhile, its average thread time of increases from 18ms to 21ms (i.e., $1.16\times$ slowdown). As Fig. 3.5 depicts, the behavior of the average thread is closely related with the cache efficiency, which is to be detailed in the next section.

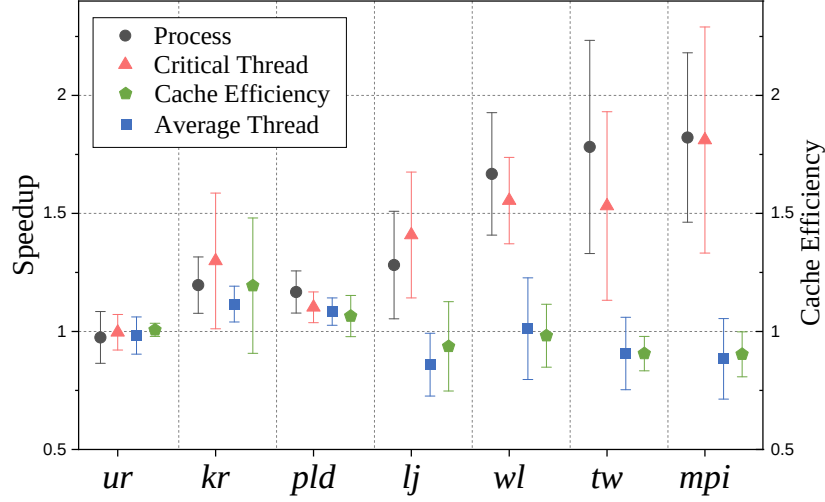


Figure 3.5: The impact of Corder on thread behaviors and cache efficiency. The speedups (left) of graph processing, critical thread, and average thread are plotted with the improvement of cache efficiency (right). The results are calculated by the mean of different graph applications with 90% confidence interval.

3.9.2 Cache Utility

Since our optimization on the cache utility is focused on the L2 cache, we collect the L2 cache misses for evaluation. In particular, we characterize the *improvement of cache efficiency* (ICE) by the cache misses prior and subsequent to the deployment of graph reordering: $ICE = \{\text{original cache misses}\} / \{\text{reordered cache misses}\}$. When $ICE > 1$, cache misses are reduced and therefore the cache is better utilized, and vice versa. Due to space limit, only a subset of concrete data regarding the cache misses are presented in this chapter, such as Table 3.6.

As depicted in Fig. 3.5, the ICE delivered by Corder shows that the behavior of the average thread corresponds with the trend of cache efficiency. Sometimes, the cache efficiency deteriorates due to the increased graph edges (see explanations in Section 3.9.3). Accordingly, the average speed of threads declines. However, the slowdown of the average threads cannot prevent the overall processing from being speeded up.

For instance, the cache misses of PR on *live* grow from 0.49 Billions to 0.56 Billions (i.e., ICE = 0.86). At the same time, its average thread time extends from 18ms to 21ms (i.e., $1.16\times$ slowdown). This phenomenon follows our common sense: poor cache utilization decelerates the execution of (a majority of) threads. Nevertheless, as discussed in last section, the critical thread is accelerated, which plays a crucial role in deciding the processing time of a parallel region. The reduction in execution time of critical thread compensates the increased computing cost of other threads. For this reason, the graph processing is accelerated.

To conclude, cache efficiency exhibits less importance compared to the issue of workload balance. Its behavior synchronizes with the average thread. However, its impact on the overall performance of graph processing is shadowed by the critical thread, specially for the graphs with high L-S, such as *wiki*, *twitter* and *mpi*.

3.9.3 Memory Dynamics

Corder leads to an upsurge of inter-edges, which acts as a side effect to the partition-centric paradigm. The lower part of Fig. 3.6 shows the ratio of compressed edges (i.e., inter-edges) compared to original vertex-to-vertex edges. After employing Corder, hot vertices are evenly distributed among subgraphs, incurring frequent message exchanges between them inside different processing cores. Therefore, the inter-edges of graphs, representing the core-to-core communication, are increased.

The inflation of graph inter-edges results in heavier computation loads, thus more cache misses (discussed in Section 3.9.2) and memory traffic. Fig. 3.6 presents the growth of memory accesses (upper) due to the addition of edges (lower). Compared to graphs with low L-S (e.g., *kr*), skewed graphs with high L-S (e.g., *mpi*) tend to invoke less memory accesses per edge and thus consume lower memory bandwidth. Also, the impact of Corder on the skewed graphs are significantly larger too.

Corder burdens the efficiency of memory (i.e., $r(M)$ in Eq. 3.2) because of the raised memory accesses incurred by additional inter-edges. The accesses to memory are typically slower than those to caches by orders of magnitude. Hence, the extra memory accesses might potentially decelerate the graph processing, though its negative impact is compensated by the improvement from other factors, such as the shortened critical thread. Comparing Fig. 3.5 with Fig. 3.6, it shows an inverse correlation between the speedup of graph processing and the memory efficiency; that is, the higher the speedup, the more the memory accesses, and the poorer the memory efficiency. Therefore, we expect a negative coefficient for $r(M)$.

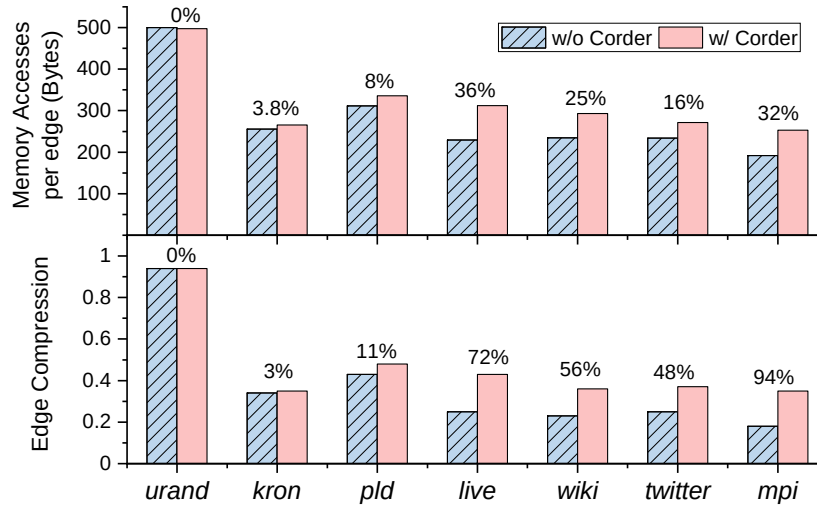


Figure 3.6: The correlation between memory traffic and edge compression. Upper: Memory Accesses Volume per Edge. Lower: Edge Compression Ratio. Corder inflates graph inter-edges and thus raises memory traffic. Every pair of results (w/o vs w/ Corder) is denoted with the increased percentage.

3.10 The Derivation of Performance Model

Based on previous discussions, we obtain a preliminary observation about the influential factors for the speedup of partition-centric graph processing in the multicore system: the *workload imbalance* among threads dominates the *cache efficiency* and the *memory efficiency*, and moreover, the latter behaves oppositely to the trend of speedup.

For concreteness, we perform a multivariate regression on the observed data. It characterizes the association among the input variables and explains the dependence of the outcome on the inputs [99]. Specifically, we feed the model in Eq. 3.2 with the critical thread execution time, L2 cache misses and memory accesses. Firstly, for simplicity, we only use the experimental results from PageRank, during which all vertices stay active. The fitted model is given as follows:

$$speedup = 0.87 \cdot r(T) + 0.27 \cdot r(C) - 0.16 \cdot r(M) \quad (3.3)$$

Eq. 3.3 validates our observation. The importance of the critical thread outweighs other factors. Cache efficiency is improved, though its contribution is relatively minor. Also, the memory access is associated with a negative coefficient, indicating it is itself a (minor) cost for the speedup. As described in Section 3.5, this model employs a mix of Case 1 and Case 3. Then, we use all data collected from all graph applications for regression. The resulting

model is formulated as follow:

$$speedup = 0.92 \cdot r(T) + 0.05 \cdot r(C) + 0.08 \cdot r(M) \quad (3.4)$$

As Eq 3.4 shows, the importance of the critical thread is further elevated, while the other two are drastically decreased. Such adjustment originates from the active vertex set – a new factor introduced by all graph algorithms except for PR. In every iteration, the active vertices dynamically change according to the internal structure of the graph, which considerably complicates the performance model (and goes beyond the scope of this chapter). As a consequence, the weight of imbalanced threads is enlarged, falling into the category of Case 1.

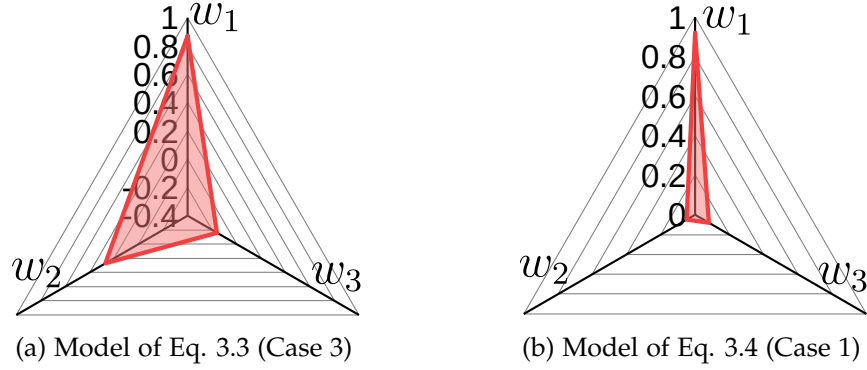


Figure 3.7: The coefficients of performance models fitting the experimental data of PageRank (a) and all graph applications (b).

The models proposed for Corder also explain the behaviors of other reordering methods. For example, as discussed in Findings 2, RND substantially boosts the performance of PageRank on the highly skewed graph *mpi*. After deploying RND, though the cache misses are increased by $3\times$ as in Table 3.6, the critical thread execution time is accelerated by $8\times$. Since the importance of critical thread (i.e., workload imbalance) outweighs the cache efficiency, the overall execution time is consequently shortened.

Lastly, it is worth noting that the performance models in Eq. 3.3 and 3.4 are derived to rank the relative importance of the thread performance, cache utilization and memory traffic from a statistical point of view. The models simplifies the interaction among the counted factors and neglects many other issues, such as the dynamic active vertices and the choice of root vertex. Hence, we do not expect to solely rely on these models to precisely predict the performance of a given graph application.

3.11 Sensitivity Analysis of Partition Size

The partition size of Corder represents a trade-off. Small subgraphs facilitate fine-granularity workload balance; whereas large subgraphs promote high locality and preserve better graph structure. At the extreme, if each subgraph is so small that it contains only a single vertex, Corder becomes the RND reordering. On the other side, if the subgraph is large enough to feature the same size as original graph, there exist only one subgraph including all vertices. In such sense, Corder eventuates to be HC. However, as Table 3.4 shows, neither of them promises performance boost.

In order to evaluate the impact of Corder’s partition size on the execution time of graph processing, we examine the performance of PR on various graphs. The PR is selected as the benchmark application for demonstration due to its high sensitivity to vertex order. Fig. 3.8 presents the execution time with partition size of Corder varying from 128KB (32K vertices) to 16MB (4M vertices). Meanwhile, the underlying GPOP is fixed with 1 MB partition size to eliminate extraneous variables.

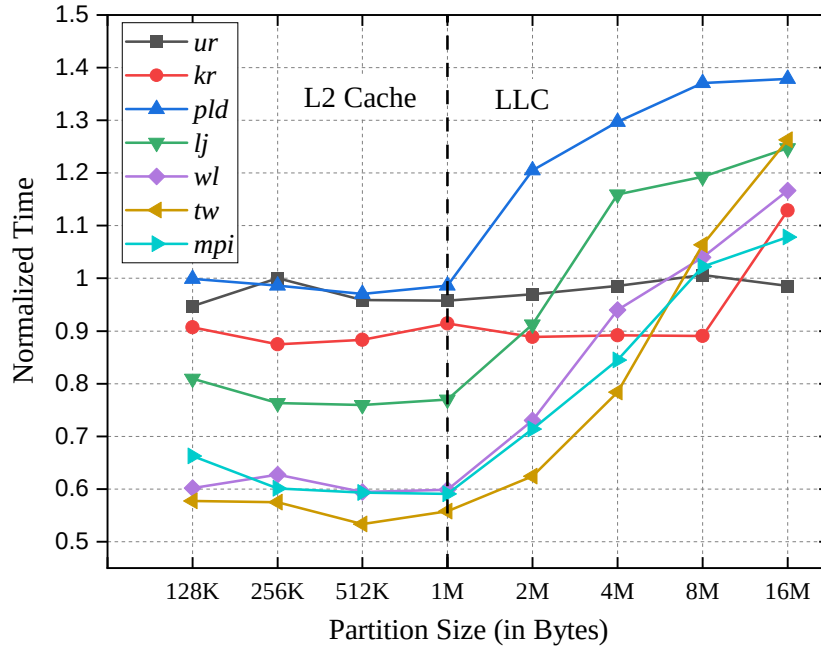


Figure 3.8: The impact of partition size on the execution time. The time is normalized by the result of unordered graph.

As in prior experiments, graph *urand* remains almost unaffected by partition size due to its low graph locality and uniform distribution of vertex degree. For skewed graphs, there is a distinguish threshold at 1M bytes, which equals the size of L2 cache. As long as the partition size varies within 1M, Corder achieves the same level of acceleration. In this range,

1M is preferable to other options because it results in less disruption to graph structure.

Nevertheless, once the partitioning size exceeds the capacity of L2 cache, the vertices are spilled over from L2 into LLC. The spillover incurs drastic increase of cache misses as well as memory traffic. Hence, the computation is decelerated. As an exception, *kron* is not heavily influenced by the partition size ranging from 128K to 8M due to its well balanced workload (e.g., low Locality-Skew). The partition size of 16M, however, diminishes its internal balance, and thus slows the execution down.

3.12 Scalability onto Vertex-Centric Paradigm

With the awareness of cache hierarchy, Corder is developed based on the recently proposed partition-centric graph processing paradigm. Nevertheless, the conventional vertex-centric paradigm is popularly adopted in numerous graph frameworks over the past decades [17, 4, 5, 10]. The reordering methods proposed for the vertex-centric framework (e.g., Ligra), such as HC, FBC and DBG, often deteriorate the performance of graph analytics when deployed in the partition-centric GPOP. Hence, in reverse, it is necessary to investigate the scalability of Corder onto the vertex-centric platforms.

Fig. 3.9 compares the speedups of four different frameworks. Three of the four are programmed in vertex-centric paradigm, including Ligra, Polymer and the Non-Uniform Memory Access scheduled Ligra (NUMA-Ligra). The partition-centric one, PCPM [93], is designed for efficient Pagerank algorithm. Again, we use PR as the benchmark application because of its sensitivity to vertex order. Overall, Corder demonstrates higher scalability than any other reordering methods. It consistently achieves speedups (at least no slowdown) for skewed graphs across all frameworks, except for *live* in NUMA-ligra.

PCPM: The partition-centric PCPM behaves similarly to GPOP. The aggravation of hot vertices promoted by HC, FBC, DBG and Sort intensified the issue of workload imbalance. Therefore, the speed of graph processing is degraded. RND leads to performance boost, when graphs exhibit inherent high imbalance (i.e., high Locality-Skew), such as *mpi* and *twitter*. It results in a slowdown, when the vertices are already well distributed in the graphs (e.g., *kron* and *pld*). Corder, with the optimization of workload balance as well as graph locality, consistently achieves speedups for the skewed graphs, which is up to $1.77\times$ and on average $1.41\times$.

Ligra: Unexpectedly, the performance of vertex-centric Ligra oftentimes deteriorates when HC, FBC and DBG are deployed. Corder is the only reordering method that accelerates the execution of all graphs with speedup of up to $1.64\times$ (on average $1.37\times$).

A key feature of Ligra-based PageRank is the atomic update of data [4]. Atomic operations are used to ensure the correctness of multiple threads accessing a common vertex. It is an expensive synchronizing operation that stalls the execution of other threads.

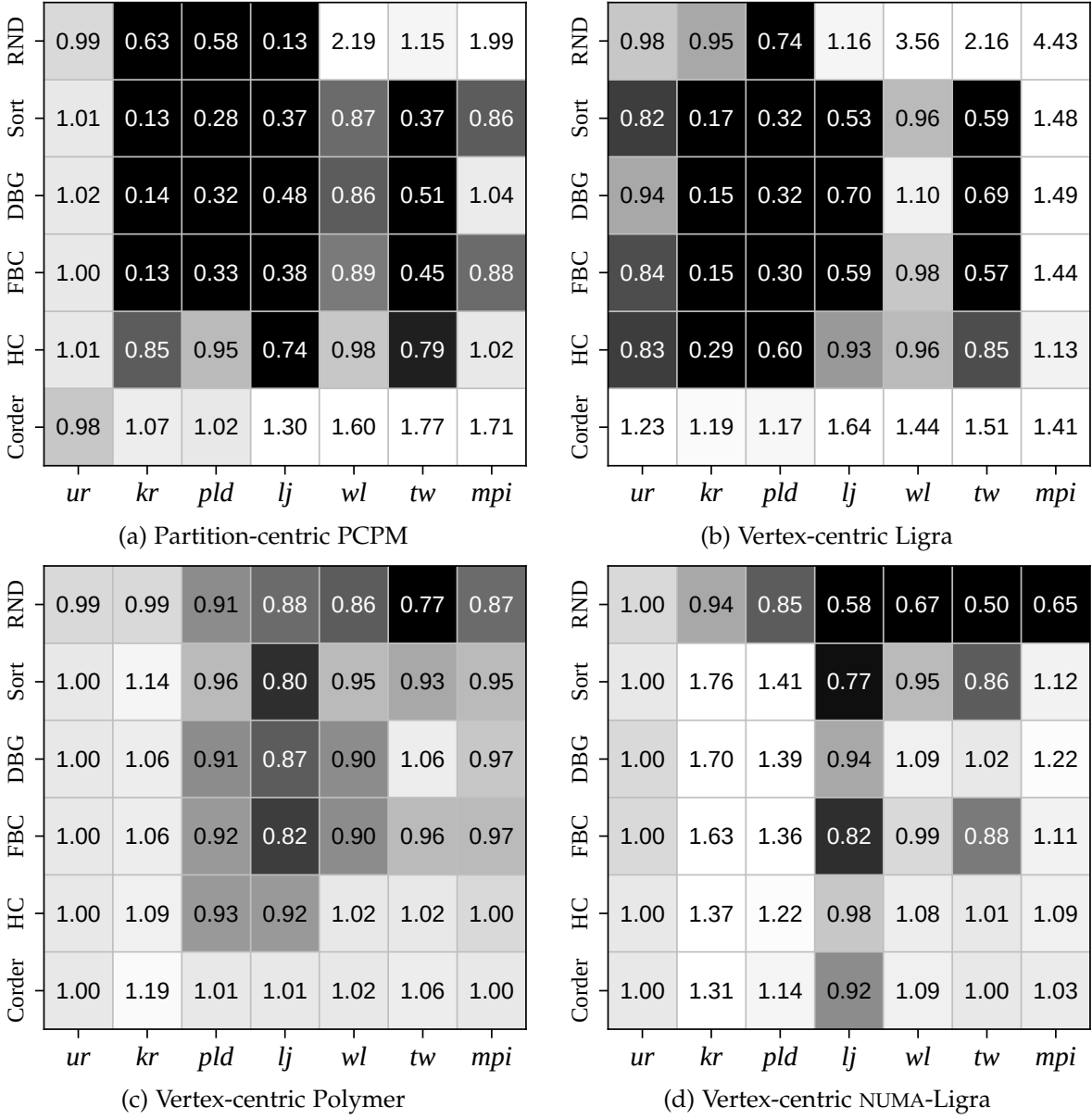


Figure 3.9: Speedups in different graph processing frameworks. The lighter the color, the higher the speedup. Corder delivers consistent performance boost across all frameworks.

As hot vertices are concentrated via graph reordering (e.g., DBG), collisions between threads are more prone to occur. Therefore, the parallel execution of multi-threads is frequently stalled by atomics, shrinking to single-thread processing. In other words, the problem of thread blocking is exacerbated by the concentration of hot vertices, which outweighs the benefit from enhanced graph locality. An abnormality is graph *mpi* with $L-S = 432.43$ ($\lambda = 1\%$). Since *mpi* already suffers from severe thread blocking due to its high

skew, the side effect of graph reordering is relatively minor on it. Consequently, enjoying the enhancement of graph locality, *mpi* achieves substantial speedup.

Polymer: Polymer is implemented based on the code of Ligra and follows the vertex-centric paradigm. However, the effect of reordering techniques on Polymer is insignificant. With the awareness of NUMA, Polymer evenly partitions and allocates graph data in the local memory of NUMA nodes. This minimizes the effectiveness of graph reordering. Hence, compared with other frameworks, the performance variation of Polymer across the graphs and reordering techniques is trivial. Corder performs as the only reordering method that does not cause a slowdown.

Nevertheless, the advancement of Polymer is achieved at the price of sophisticated optimizing techniques. To utilize the NUMA feature, it redesigns the data layout, and requires graph partitioning as well as thread binding. Together, these implementations contribute to thousands of lines of codes. Moreover, such complexity produces staggering overhead. For instance, the preprocessing overhead of Polymer ranges from 7.61s of *live* to 346s of *urand*. On the other hand, Corder caters to NUMA without manipulation of underlying framework. It involves less coding effort, e.g., about hundred lines of codes. Also, it introduces minimum overhead, which is to be discussed in following section.

NUMA-Ligra: Though NUMA-Ligra facilitates graph processing with the support of NUMA too, it differs from Polymer. NUMA-Ligra is enabled by the command line utility on Linux-based NUMA machines. The NUMA scheduling and memory placement are automated by the NUMA policy kernel [100]. Simply with one line of command (e.g., `numactl [-options] program`), users can execute a program with specific NUMA policy.

The interleave memory allocation of NUMA-Ligra alleviates the workload imbalance incurred by graph reordering. Moreover, compared with Polymer, it ensures the graph locality enhanced by those methods. Therefore, graphs with low locality, e.g., *kron* and *pld*, acquires substantial performance boost due to the enhancement of graph locality using HC, FBC and DBG. Other graphs, which are already associated with high locality, achieve limited improvement (e.g., *wiki*, *twitter* and *mpi*) or even degradation (e.g., *live*).

3.13 Reordering Overhead

The practicality of a reordering method is heavily influenced by the overhead it imposes. For instance, as listed in Table 3.7, it costs Gorder 66.8 seconds to finish the computation on the smallest graph *live*. Nonetheless, the downstream graph applications typically ends in 10 seconds even without reordering preprocessing. Considering such difference, the overhead of Gorder is prohibitively expensive.

The reordering overhead consists of two phases: *mapping* and *composing*. In the mapping phase, a new vertex ID is assigned to every vertex in the original graph by a reordering

Table 3.7: Reordering Overhead in seconds. The overhead consists of the mapping time and composing time. For Gorder, the reordering process is terminated once it exceeds 1 hour. The last row lists the average number of iterations of PR on original graphs needed to amortize the time cost.

Graph	Mapping					Composing
	Corder	HC	FBC	DBG	Gorder	
<i>urand</i>	0.393	0.912	1.008	0.091	>1 h	3.081 $\pm$ 0.271
<i>kron</i>	0.222	0.192	0.700	0.104	>1 h	2.151 $\pm$ 0.125
<i>pld</i>	0.154	0.136	0.435	0.164	>1 h	0.825 $\pm$ 0.035
<i>live</i>	0.017	0.029	0.058	0.007	66.8	0.073 $\pm$ 0.000
<i>wiki</i>	0.060	0.115	0.214	0.027	220.6	0.250 $\pm$ 0.007
<i>twitter</i>	0.113	0.114	0.370	0.049	>1 h	0.901 $\pm$ 0.042
<i>mpi</i>	0.153	0.185	0.574	0.169	>1 h	1.520 $\pm$ 0.065
PR Iter.	0.284	0.398	0.879	0.175	>1000	1.840

method. Thereby, a mapping pair of vertex IDs prior and after the reordering assignment is created, which point to the same vertex. For instance, $pair(old, new) = (v1, v100)$. In the composing phase, a new graph is constructed based on the new IDs of vertices. The composition of a graph (e.g., CSR, edge list or adjacency matrix) is decided by the data layout implemented in underlying graph frameworks. Adding up the mapping time and the composing time, we acquire the result of the reordering overhead.

To compare the overheads, we deploy the reordering algorithms proposed by prior works, including HC, FBC, DBG and Gorder, using the open-sourced code provided by the authors. The source code of HC, FBC, DBG is developed in the framework of Ligra. Hence, to provide a fair comparison, we re-implement Corder and evaluate its overhead in Ligra too.

Table 3.7 presents the reordering overheads in terms of the mapping time and the composing time. The mapping time reflects the efficiency of a reordering algorithm. Corder offers the fastest mapping speed for graph *mpi*. Overall, it is the second fastest reordering algorithm only behind DBG. On average, the mapping time of Corder can be amortized by 0.284 iteration of PR on the original graph. Featuring a lightweight design of parallelism, Corder achieves high efficiency.

The composing time is independent of the reordering methods, but subject to the size of a graph. Compared with the mapping phase, the composing phase charges time cost by an order of magnitude, which is 1.84 iterations on average. Hence, the overall reordering overhead, especially for applications such as BFS that finishes within 3 seconds and can be roughly amortized by 2 or 3 iterations of PR, is a non-trivial consideration.

3.14 Summary

Graph reordering functions as a critical prerequisite to facilitate the processing of large-scale graphs. In this work, we demonstrate that workload imbalance severely deteriorates the performance of parallel graph processing. A novel measurement approach, i.e., Locality-Skew, is proposed to estimate the imbalance in a graph. Inspired by it, we present Corder, a reordering method designed for multicore systems with awareness of the cache hierarchy. Firstly, Corder partitions a graph into subgraphs, of which the vertex subsets fit into L2 cache. Then, Corder adopts two optimization strategies to facilitate graph processing. Within the shared memory, hot vertices are evenly distributed across subgraphs. Inside the local cache, hot vertices are concentrated to improve graph locality.

Our evaluation shows the effectiveness of Corder not only in a variety of graph datasets and graph applications, but also in diverse graph analytics frameworks. Moreover, we analyze and rank the influential factors correlating with Corder in the multicore systems, including thread behavior, cache and memory efficiencies. Amongst them, the balancing of thread execution time plays the most weighty role in the boost of graph processing.

The future work includes the integration of cache-aware reordering methodology with NUMA. Besides, we plan to investigate the difference in behaviors for directed graphs when hot vertices are classified by in-degrees instead of out-degrees.

Chapter 4

Unequal-Sized Caching

Recent advances in computer architecture significantly enhance the computational capacity of multicore systems. It allows large-scale graphs to be processed inside a single machine. Nevertheless, the irregular processing pattern of graph-structured data constrains the hardware resources from being productively utilized. In this chapter, we investigate the constraints in two aspects: workload imbalance and parallel inefficiency. When a graph analytics algorithm is multithreaded, the thread time is highly diversified, indicating an uneven work distribution. Also, the intensive thread contention frequently stalls the computing cores, thereby hindering the effectiveness of parallelization. To address these challenges, we present a proactive graph caching strategy that unequally segments graph components into cache-able subsets of varying sizes, namely Syze [41].

4.1 Motivation

This section investigates the computing challenges in prior CSR-segmenting graph processing frameworks. We experimentally demonstrate that processor cores severely suffer from workload imbalance as well as parallel inefficiency. These issues motivate the proposal of our work.

Workload Imbalance

The contrast between hot vertices and cold ones causes workload imbalance when a graph algorithm is multithreaded. To tackle this issue, conventional shared-memory graph analytics frameworks dynamically parallelize graph algorithms by using the work-stealing policy of Cilk [4, 5] or the dynamic scheduler of OpenMP [101, 8]. However, it is insufficient to solely rely on the programming tool to address the problem of imbalance.

For instance, in CSR-segmenting works, a subgraph is the basic unit for threads to execute. Since the number of vertices in all subgraphs are equated, the workload for each

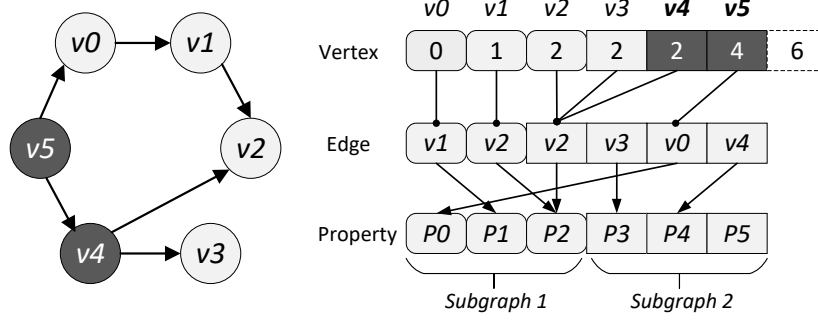


Figure 4.1: A directed graph and its CSR representation. Assuming a private cache can hold three vertices, the graph is divided into two subgraphs as the round and square boxes respectively. In particular, hot vertices $v4$ and $v5$ are highlighted because they are densely connected with two out-edges. They lead to workload imbalance between the two subgraphs.

subgraph mainly depends on the edges. The local concentration of hot vertices leads to the imbalance among subgraphs. In the example of Fig. 4.1, vertices $v4$ and $v5$ together contribute 4 out-edges in subgraph 2, while there are only 2 out-edges in subgraph 1. Thus, subgraph 2 tends to involve heavier communication as well as computation.

To validate our reasoning, we measure the execution time of parallel threads by running PageRank under a CSR-segmenting framework GPOP [8]. The experiment is performed on graph *twitter* [50]. A dual-processor Intel Xeon CPU is used. 20 threads are utilized, and the size of each vertex subset equals 256 KB. Also, the vertex degrees of each segment are summed up.

Table 4.1: Statistical summary of GPOP-based PageRank. The thread execution time per iteration and the sum of vertex degree per segment are presented in terms of maximum, minimum and average results. The bottom row of each attribute shows the ratios between the max result and each result.

Attribute		max	min	average
Thread	Time (ms)	320.39	1.78	6.06
	Ratio (max/*)	1.00×	180.16×	52.86×
Degree	Sum (million)	213	0.626	2.3
	Ratio (max/*)	1.00×	824×	92×

Table 4.1 lists the statistical summary of the experiment results. It can be observed that the max thread costs considerably longer time than the average and min threads. Such difference results from the varied distribution of hot vertices. The densely connected vertices

are highly concentrated in a few subgraphs, exhibiting the intrinsic irregularity of the graph.

For a parallel region, the overall processing time is decided by the longest thread time. The results in Table 4.1 suggests that the partition-centric paradigm is undermined by the fluctuating thread execution time due to the concentration of hot vertices. This encourages us to develop a strategy that bridges the gaps between threads, i.e., shortening the longest thread time when multithreading.

Parallel Inefficiency

Besides uneven thread time, CSR segmentation also inhibits the full utilization of processor cores [8]. The cache-able vertex subset assigned to each thread requires full occupation of caches, which intensifies thread contention. Also, the cache-sized data per thread invoke heavy data movement, thereby consuming memory at high bandwidth [93]. When the hardware resources are exhausted, the processor cores become saturated before 100 percent usage.

The efficiency of a program utilizing available cores is characterized by *parallel efficiency*. This metric reports the percent of average utilization by all cores, which can be measured via Intel Vtune Profiler [102]. By multiplying it with the number of available cores, the number of effective cores is acquired. For example, assuming a 20-core system, a parallel efficiency of 50% means that the system behaves as if $20 \times 50\% = 10$ cores are fully loaded by computations.

Since today's CPU usually offers two logical cores (i.e., threads) per physical core by Hyper-threading [103], the parallel efficiencies of logic cores (PE_L) and physical cores (PE_P) are measured simultaneously. Again, GPOP-based PageRank is tested by adjusting the size of thread pool T on a system offering 20 physical cores and 40 logical cores. The processing time is depicted as the speedup over the single-threaded implementation.

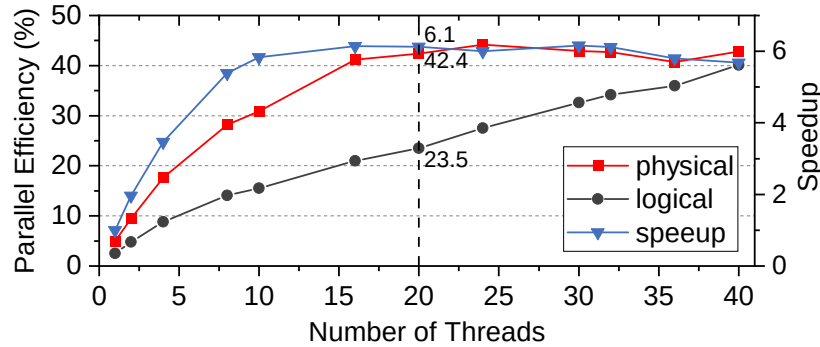


Figure 4.2: Parallel efficiency of physical cores and logical cores during the running of PageRank on graph *twitter*

As shown in Fig. 4.2, both PE_L and PE_p scale well with T increasing from 1 to 20. Nevertheless, PE_p is saturated when $T = 20 =$ the number of physical cores. In this sense, there are approximately 9 physical/logical cores effectively running. As T grows from 20, PE_L continues to increase; oppositely, PE_p experiences a slight decline, so for the speedup.

Therefore, it is the real physical cores instead of the virtual logical cores that determine the performance of graph processing. The saturation of PE_p indicates that the thread pool is overcrowded when $T > 20$, so the program cannot enjoy the benefit of Hyper-threading. Available resources, such as memory bandwidth, are depleted by the excessive number of threads engaged in the computation [8]. As a result, threads are frequently halted because of intensive thread contention, thus slowing the processing down.

In order to avoid the exhaustion of resources, prior works simply utilize half of available threads; that is, the number of working threads equals the number of physical cores [93, 8, 94, 101]. Nevertheless, this leads to a waste of computing capacity. To this end, we are motivated to improve the parallel efficiencies of multicores by alleviating thread contention.

4.2 Design of Syze

To ameliorate workload imbalance and enhance parallel efficiency, we propose Syze, an optimization strategy for graph processing based on CSR segmentation. The abstraction of Syze is visualized in Fig. 4.3 and the design goal of it is to dilute the hot subgraphs. In principle, the subgraph identified as "hot" are further subdivided into sub-units with smaller sizes to alleviate the concentration of hot vertices. The details of Syze are elaborated in following sections.

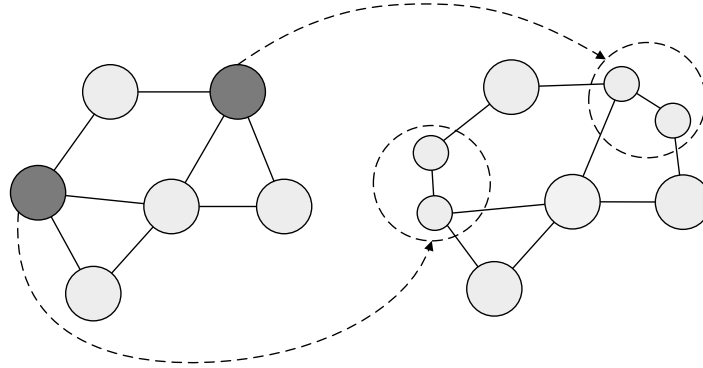


Figure 4.3: Abstraction of unequal-sized caching. A graph is firstly segmented into cacheable subgraphs (grey cycles) with uniform vertex subset size. Then, the hot subgraphs accommodating an excessive number of hot vertices (in dark grey) are further subdivided into smaller units.

4.2.1 Unequal Caching

Before describing the mechanism of unequal caching, we firstly formally define the terminologies with respect to the graph and subgraphs. A graph is represented by $G = (V, E)$, where V is the vertex set and E is the edge set. The capacity of cache is denoted as $C = \{\text{size of cache}\} / \{\text{size of a vertex}\}$. Further, *Subgraph degree* D^s is defined as the total sum of vertex degrees in a vertex subset. Based on the definitions, we unequally cache a graph in 3 steps:

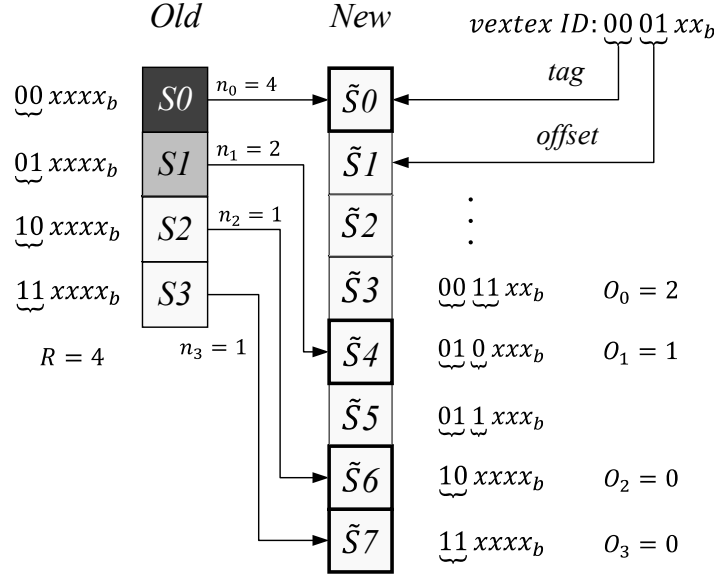


Figure 4.4: An example of unequal-sized segmenting. Hot subgraphs in the old array are subdivided into smaller units in the new array. The anchored subgraphs are plotted with thick boxes and mapped to their original subgraphs. The precise location of a target subgraph in the new array depends on both anchoring bits and chaining bits. The coverage of each subgraph is annotated in binary format.

Step 1: Initial Segmenting. First of all, the vertex set V is segmented into N cache-able subset V_i with identical sizes of C : $N = |V|/C$ and $|V_i| = C$, where $0 \leq i < N$ and $|\cdot|$ means the size of a set. A global operand $R = \log_2 C$ is used by any given vertex v to locate its belonging subgraph S_i , as formulated in Eq. 4.1:

$$i = v \gg R \quad (4.1)$$

Here, $\gg$ is the right shift operator. The locating of a subgraph is a critical operation in partition-centric paradigm, because the destination subgraph (containing the destination vertex) is needed when an update is propagated across subgraphs. This bit-aware propagating mechanism is detailed in Section 3.3.

An example is offered as in Fig. 4.4. The *old* array lists the subgraphs constructed in

the initial segmenting step. Assume 64 vertices are separated into 4 subsets in the order of vertex ID. Each subset contains 16 vertices, so the R equals 4. The vertex ID can be represented in 6-bit binary format¹. To locate the belonging subgraph of a vertex, only the leftmost 2 bits are needed; the rightmost $R = 4$ bits are ignored due to right shift. In other words, the coverage of each subgraph is expressed by the leftmost 2 bits of vertex ID. This is exactly the same as prior CSR-segmenting graph processing, where the sizes of all vertex subsets are equally fixed [93].

Step 2: Identifying Hot Subgraphs. With all subgraphs segmented with a uniform size, we measure the hotness of subgraphs by the sum of vertex degrees per subgraph. The average subgraph degree $\overline{D}^s$ is obtained by multiplying the average vertex degree over the entire graph $\overline{D}$ with the size of vertex subset: $\overline{D}^s = \overline{D} \cdot C$. It serves as a threshold. The hotness of a subgraph is evaluated by simply comparing its D_i^s with $\overline{D}^s$ as in Eq. 4.2. When $hot_i \geq 2$, the corresponding subgraph is identified as hot.

$$hot_i = D_i^s / \overline{D}^s \quad (4.2)$$

Step 3: Subdividing Hot Subgraphs. According to the hot_i of hot subgraphs, further subdivision is conducted. The number of sub-units n_i split from each hot subgraph is calculated by performing binary exponentiation and binary logarithm together as in Eq. 4.3:

$$n_i = 2^{O_i} = 2^{\lfloor \log_2(hot_i) \rfloor} \quad (4.3)$$

The $\lfloor \cdot \rfloor$ stands for the rounding of a number, and the rounded value for each subgraph is recorded as local offset $O_i = \lfloor \log_2(hot_i) \rfloor$ later used in Section 4.2.2. Eq. 4.3 forces n_i to be an integer that is a power of 2, e.g., 1, 2, 4 and 8, where $n_i = 1$ means no subdivision. By contrast, hot_i can be any floating number. As a result, a hot subgraph is subdivided into n_i sub-units, the size of each which is C/n_i .

For the example of Fig. 4.4, we further assume that S_0 and S_1 are hot subgraphs and can be subdivided into 4 and 2 sub-units respectively. Hence, S_0 is associated with $(n_0 = 4, O_0 = 2)$, S_1 is with $(n_0 = 2, O_0 = 1)$, and both S_2 and S_3 are with $(n_0 = 1, O_0 = 0)$. After the subdivision, new subgraphs, including the sub-units, are constructed as in the *new* array of Fig. 4.4.

4.2.2 Locating A Subgraph

The subdivision of hot subgraphs leads to an increase in the number of subgraphs, containing the sub-units with sizes $< C$. The varying size of subgraphs invalidates the bitwise operation for locating a subgraph by a given vertex ID, i.e., Eq. 4.1. One possible alternative is to

¹The subscript b denotes a binary number

use `if & else` or `switch & case` statements to exhaustively examine the coverage of each subgraphs until the correct one is found.

Nevertheless, those conditional statements are computationally inefficient. For example, they might cause a massive number of mispredicted branch instructions, which wastes the pipelines of CPU. Thereby, for computational efficiency, we advance the bitwise operation by *Anchoring & Chaining*:

Anchoring. A mapping table is maintained between the original subgraphs and the new subgraphs after subdivision. As in Fig. 4.4, every subgraph in the old array points to its first sub-units or itself (with new ID if not subdivided) in the new array. For the newly created subgraphs, we refer to the pointed elements as the *anchored* subgraphs and the remaining ones as the *chained* subgraphs. Hence, we can still rely on the leftmost 2 bits, which we label as anchor bits, to locate an anchored subgraph. For instance, given vertex ID $11xxxx_b$, where $\forall x \in \{0, 1\}$, shift it rightwards by $R = 4$ bits, and its anchor equals $11_b = 3$. Thus, it originally lies in S_3 , which is now pointing to $\tilde{S}_6$, forming a mapping relationship: $map(S_3, \tilde{S}_6)$ and $map[S_3] = \tilde{S}_6$.

Nonetheless, the mapping table only allows us to recognize a fraction of subgraphs (i.e., anchored subgraphs) in the new array. The chained subgraphs stay out of reach. For a complete solution, one more step is planned.

Chaining. To find the chained subgraphs in the new array, extra bits in the vertex ID are utilized. Recall that, in Step 3, an offset O_i is affiliated to each subgraph in the old array. It indicates the number of extra bits, labeled as *chain*, needed to scan right after the anchor bits.

For example, since S_0 is attached with $O_0 = 2$, it requires 2-bit anchor and additional 2-bit chain, together the leftmost 4 bits, to locate its sub-units. Given vertex ID $0001xx_b$, by its anchor 00_b , we identify that it is initially contained in S_0 , which now links to the anchored subgraph $\tilde{S}_0$. Then, according to the chain $01_b = 1$, we can locate the target subgraph by offsetting its anchored subgraph $\tilde{S}_0$ by 1, which is $\tilde{S}_1$.

In a compact form, Anchoring & Chaining can be mathematically described as following formulas:

$$anchor = v \gg R \quad (4.4a)$$

$$anchored_sub = map[anchor] \quad (4.4b)$$

$$chain = (1 \ll O_i - 1) \& (v \gg (R - O_i)) \quad (4.4c)$$

$$chained_sub = anchored_sub + chain \quad (4.4d)$$

Eq. 4.4a firstly finds the old subgraph before the subdivision is performed. Eq. 4.4b depicts the mapping procedure between the original subgraph and the anchored subgraph. Eq. 4.4c

shows the arithmetic operations of truncating the chain bits from a vertex ID. Finally, Eq. 4.4d locates the chained subgraph after subdivision.

It is worthy to note that, in Step 1, the optimal initial segmenting size for CSR segmentation does not always equate to the size of caches, e.g., neither L2 nor LLC. The exploration regarding the proper initial segmenting size is discussed in Section 4.3.6.

4.2.3 Bit-aware Propagation

The propagation of updates in Syze behaves as a variant of the classic Scatter-Gather model. A source (src) vertex firstly broadcasts its property data over its active neighboring subgraphs (Scatter). The data are stored in the buffers of these subgraphs. Then, the destination (dst) vertex collects all the received data from the local buffer of its belonging subgraph (Gather).

The propagating mechanism of Syze exhibits the uniqueness in its bit-awareness of vertex ID. During the scattering phase, the leftmost bits of the dst vertex are extracted to locate the dst subgraph. Also, during the gathering phase, an extra bit is inserted in front of the dst vertex to reduce communication between subgraphs.

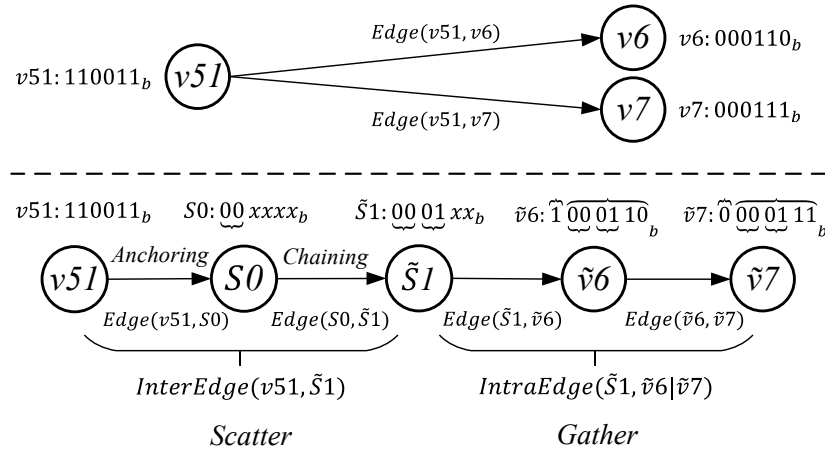


Figure 4.5: Bit-aware propagation of Syze

Fig. 4.5 explains the bit-aware propagation of Syze with a concrete example. The upper part of Fig. 4.5 presents a conventional message passing from one src vertex to two dst vertices. We assume the src vertex and dst vertices reside inside two different subgraphs (within two L2 caches) in the context of CSR-segmenting graph processing. Hence, the propagation of data along the edges would cross the private caches of cores, and traverse the LLC or even main memory, causing a high latency.

The lower part of Fig. 4.5 plots the data flow triggered by the same vertices in Syze. The Scatter and Gather phases are presented, which govern the transmission of updates. Notice that the dst vertices $v6$ and $v7$ only differ in one bit, so they are included in the same

subgraph $\tilde{S}_1$.

In the Scatter phase, according to composition of v_6 (i.e., anchor bits 00 and chain bits 01), the update reaches to the subgraph $\tilde{S}_1$ via S_0 . The edges (v_{51}, S_0) and $(S_0, \tilde{S}_1)$ are delineated conceptually. In actual implementation, they are enacted by Anchoring & Chaining, which form an inter-edge transmitting data between v_{51} and $\tilde{S}_1$. The linking procedure is detailed in Section 4.2.2.

In the Gather phase, the data buffered in $\tilde{S}_1$ is locally fetched by the dst vertices. The edge compression [93] is adopted for the reduction of memory traffic. Since v_6 and v_7 share the same src vertex v_{51} , the data fetched by v_6 is copied to v_7 , instead of transmitting again from v_{51} to v_7 . To achieve this, one compress bit is inserted in front of the dst vertices: $v_6 \rightarrow \tilde{v}_6$ and $v_7 \rightarrow \tilde{v}_7$. When compress equals one, the vertex (e.g., $\tilde{v}_6$) is responsible for collecting the data from local buffer. If the compress of next vertex is zero, then the data assigned to the previous vertex (e.g., $\tilde{v}_6$) is duplicated to the next vertex (e.g., $\tilde{v}_7$).

Similarly, the edges $(\tilde{S}_1, \tilde{v}_6)$ and $(\tilde{v}_6, \tilde{v}_7)$ are conceptual designs. They are actually implemented by a simple bitwise operation: examining the compress bit of vertex ID. This bit compresses two edges into one intra-edge. Following prior works [93], we employ the most significant bit (MSB) of vertex ID (32-bit unsigned integer) as the compress bit. The encoding of compress bit is performed only once during the segmenting of graphs, hence leading to no overhead during the propagating of messages.

The cooperation between the inter-edge and the intra-edge effectively reduces memory traffic at the (negligible) cost of additional computation. In the original propagation process (upper part of Fig. 4.5), two cross-core messages are transmitted. In comparison, in the lower part, the propagation of Syze compresses two cross-core messages into one inter-edge, and then locally sends out the message via one intra-edge.

4.2.4 Program Execution

Like preceding CSR-segmenting works [104, 101, 93], the iterative execution of Syze are decomposed into two separate parallel regions as outlined in Algorithm 6. The first region (line 14-17) includes the Scatter phase; the second covers the Gather phase (line 18-21). Besides, inheriting from GPOP [8], the Reset phase and the Apply phase are provided at the end of each region. They convey similar semantics, both which can update vertex data and control vertex state in the frontier (i.e., active or inactive). The cooperation of Reset and Apply improves the flexibility of expressing graph algorithms and reduces programming efforts.

During the execution, each phase corresponds to an interface calling a specific user-defined function. The example of PageRank is illustrated in line 1-12. Function `PR.scatter` returns the scaled rank scores of vertices (line 2), which is passed to `Scatter` for broadcast

Algorithm 6 Iterative execution of PageRank

```

1: Structure PR {                                     ▷ define a graph algorithm
2: Function scatter( $v$ )
3:   return  $rank[v] / deg[v]$ 
4: Function reset( $v$ )
5:    $rank[v] = 0$ 
6:   return true
7: Function gather( $v, update$ )
8:   return  $rank[v] += update$ 
9: Function apply( $v$ )
10:   $rank[v] = (1 - d) / |V| + d \cdot rank[v]$ 
11:  return true
12: }
13: while !converged & iter < max_iter do               ▷ run a task
14:   for all  $curr \in active\_subs$  do in parallel         ▷ region 1
15:     for  $next \in active\_subs$  do
16:       Scatter(PR.scatter, curr, next)
17:       Reset(PR.reset, curr)
18:   for all  $curr \in active\_subs$  do in parallel         ▷ region 2
19:     for  $next \in active\_subs$  do
20:       Gather(PR.gather, curr, next)
21:       Apply(PR.apply, curr)

```

from current subgraphs to neighboring subgraphs. Then, the rank scores of vertices are reset to zero and vertex states stay active with PR.reset (line 4) called by Reset. PR.gather (line 7) belonging to Gather accumulates the rank scores from the local buffers of active subgraphs. Finally, the rank scores are normalized by PR.apply (line 9) that is passed to Apply.

Apparently, the Scatter and Gather phases are the main source of workloads. They introduce sequential inner loops over all active subgraphs. Also, heavy memory traffic are invoked due to the data propagation across these subgraphs. On the contrary, the Reset and Apply phases merely perform on current subgraphs with basic arithmetic operations.

Meanwhile, Syze enjoys the hierarchic frontier of GPOP too, which maintains the activity states of graph components [8]. The frontier consists of two levels that record the states of subgraphs and vertices respectively. A subgraph turns active if it accommodates at least one active vertex. The active subgraphs and vertices are filtered out by the hierarchic frontier. Hence, for dynamic graph applications, such as BFS, the access to the inactive is avoided.

4.2.5 Benefits and Costs

The most prominent difference between Syze and prior CSR-segmenting works lies in the segmenting procedure of graphs. In Syze, a graph is unequally cached. Hot subgraphs are identified and further subdivided into smaller units. By contrast, previous methods only deploy a equal-sized segmenting strategy where the sizes of all segments are equivalent, which is just the first step of Syze’s unequal caching strategy.

The benefits of unequal caching strategy involves two aspects. Firstly, the workloads amongst threads are balanced, since hot subgraphs are split into smaller ones. This effectively shortens the longest threads execution time, and thus reduces the overall graph processing time (see Section 4.3.4). Secondly, the thread contention is alleviated. Thanks to the subdivision of hot subgraphs, threads (i.e., logical cores) are assigned with smaller data unit to process. As a result, the computing pressure of processor cores are relieved, and therefore the exhaustion of resources are postponed (see Section 4.3.4).

Though Syze improves the performance of processor cores and accelerates graph processing, it imposes side effect on the memory-cache hierarchy in multicore systems. The subdivision creates extra subgraphs, which leads to more inter-edges. Moreover, the Anchoring & Chaining procedure requires supplementary memory space for the mapping table and additional arithmetic operations. Consequently, memory traffic is raised and cache misses are increased (see Section 4.3.4).

Another distinct feature of Syze is that it employs more threads for multithreading than prior CSR-segmenting methods, e.g., 36 vs 20. Syze takes the advantage of relieved thread contention by increasing the size of thread pool. Therefore, the available hardware resources are better exploited. The benefit of adding more threads relies on the unequal caching strategy, and is relatively minor (see Section 4.3.5).

4.3 Evaluation

In this section, dual evaluations combining theoretical calculations and experimental observations are undertaken. Firstly, the intrinsic characteristics of graphs are profiled with respect to skewness and imbalance, which theoretically affect the effectiveness of Syze.

Then, the performance of Syze is evaluated by comparing with state-of-the-art graph processing frameworks. We also investigate the impact of Syze on processor cores as well as cache and memory. Furthermore, Syze is decomposed for a better understanding of its contribution.

4.3.1 Estimating Imbalance

The power-law degree distribution (i.e., skewness) of graphs does not necessarily guarantee the graph are imbalanced as well. It is possible that the hot vertices, though being a minority, are evenly distributed throughout the graph; that is, in the context of equal-sized CSR segmentation, subgraphs contain comparable hot vertices.

To estimate the imbalance of a graph, the subdivision factor n_i from Eq. 4.3 is utilized, which is the number of sub-units to be subdivided from a hot subgraph. It indicates the density of subgraphs before further subdivision. The larger the value of n_i is, the more hot vertices are concentrated in the corresponding subgraph, and hence the more imbalanced the graph is. For an imbalanced graph, it is expected to consist of a tiny portion of hot subgraphs with $n_i \gg 1$ and a large portion of cold subgraphs with $n_i = 1$. In opposition, when $n_i = 1$, a subgraph needs no subdivision. If all subgraphs of a subgraph are associated with $n_i = 1$, the graph is perfectly balanced.

Table 4.2: Graph features in terms of the hot vertices percentages (V), the edge coverage of hot vertices (E), the hot subgraphs percentage (H), and the maximum n_i .

Graphs	Description	Skewness		Imbalance	
		V (%)	E (%)	H (%)	$\max(n_i)$
<i>track</i>	Web Tracker[105]	18	63	2	128
<i>live</i>	Live Journal[106]	22	91	2	32
<i>wiki</i>	Wiki Link[107]	19	94	4	32
<i>twitter</i>	Twitter Follower[50]	9	79	2	64
<i>mpi</i>	Twitter Influence[51]	11	81	3	128
<i>kron</i>	Synthetic Graph[46]	8	93	0	1
<i>urand</i>	Synthetic Graph[46]	51	59	0	1

Table 4.2 lists a diversity of graphs collected from the real world or generated by software tools. Graphs are equally segmented into subgraphs in which the vertex subsets are of size 256 KB. Their skewness is indicated by (1) the ratio of hot vertices over all vertices and (2) the percentage of hot vertices' edges over all edges. Their imbalance is portrayed by (1) the ratio of hot subgraphs over all subgraphs and (2) the maximum n_i of all subgraphs.

The real-world graphs exhibit high skewness as well as high imbalance. A minority (9 – 22%) of vertices are responsible for constituting a majority (63 – 94%) of edges. Meanwhile, their hot subgraphs, in spite of being rare (2 – 4%), are crowded with hot vertices, delivering n_i up to 128.

The synthetic graphs *kron* and *urand* are well balanced (i.e., $n_i = 1, \forall i \in [0, N)$,) regardless of their skewness. Though *kron* follows the power-law degree distribution (i.e.,

9% vertices for 93% edges), its hot vertices are evenly distributed as none of its subgraphs requires subdivision. In *urand*, a connection is established by any two vertices with equal probability. Thereby, *urand* is absolutely unskewed and balanced.

The statistical summary of graphs in Table 4.2 demonstrates that the skewness of a graph does not promise its imbalance. It is possible that a highly skewed graph has a uniform distribution of hot vertices across its subgraphs, such as *kron*. In such case, subgraphs will not be further subdivided, which cancels out the optimization of Syze. In theory, Syze cannot boost the performance of balanced graphs. Hence, in following sections, we only conduct experiments on the imbalanced graphs.

4.3.2 Experiment Setup

The experiments are performed on a dual-processor machine. The model of processor is Intel Xeon Silver 4210 with Skylake microarchitecture. One processor is composed by 10 physical cores and 20 threads with Hyper-Threading enabled. The volume of main memory, shared LLC, private L2, and private L1 are 256 GB, 13.75 MB, 1MB and 64 KB respectively. The operating system is Ubuntu 20.04.

Syze is implemented by modifying the source code of GPOP [8]. The code is compiled is written in C++ with g++ 9.3.0 and compiled with optimization level O3. The multithreading is enabled by the use of OpenMP [108]. The cache utility and the memory dynamics are monitored by Perf [95] and Likwid [96] respectively.

The initial segmenting size is tuned to 256 KB, a quarter of L2 cache (explained in Section 4.3.6). 36 threads are used for the parallelization of graph algorithms (explained in Section 4.3.4). Each experiment is repeated for 10 rounds, and the average result is reported. For each round of PageRank, program runs for 20 iterations.

Graph Frameworks. The performance of Syze is compared with three cutting-edge frameworks: Ligra [4], GPOP [8] and Gemini [19]. To obtain the best results, Ligra is multithreaded by OpenCilk [109] with the optimization of numactl. The native Ligra parallelized via OpenMP yields substantially poorer performance and therefore is not demonstrated.

For the partition-centric GPOP, we also fine-tune its performance by setting 20 threads and 256KB partition size. Although developed for distributed systems, Gemini draws considerable inspiration from shared-memory graph processing, such as NUMA awareness and hybrid push-pull engine. It exhibits superior performance than many shared-memory frameworks on single machine [19]. In addition, we omit the comparison with Polymer [5] and Grazelle [9] because their results are far inferior to other competitors on our two-NUMA-node machine.

Graph Algorithms. Under these graph frameworks, four graph algorithms are tested.

PageRank (PR) ranks the importance of vertex based on their connectivity [85]. Breadth-first search (BFS) traverses a graph from a given root vertex in breadth-first order. Single source shortest path (SSSP) finds the shortest path between a given root vertex and all other reachable vertices. Connected component (CC) classifies connected vertices into components using label propagation.

It is reported that [8], GPOP exhibits dominant advantage in PR, CC, SSSP thanks to its aggressive caching technique, compared to state-of-the-art works (e.g, GraphMat [82] and Galios [110], Ligra [4]). Meanwhile, Ligra offers the fastest implementation for BFS. Also, Gemini oftentimes delivers better performance than Ligra and Galios [19] on a single machine.

Due to limited space, our discussions focus on PageRank and BFS. They behave oppositely as the two ends of a spectrum. In our implementation, PageRank is exempted from the deployment of frontier because every vertex is updated in very iteration. It enables a system to run at the peak performance. Also, the examination of convergence is removed in all frameworks, so the accurate performance per iteration is obtained.

In contrast to PR, BFS is completely driven by frontier. During the whole processing, a vertex is accessed for only once. Therefore, BFS introduces lightweight communication and computation. Additionally, for root-dependent algorithms, the vertex with the largest degree is selected as the input to start graph traversal.

Table 4.3: Graph Information (K: Thousand, M: Million, B: Billion).

Graphs	Vertices	Edges	Max Deg.	Refer.
<i>track</i>	27.7M	140.6M	1.1M	[105]
<i>live</i>	7.5M	112.3M	300	[106]
<i>wiki</i>	18.3M	172.1M	9300	[107]
<i>twitter</i>	41.7M	1.5B	3.0M	[50]
<i>mpi</i>	52.6M	2.0B	779K	[51]

Graph Datasets. Five large-scale real-world graph datasets are selected for performance evaluation. Graphs *wiki* is extracted from the hyperlinks of wikipedia [107]. *track* collects web domains and their embedded tracker [105]. *live*, *twitter* and *mpi* are social graphs modeling social networks [106, 50, 51]. The numerical information of the graphs are provide in Table 4.3.

4.3.3 Execution Time

Table 4.4 lists the execution time for the four graph algorithms implemented under Syze and cutting-edge frameworks. Syze constantly outperforms other competitors in all analytics

algorithms except for BFS. Syze achieves speedups over Ligra, Gemini and GPOP by up to $17.76\times$, $11.67\times$ and $2.81\times$, and on average $3.34\times$, $2.55\times$ and $1.80\times$ respectively. Nevertheless, it worth mentioning that Syze is theoretically ineffective at the intrinsically balanced graphs as previously analyzed in Section 4.3.1.

Table 4.4: Graph processing time (in seconds) for graph algorithms under different graph frameworks.

PageRank						Connected Component					
Frwk.	<i>live</i>	<i>mpi</i>	<i>track</i>	<i>twitter</i>	<i>wiki</i>	Frwk.	<i>live</i>	<i>mpi</i>	<i>track</i>	<i>twitter</i>	<i>wiki</i>
Syze	0.866	7.990	2.105	5.562	0.888	Syze	0.231	0.679	0.466	1.153	0.174
GPOP	2.193	17.915	2.753	9.897	1.542	GPOP	0.650	1.587	0.521	1.484	0.409
Ligra	15.369	92.885	10.949	40.895	7.240	Ligra	1.122	5.660	2.096	2.814	0.790
Gemini	1.602	22.573	2.826	10.767	2.677	Gemini	0.372	7.916	0.915	5.326	1.009
Single Source Shortest Path						Breadth-First Search					
Frwk.	<i>live</i>	<i>mpi</i>	<i>track</i>	<i>twitter</i>	<i>wiki</i>	Frwk.	<i>live</i>	<i>mpi</i>	<i>track</i>	<i>twitter</i>	<i>wiki</i>
Syze	0.249	1.688	0.094	1.410	0.266	Syze	0.090	0.827	0.065	0.829	0.143
GPOP	0.652	2.906	0.093	1.661	0.497	GPOP	0.228	1.432	0.070	1.079	0.206
Ligra	0.269	3.037	0.123	3.002	0.501	Ligra	0.054	0.526	0.050	0.408	0.152
Gemini	0.305	3.259	0.461	1.813	1.149	Gemini	0.339	1.063	0.649	0.805	1.612

For PR, CC and SSSP, Syze provides the fastest results on most of datasets, except SSSP on *track*. GPOP, which is also partition-centric but with equal segmenting size, performs as the second fastest framework. The partition-centric paradigm (e.g., Syze and GPOP) shows considerable advantages over the vertex-centric paradigm (e.g., Ligra and Gemini) on shared-memory multicore systems. This attributes to the aggressive cache and memory optimizations in the former, including cache-aware graph partitioning [52] and edge compression [93].

For BFS, Ligra performs as the best framework. We deliberately fine-tune Ligra to maximize its performance. Cilk is employed by Ligra for multithreading, which is substantially faster than the OpenMP-based Ligra. The hybrid push-pull processing engine and the lightweight vertex frontier plus the NUMA-awareness significantly shortens the execution time for BFS. On the contrary, Syze is parallelized by OpenMP. It is neither optimized in the propagation direction nor aware of NUMA. Also, the hierarchic frontier adopted by Syze imposes an extra burden of filtering subgraphs. Nevertheless, Syze still acts as the second fastest frameworks for BFS, offering performances comparable to Ligra.

Syze far outperforms GPOP due to the unequal caching strategy deployed on the imbalanced graphs. The hot subgraphs are identified and subdivided into smaller units.

However, the subdivision inevitably induces more subgraphs and complicates the connectivity amongst them. Also, the bit-wise operations Anchoring & Chaining lead to additional computations that may affect the overall performance. In the next section, the benefits and costs of Syze are elaborated on an experimental basis.

4.3.4 Impact on Multicores

Following our motivation in Section 4.1, we investigate the impacts of Syze on the multicores in two aspects: (1) workloads balance and (2) parallel efficiency. We also analyze the side effect of Syze in terms of cache and memory utilities. Moreover, the cost of Anchoring & Chaining is measured. Syze is benchmarked against a degraded version of itself that is deprived of any optimizations. In other words, the comparing baseline does not allow subdivision of hot subgraphs and only utilizes 20 threads, which is the same as prior partition-centric frameworks.

Workload Balance

The workload distribution amongst multicores is profiled by the execution time of threads. The thread with the *longest* completion time is evaluated, since it is decisive in a parallel region. Also, the majority of threads is represented by the *average* execution time of all threads.

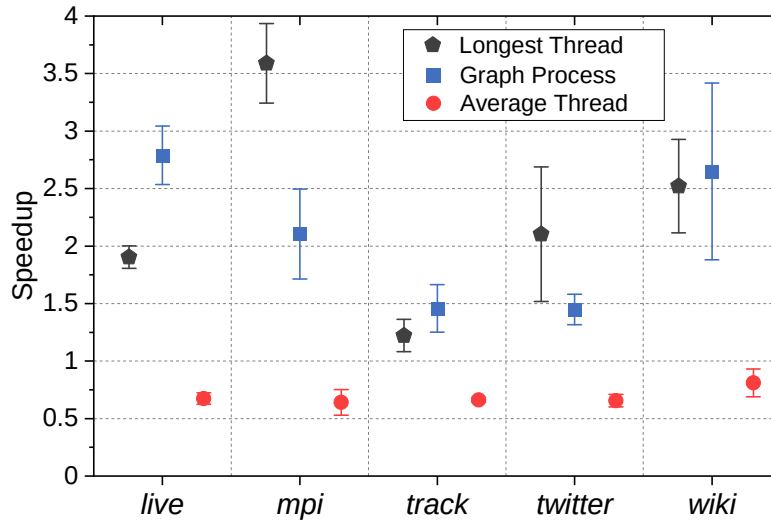


Figure 4.6: The speedup of the graph processing, the longest thread and the average thread over the baseline. The results stand for the mean of different graph algorithms with 80 percent confidence interval.

Fig. 4.6 illustrates the speedup of Syze over the baseline in terms of graph processing,

corresponding longest thread and average thread. Both the overall process and the longest thread (i.e., the most imbalanced workload) achieve remarkable accelerations. The longest thread time is reduced by $1.02 - 4.07\times$ (on average $2.27\times$) over all trials. Meanwhile, the speedup of graph processing is $1.20 - 4.20\times$ (on average $2.70\times$). The reduction of the longest thread time demonstrates the effectiveness of Syze in balancing workloads.

By contrast, the majority of threads are decelerated since the average speedup is $0.51 - 1\times$ (below 1 indicting a slowdown). This validates that it is the longest-running executor determining the overall processing time of a parallel region, instead of the majority. Moreover, the slowdown of the average thread encapsulates the cost of Syze, including the compromised efficiencies of memory and cache.

Parallel Efficiency

Besides workload distribution, Syze also causes drastic variability in the parallel efficiency of multicores. Fig. 4.7 shows the performance of Syze and the baseline by running PageRank on graph *twitter*. The parallel efficiencies of physical cores PE_P and logical cores PE_L are presented with varied number of working threads. Also, the execution time for PageRank is normalized by the single-threaded PageRank. The normalized results (i.e., speedups against the single thread) demonstrate the scalability of Syze. It can be discovered that PE_P , rather than PE_L , constrains the speed of graph processing.

Syze exhibits stronger scalability than the suboptimal baseline. As in Fig. 4.7b, when thread number > 20 , the performance of the baseline starts to decline in terms of PE_P and scalability. The baseline behaves almost the same as GPOP in Fig. 4.2. On the contrary, the saturation of Syze is deferred. It achieves the highest PE_P at 40 threads and the highest speedup at around 36 threads. Multithreading with 36 out 40 threads serves as a good heuristic for all tested graph algorithms. The leftover capacity (e.g., 4 threads) is considered to be occupied by the background processes initiated by OS.

Syze improves PE_P and PE_L by up to 20 percent in comparison with the baseline. The improved parallel efficiency significantly contributes to the high performance of Syze. Against the single-threaded implementation, Syze with 20 threads and 36 threads deliver speedups of $9.85\times$ and (the highest) $10.70\times$ respectively. In comparison, the baseline achieves the highest speedup by only $5.99\times$ with 20 threads.

Cost of Syze

Though Syze brings substantial accelerations to graph tasks, it also causes various side effects on the multicore systems. Hence, additional multicore activities are monitored, such as memory accesses and L2 cache misses. To better illustrate the cost of Syze, we report

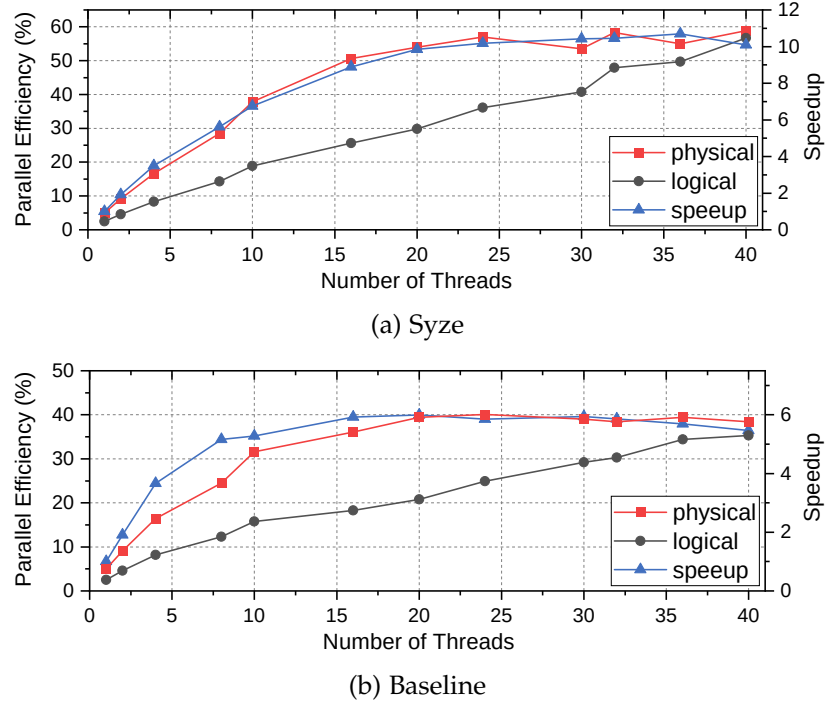


Figure 4.7: The speedups of Syze and baseline and the parallel efficiencies of physical cores and logical cores, by performing PageRank on *twitter*.

these metrics using $performance\ decline = \{\text{Result of Syze}\} / \{\text{Result of Baseline}\}$. The larger the value, the worse performance Syze incurs.

The performance deterioration related to memory and cache are plotted in Fig. 4.8. The average thread (defined in Section 4.3.4) is presented as well. Its performance decline equals the inverse of its speedup depicted in Fig. 4.6. The deployment of Syze raises memory accesses as well cache misses, which results in the slowdown of the average thread.

The increased memory accesses and cache misses origin from the newly created sub-graphs. As listed in Table 4.5, for Syze, the subdivision of hot subgraphs causes an upsurge of sub-units (i.e., smaller subgraphs) as well as inter-edges connecting these subgraphs. Hence, cross-subgraph communications are raised, which consequently leads to higher memory traffic.

Also, Syze incurs additional cost in locating the subgraphs of vertices. For the baseline, the Chaining procedure is no more needed for locating the subgraphs due to its equal-sized segmenting; that is, Eq. 4.4b, 4.4c and 4.4d are not performed by the baseline. In comparison, Syze requires complete Anchoring & Chaining for every vertex to locate its belonging subgraphs. Thus, Syze involves heavier computations and memory usage. Nonetheless, the overheads of locating the subgraphs are minor. As listed in Table 4.5, it

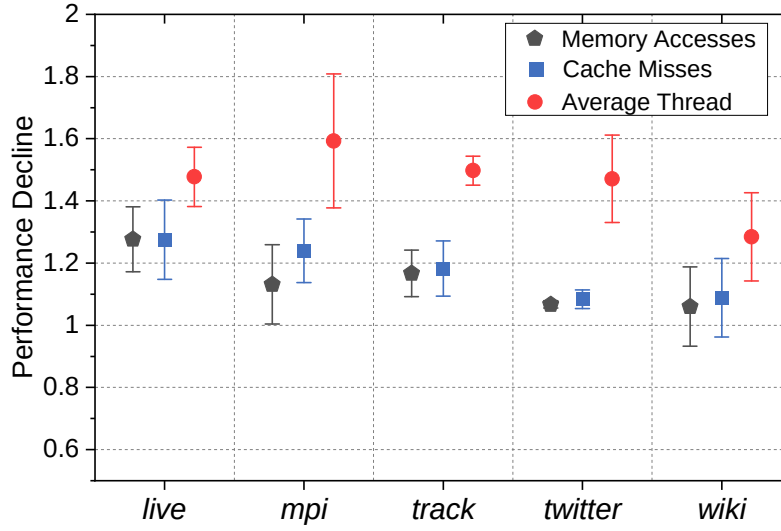


Figure 4.8: The performance decline of the memory, the cache and the average thread compared with the baseline. The results stand for the mean of different graph algorithms with 80 percent confidence interval.

costs less than $0.3ms$ for any graphs to locate the subgraphs of all vertices.

Table 4.5: The numbers of subgraphs and inter-edges (in Million), and the time for locating the subgraphs of all vertices (in Milliseconds) by Syze and the baseline

		<i>live</i>	<i>mpi</i>	<i>track</i>	<i>twitter</i>	<i>wiki</i>
Subgraphs	Syze	140	1055	566	747	350
	Baseline	115	803	423	636	279
Inter-Edges	Syze	40.70	888.44	69.28	498.78	83.44
	Baseline	18.08	548.76	45.00	437.36	56.42
Time Cost	Syze	0.042	0.209	0.109	0.146	0.073
	Baseline	0.039	0.16	0.094	0.12	0.063

Given the costs in the memory-cache hierarchy, Syze still delivers remarkable acceleration to the overall graph processing, which is on average $2.70\times$. This is because the gain from workload balancing and effective parallelism far outweigh the loss from cache and memory inefficiencies. For instance, comparing Fig. 4.6 with Fig. 4.8, it can be observed that the speedup of the longest thread (on average $2.27\times$) is much higher than the performance decline of memory (on average $1.14\times$) and cache (on average $1.17\times$). In the next section, we explicitly quantify the respective contributions of balanced workload and improved parallelism.

4.3.5 Decomposition of Syze

The design of Syze differs from existing partition-centric paradigm in two distinct features. (1) Dynamic graph caching: the segment sizes are diversified due to the subdivision of hot subgraphs. Contrarily, prior works equally segment a graph into vertex subsets of the same size. (2) Optimized thread utilization: we exploit higher utilization of threads by deploying 36 out of 40 available threads. Other partition-centric methods, however, only employs half of available threads (e.g., 20) for task parallelization to prevent saturation.

To understand the contributions of the two features, the design of Syze is disintegrated and the components are evaluated separately. We employ Syze without unequal caching and with 20 threads as the plain baseline. Then, the baseline is compared with three variants of Syze. (1) Syze without unequal caching but with 36 threads, denoted as Syze-equal. (2) Syze with unequal caching and half-size thread pool, e.g, 20 out of 40 threads, denoted as Syze-half. (3) The fully functional Syze with unequal caching and 36 threads. The comparison results are presented in the form of speedup over the baseline as in Fig. 4.9.

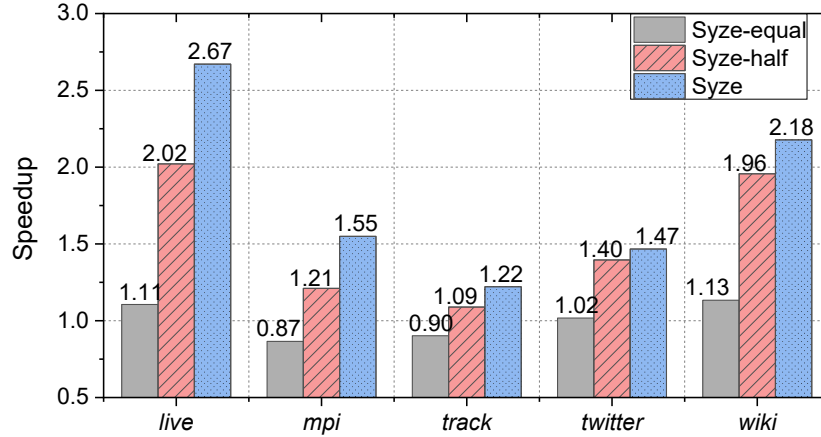


Figure 4.9: The speedups delivered by Syze and its variants in comparison with the baseline, by performing BFS on *twitter*.

Fig. 4.9 demonstrates that the unequal caching strategy acts as the main contributor to the effectiveness of Syze. Without the unequal solution, Syze-equal oftentimes causes a slowdown with speedup $< 1\times$. Simply adding more threads into the parallelization of a graph tasks is prone to deteriorate the processing speed. When the unequal caching is adopted by Syze-half, the performance immediately receives a substantial boost. Based on Syze-half, the addition of threads further accelerates graph processing, though the improvement from Syze-half to Syze is relatively minor.

On the average of all graph algorithms and datasets, Syze-equal, Syze-half and Syze achieves speedups over the baseline by $1.06\times$, $1.76\times$ and $2.09\times$ respectively. The whole is

greater than the sum of the parts. Also, the unequal caching strategy is ranked with higher importance than the large size of thread pool.

4.3.6 Initial Segmenting Size

To acquire the optimal size for the initial segmentation of graphs, a sensitivity analysis is conducted. The performance of Syze is examined by setting different initial segmenting size. As shown in Fig. 4.10, the results are normalized by the execution time of 16 KB.

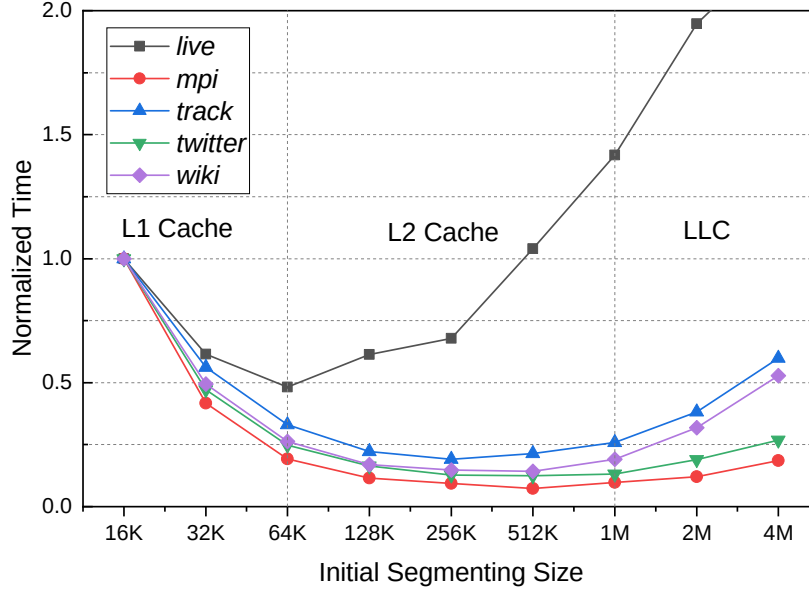


Figure 4.10: The normalized execution time of Syze with different initial segmenting size, by performing BFS.

The majority of graphs achieve the best performance when their initial segmenting sizes range *within* L2 cache: $L1 \text{ cache size} < \text{segmenting size} < L2 \text{ cache size}$. If the vertex subset is too small (i.e., fitting into L1 cache), there will be an upsurge in inter-edges and computational loads. Oppositely, if the size is too large, subgraph will spill over into LLC or even main memory, resulting in low cache locality and high memory traffic. In such case, the performance of Syze is drastically deteriorated.

Graph *live* behaves abnormally because its small size of vertex set. If segmented by 256 KB, *live* is subdivided into 149 subgraphs. On average, each thread is allocated with approximately 4 subgraphs. It is insufficient to fully exploit OpenMP's dynamic scheduler as threads might have to wait idly without new tasks.

Consequently, we choose 256 KB as the basic segmenting parameter at the stage of initial segmentation. Furthermore, a sufficient number of subgraphs N is required to facilitate

dynamic scheduling policy. Thereby, we set $N \geq 8 \cdot T$, where T stands for the size of thread pool. For small-scale graphs, the initial segmenting size is halved consecutively until this condition is fulfilled.

4.4 Summary

In this chapter, we propose an unequal caching strategy for high-performance shared-memory graph analytics, namely Syze. The proposal of Syze is motivated by the in-depth analyses of existing CSR-segmenting graph processing works. We experimentally demonstrate that, due to the deployment of equal-sized CSR segmentation, processor cores suffer from workload imbalance and parallel inefficiency.

To tackle these issues, Syze identifies the hot subgraphs, and then subdivide them into smaller units. To trace the belonging subgraph of a vertex, the bitwise operations Anchoring & Chaining are designed, which efficiently utilize the individual bits of vertex IDs. In this sense, updates of data are propagated with the awareness of vertex bits.

Syze is effective at graphs characterized by inherent imbalance, the feature of which can be theoretically calculated. Running on multicore systems, Syze is able to optimize the utilization of processor cores by balancing the workloads amongst logical cores (i.e., threads) and by improving the parallel efficiency of physical cores. As a penalty, Syze places additional burdens on the cache and memory. Fortunately, the performance boost from the multicores far exceeds the cost from the memory-cache hierarchy. In consequence, Syze delivers substantial speedup over fine-tuned contemporary frameworks.

Chapter 5

Hierarchical Partitioning

PageRank, weighing the importance of vertices in a graph, serves as a fundamental algorithm for graph-structured tasks in a variety of domains. However, the processing capacity of multicore systems is oftentimes poorly utilized for large-scale PageRank due to the irregular memory accesses and poor cache efficiency. In this chapter, we present HiPa, a novel hierarchical partitioning methodology to accelerate PageRank by utilizing the memory-cache hierarchy of the multicore system [42].

5.1 Background

5.1.1 PageRank

Analyzing graphs in depth, PageRank is one of the most popular graph algorithms. It was initially proposed to measure the importance of a webpage for web search engines [85].

$$PR_{new}(v) = \frac{1-d}{|V|} + d \times \sum_{u \in E_{in}(v)} \frac{PR_{old}(u)}{|E_{out}(u)|} \quad (5.1)$$

The mathematical description of PageRank is presented in Eq. 5.1. d is a damping factor; $|V|$ is the total number of vertices in a graph; u are the in-neighbors of v ; $|E_{out}(u)|$ is the number of outgoing edges of u . During the processing, the adjacency matrix of the graph is iteratively multiplied with an old rank vector to obtain a new rank vector. The mathematical representation of a skewed graph is a sparse adjacency matrix. Therefore, the computation of PageRank can be interpreted as iterative sparse matrix-vector multiplications (SpMV) [93]. Our discussions and optimizations proposed for PageRank can also be applied to SpMV as well as other graph algorithms.

5.1.2 NUMA Architecture

Modern server-class machines are typically featured with NUMA memory design. In these machines, a single motherboard utilizes more than one processors (i.e., sockets or CPUs). Each processor consists of multiple cores. The cores are connected with a local memory via a local bus, which together compose a NUMA node. The NUMA nodes are bound by a interconnect, finally building a multicore system [111].

Though the local memory of every NUMA node is globally shared with other nodes, there exists performance variability when a processor accesses different memory locations. The fastest speed is achieved if a processor interacts with its local memory within the same NUMA node. However, if a processor needs to reach out to a remote memory inside another NUMA node, the request must go through the remote memory controller of that node [103]. The indirect access to remote memory causes high memory latency, therefore leading to severe performance degradation. For instance, on a multicore systems with two Intel Xeon Silver 4210 CPUs (i.e., NUMA nodes), it costs a core 0.06 seconds to sequentially read 1GB data within its local memory, but 0.40 seconds for the remote one.

To exploit the benefits of NUMA architecture, a genre of NUMA-aware graph analytics frameworks and techniques are proposed [5, 19, 10]. In general, these approaches partition the graph into several subgraphs and allocate them to NUMA nodes. A balanced partitioning strategy (e.g., by edges or vertices) is required to promote workload balance among the NUMA nodes. Various optimization techniques are utilized, including NUMA-aware data layouts [5] and work-stealing scheduler [10]. The NUMA-awareness is typically implemented in vertex-centric graph systems. To the best of our knowledge, all existing partition-centric works are NUMA-oblivious [52, 94, 8].

5.1.3 Partition-Centric Graph Processing

The essence of partition-centric paradigm lies in the *cache-able* disjoint partitions of a graph. All vertices of a graph are segmented into vertex subsets, the sizes of which equal the size of caches. When a graph is processed, the vertex subsets are taken as the input by threads for user-defined functions (e.g., PageRank).

The optimal partition size is empirically determined based on the characteristics of cache hierarchy. The cache hierarchy of a multicore system typically consists of three levels: level 1 (L1) cache, level 2 (L2) cache, and last level cache (LLC). Among them, L1 cache provides the fastest speed but the smallest storage, which is exactly opposite to LLC. As a compromise between speed and storage, L2 cache is widely selected as the partitioning basis for partition-centric graph frameworks [94, 93, 8].

By restricting each core to access a subset of graph vertices within its local L2 cache, the spatial locality is significantly improved as accesses to remote vertices are eliminated.

Besides the vertex subset, each partition also includes a edge subset. If a edge travels from a source vertex in one partition to a destination vertex in another partition, it is labeled as the inter-edge. Inter-edges incur core-to-core communication through the LLC or even the memory, which causes high latency. In comparison, when the source and destination vertices of a edge reside in the same core, this edge is labeled as the intra-edge. Intra-edges indict that the operations on vertices occur in local caches, which benefits the performance.

5.2 NUMA-Aware Partitioning

Inspired by the NUMA characteristics of multicore systems and the partition-centric graph analytics, we proposed HiPa, a hierarchical partitioning methodology that leverages the memory-cache architecture. The abstraction of HiPa is illustrated in Fig. 5.1. It is designed with the goals of reducing remote memory accesses and improving cache efficiency. The details are described in following sections.

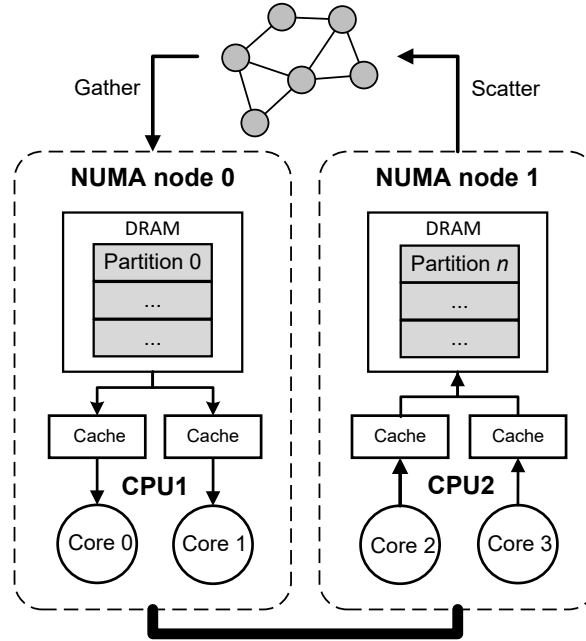


Figure 5.1: Conceptual Design of HiPa. HiPa applies hierarchical partitioning to a graph based on the characteristics of NUMA memory and private cache. Node 1 accumulates data in Gather phase for processing, and node 2 sends out updated data in Scatter phase.

The key to utilize a NUMA machine is co-locating the computation and data within the same NUMA node. Improper data allocation strategy might cause high overhead and workload imbalance for parallel graph processing, thus deteriorating the performance [5]. Therefore, it is a crucial task to partition the graph components, including vertices and

edges, for the NUMA nodes.

When partitioning a graph, an intuitive idea is to evenly allocate vertices over the NUMA nodes. Define the graph as $G = (V, E)$, where V is the vertex set and E is the edge set. The disjoint vertex subsets V_i allocated to N NUMA nodes have the identical size of $|V|/N$, where $1 \leq i \leq N$. It should be emphasized that the vertex subsets preserve the vertex order as in the original graph, and fulfill such conditions: $\cap_{i=1}^N V_i = \emptyset$ and $\cup_{i=1}^N V_i = V$. The same rules apply to edges too. Nevertheless, for graph algorithms where all graph edges are engaged in the processing, e.g., PageRank, the computation complexity mainly relies on the edges instead the vertices. For the skewed graphs, the even allocation of vertices leads to workload imbalance, thus slowing down the computation.

To cope with the above issue, previous works prefer to prioritizing edges for balanced partitioning [5, 10]. Specifically, each NUMA node is allocated with the same number of edges $|E|/N$. Thereby, the vertices are split into several subsets V_i with varied size $|V_i|$ holding these edges. In other words, the sum of vertex degrees $\sum_{v \in V_i} D(v)$ in every subset equals $|E|/N$, where v is a vertex and $D(v)$ is the degree of it. The allocations of edges and vertices on the i^{th} NUMA node can be mathematically described as follows:

$$\begin{aligned} |E_i| &= \frac{|E|}{N} \\ V_i &= \{v \in V | \sum_{v \in V} D(v) = \frac{|E|}{N}\} \end{aligned} \quad (5.2)$$

Enlightened by the predecessors, HiPa also adopts the edge-oriented partitioning approach, but at a *coarser* granularity of cache-able vertex subsets, for instance, L2-cache-sized partitions (L2-partitions). The number of vertices allocated to a NUMA node must be a multiple of L2-partitions. The size of a L2-partition P is fixed to $|P| = \{\text{L2 cache size}\} / \{\text{single vertex size}\}$, and the resized vertex subset is denoted as $\tilde{V}_i$. Then, the number of vertices assigned to each NUMA node can be written as $|\tilde{V}_i| = n_i \cdot |P|$, where n_i is the least succeeding integer of $|V_i|/|P|$. Due to the resizing of vertex subsets, the corresponding edge subsets are resized too, which is denoted as $\tilde{E}_i$. Based on Eq. 5.2, the number of edges and vertices reallocated to the i^{th} NUMA node can be formulated as follows:

$$\begin{aligned} |\tilde{V}_i| &= \text{ceil}\left(\frac{|V_i|}{|P|}\right) \cdot |P| = \left(\frac{|V_i| - 1}{|P|} + 1\right) \cdot |P| = n_i \cdot |P| \\ |\tilde{E}_i| &= \sum_{v \in \tilde{V}_i} D(v) \end{aligned} \quad (5.3)$$

In addition, the last NUMA node is an exception, which accommodates the leftover vertices and edges from other nodes. Also, the edges used for partitioning are either in-edges or out-edges. In this chapter, the out-edges are selected for demonstration.

5.3 Cache-Aware Partitioning

The NUMA-aware partitioning is aimed to minimize cross-node memory traffic and workload imbalance. Inside each NUMA node, the partitioning methodology is further refined.

In conventional partition-centric paradigm, threads handle the L2-partitions (the partitions with size equivalent to L2 cache) on a many-to-many basis. Any thread can arbitrarily access any partition. When a graph is being processed, the threads iterate over all graph partitions according to the first-come-first-serve policy. Threads have to contend for the ownership of a partition for exclusive processing within the private cache.

To avoid the potential contention of threads on the same partition, HiPa deploys a one-to-many model to the relationship between the threads and the partitions. Given Eq. 5.3, the i^{th} NUMA node contains n_i partitions. These partitions are distributed to local cores in groups. Each group G contains the same number of edges, which applies edge-oriented partitioning again. When the graph is being processed, the access of a thread is confined to its belonging group of partitions. For example, assuming the i^{th} NUMA node consists of C cores, every thread (i.e., working core) in this node is assigned with a group of m_j partitions, where $1 \leq j \leq C$. Following conditions are satisfied:

$$\begin{aligned} n_i &= \sum_{j=1}^C m_j \\ |G_j| &= m_j \cdot |P| \\ \frac{|\tilde{E}_i|}{C} &= \sum_{v \in G_j} D(v) \end{aligned} \tag{5.4}$$

In brief, the partitioning methodology of HiPa imposes a two-level constraints on the working cores. Firstly, a graph is coarsely partitioned based on the NUMA design of memory, such that graph data can be processed by local processors. Secondly, within individual NUMA node, the accessibility of cores are further restricted to a group of cache-able partitions to eliminate thread contention.

Fig. 5.2 exemplifies the partitioning result of HiPa. Assume a graph can be subdivided into 7 cache-able partitions with equal number of vertices. In the first stage, the first five partitions and the last two are allocated to the NUMA node 1 and 2 respectively for workload balance (e.g., $n_1 = 5$ and $n_2 = 2$). Then, in the second stage, the first 3 sparse partitions are separated and fed into core 0. The results of cache-level partitioning are $m_1 = 3, m_2 = 2, m_3 = 1$, and $m_4 = 1$. Also, to deal with real-world graphs that cannot be precisely partitioned according to Eq. 5.4, we loosen the condition by allowing: $\sum_{v \in G_j} D(v) \geq |\tilde{E}_i|/C$.

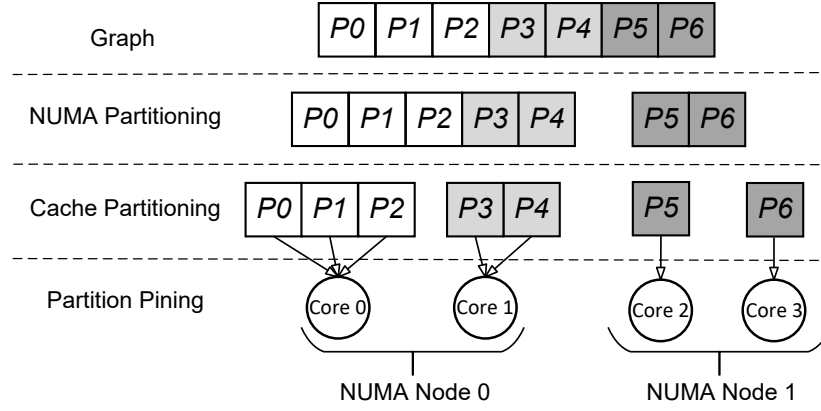


Figure 5.2: Partitioning result of HiPa. The boxes represent cache-able partitions of the graph data. Respectively, $P0-2$ hold 10 edges, $P3-4$ hold 15 edges, and $P5-6$ hold 30 edges. The final output of HiPa is that the cores are allocated with *unequal* numbers of partitions but equal number of edges.

5.4 Thread Management

5.4.1 Problem: Thread Contention

The Hyper-Threading is a popular technology offered by various modern CPUs [103]. It improves the parallelism of a multicore system by creating two logic cores from every physical core in the system. Each pair share the same hardware resources, including caches and bus, and incur lightweight overhead between context switches. Hence, the cooperation between the paired logic cores is more efficient than that of two physical cores. In addition, the number of logic cores stands for the maximum number of threads available for multithreading, which doubles the number of physical cores.

The Hyper-Threading is enabled by default in almost all graph processing frameworks to boost parallelization for speedup [4, 5, 11, 8]. Nevertheless, in the partition-centric paradigm, full utilization of logic cores often leads to a performance decline [52, 94, 93, 8]. In Section 5.4, experimental results show that existing partition-centric graph methodologies scales poorly once the number of working threads exceeds the number of physical cores. When all logic cores are utilized, the processing of graph is nearly slowed down by $2\times$ (see Fig. 5.6).

The poor scalability results from the intensive contention among threads on hardware resources, including caches and memory bus [93]. In prior partition-centric works, each thread is allocated with L2-cache-sized data for computation, which requires full occupation of the private cache. This intensifies the thread contention, because the dual threads (i.e., logic cores) on the same physical core have to compete for the usage of L2 cache. Furthermore, the memory bandwidth is designed to scale with the physical core instead of logic cores [103]. The bandwidth is heavily congested when all logic cores are flushing the

L2 caches [8].

To alleviate the thread contention, prior works simply use only half of available threads, which equates the number of physical cores [52, 94, 93, 8]. However, the problem of contention still exists due to the randomness of thread creation. When multithreading, the operating system (OS) arbitrarily generate threads from the pool of *logic* cores, regardless of the status of physical cores. Therefore, it might occur that two selected logic cores corresponds to the same physical core and thus contend for resources. Moreover, the threads schedule the processing of partitions on a first-come-first-serve policy. The competition amongst multiple threads for the ownership of a partition would further intensify the thread contention.

In order to alleviate the thread contention and enhance the scalability of partition-centric paradigm, we propose a thread-data pinning scheme for the coordination between data and threads on multicores. It consists of two steps: (1) bind the threads to NUMA nodes, and (2) pin the partitions to the bound threads. The details of this scheme are discussed in next section.

5.4.2 Solution: Thread-data Pinning

As outlined in Algorithm 7, the partition-centric processing of a graph can be generalized into a scatter-gather model that iteratively updates a graph [8]. The computation in every iteration envelops a scatter phase and a gather phase, where the updates of local vertices are propagated to or collected from neighbors. Within each phase, multiple threads are created to parallelize the task. In other words, one phase, either scatter or gather, includes the complete lifecycles of threads. A phase represents a parallel region. As a result, the graph computation is decomposed into numerous parallel regions

Algorithm 7 NUMA-oblivious scatter-gather model

Input: partitions $\rightarrow$ set of partitions

Output: Graph

```

1: for  $i \leftarrow 0$  to iteration do
2:   for all  $p \in$  partitions do in parallel                                 $\triangleright$  1st parallel region
3:     gather( $p$ )
4:   for all  $p \in$  partitions do in parallel                                 $\triangleright$  2nd parallel region
5:     scatter( $p$ )
6: Graph  $\leftarrow$  concatenate(partitions)

```

For instance, assuming a graph algorithm finishes in 10 iterations on a multicore system consisting of 2 NUMA node with 4 physical cores (i.e., 8 logic cores) per node. Within each iteration, the scatter and gather phases are respectively parallelized into discrete parallel regions. To avoid thread contention, only half of the logic cores are used. Therefore, inside

each region, a thread pool that contains 8 threads are initialized to scatter/gather all graph partitions and then terminated after synchronization. Therefore, the overall computation, which consists of 10 iterations of scatter-gather phases, creates $10 \times 2 \times 8$ threads in an interleaved routine.

In the context of NUMA-aware processing, additional overheads are involved for thread manipulations, including *thread binding* and *thread migration*. To place the computation together with data on the same NUMA node, the computing threads are bound to the particular NUMA node where the data are stored. When a thread is initialized on a wrong NUMA node, thread binding would invoke thread migration, where a thread is migrated from current core to another core on the correct NUMA node. Thread migration incurs high latency as the thread context is transferred via memory. Such latency is increased in NUMA systems, because the transfer across NUMA nodes always proceeds through remote memory [102].

If we deploy NUMA-aware multithreading on the processing model of Algorithm 7, every threads created inside the parallel region have to be bound to a NUMA node. In the worst case, where all threads are initialized in the wrong NUMA nodes, the thread migration occurs up to 160 times, since there are 160 threads created during 10 iterations. The problem of thread migration is amplified by the massive production of threads.

In order to minimize the expensive thread migrations, a new processing model is proposed as in Algorithm 8. It reduces the frequency of thread creation by extending the functionality of threads. The lifetime of a thread covers the whole period of graph processing. Also, according to the one-to-many partitioning result obtained in Section 5.3, the cache-able partitions are pinned to threads (line 8). As a result, each thread is able to perform *complete* iterative scatter-gather computation over the *fixed* group of partitions.

Algorithm 8 Numa-aware scatter-gather model

```

Input: numa_Partitions  $\rightarrow$  numa-ly allocated partitions,
        numa_Threads  $\rightarrow$  numa-ly bound threads
1: Function Th_Func(numa_part)
2:   for i  $\leftarrow$  0 to iter do
3:     scatter(numa_part)
4:     synchronize
5:     gather(numa_part)
6: for all th  $\in$  numa_Threads do in parallel  $\triangleright$  parallel region
7:   match th with p  $\in$  numa_Partitions
8:   th calls Th_Func(p)
9: Graph  $\leftarrow$  concatenate (numa_Partitions)

```

By pinning the partition to the thread on NUMA nodes, data traffic is mainly restricted within local memory, and thread contention are effectively eliminated. Hence, we can now

take the advantage of Hyper-Threading and utilize all logic cores. In NUMA-aware model of Algorithm 8, the number of created threads during the overall graph processing equals to the number of available *logic* cores, which is 8×2 . Hence, in the worst case of thread binding, the thread migration occur at most 16 times, which is significantly lower than the 160 times for Algorithm 7.

Finally, we summarize the advantages of the NUMA-aware thread-data pinning. (1) Remote memory accesses are reduced. (2) Thread migration is minimized. (3) Thread contention is alleviated. (4) Scalability is enhanced.

5.5 NUMA-Aware Partition-Based Data Layout

In order to facilitate the pinning between threads and data, an lookup table is constructed. It allows HiPa to identify the coverage of each thread on the graph data. As depicted in Fig. 5.3, it is a 2-layer tree that records the range of partitions permitted for every thread, and subsequently, the range of vertices covered by each partition. This table is globally accessible for all threads.

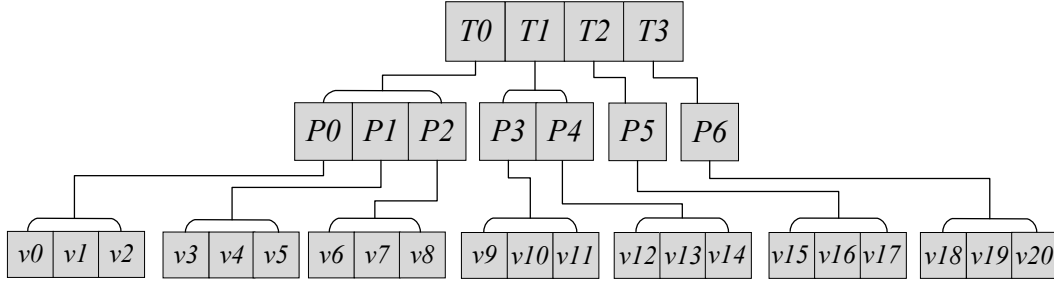


Figure 5.3: 2-level hierarchical lookup Table. The first level records the partition range for each thread, and the second level records the vertex range for each partition.

Furthermore, HiPa adopts the edge compressing technique proposed by the partition-centric work [93]. As previously discussed, the inter-edge, whose source and destination vertices reside in two different cores, induces expensive memory traffic. To reduce the cross-core communication, the inter-edges pointing to multiple vertices inside the same partition are *compressed* into single inter-edge. After the data is transferred via the compressed edge, it is then locally propagated to the destination vertices.

For the instance of Fig. 5.4, originally there are two inter-edges, $(v1, v6)$ and $(v1, v7)$, incurring two cross-core messages that carry the updated data from $v1$. Since the two inter-edges share the same source vertex, carry the same data and also reach to the same destination partition, it is redundant to keep both them. Hence, they are compressed into a single inter-edge, holding the data from one vertex to one *partition*, e.g., $(v0, p1)$.

After received, this message is decoded and locally propagated to multiple vertices via the intra-edges $(p1, v6)$ and $(p1, v7)$ inside the destination partition [93].

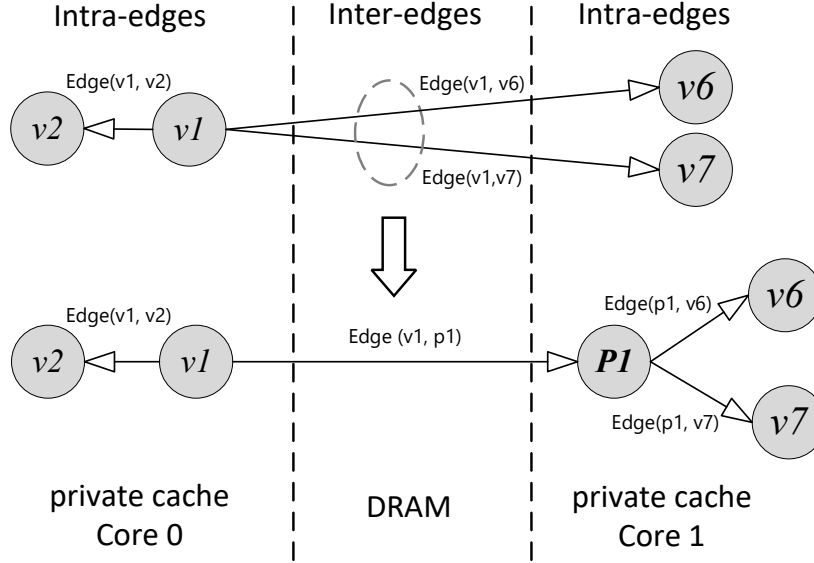


Figure 5.4: Scatter via inter-edges and intra-edges

However, unlike the predecessor, HiPa ensures the contiguous virtual address space for the graph data. Based on the partitioning result, graph vertices, edges and attributes (e.g., rank values and vertex degrees) are subdivided into discrete physical pages on different NUMA node. Since discrete address space (e.g., two arrays allocated on two NUMA nodes) requires additional overhead for indirect accesses, the attributes are mapped to contiguous virtual address to seamlessly access cross-node data. Also, within the same NUMA node, vertices as well as edges are placed in the contiguous physical (and virtual) address spaces, though they might belong to different partitions. This reduces the cost for a thread to iteratively process over multiple partitions, and avoid the risk of memory leaks due to multi-dimensional dynamic memory allocation.

5.6 Evaluation

In this section, we evaluate the effectiveness of HiPa by comparing the performance of PageRank implemented under different methodologies/frameworks. Also, extensive experiments are conducted in order to study its impact on the multicore systems.

5.7 Experimental Setup

We perform experiments on a multicore machine with two Intel Xeon Silver 4210 processors (i.e., NUMA nodes). Each processor consists of 10 physical cores (contributing to 20 logic

cores) and equipped with a 128GB local DRAM. The private L1 and L2 sizes per core are 64KB and 1MB respectively, and the shared L3 cache size is 13.75MB. The data types for vertices, edges and PageRank value are set to 4 bytes.

The code of HiPa is written in C++ and compiled using g++ 10.2.0 with optimization level O3. The partition size is empirically decided to 256KB (see Section 5.11) and 40 threads are fully utilized for multithreading. The results relating to execution time are reported using the average value over 5 executions, each of which runs for 20 iterations. To neutralize the effects of preprocessing on memory performance, such as loading graphs from disk, the experiments regarding memory and cache utilization are executed for 60 iterations.

Frameworks: We use the source codes of state-of-the-art frameworks, Polymer [5] and GPOP [8], for comparison. Polymer follows the vertex-centric paradigm, and is particularly designed for NUMA machines. GPOP follows the partition-centric paradigm. To our best knowledge, GPOP is the framework providing the fastest execution of PageRank [8]. Following authors' instruction, we set the partition size to be 1M and only use 20 threads for GPOP. However, many frameworks are designed with a trade-off between productivity (i.e., ease of use) and performance [112]. It is often possible for framework-based implementations to perform worse than well-tuned hand-optimized codes (see Section 5.8).

Hand-coded implementation: To provide a fair comparison, we use two hand-coded optimized variants of PageRank. One is a pull-based vertex-centric version (v-PR), where each vertex pulls the value from its in-neighbors for accumulation. This enables all columns of an adjacency matrix to be traversed asynchronously in parallel without storing the partial sum. The other variant is partition-centric PageRank (p-PR) with finely-tuned parameters, e.g., partition size 256KB and 20 threads. The sensitivity analysis regarding these parameters are provided in Section 5.10 and 5.11. We re-implement the source code offered by [93] with enhancement in memory safety. Additionally, Neither v-PR nor p-PR applies NUMA-awareness.

Six graph datasets with varied features are used for performance evaluation. *journal*, *twitter* and *mpi* are social networks illustrating the follower relationships among users. *pld* and *wiki* are hyperlink graphs extracted from the web pages by web crawlers. *kron* is generated by Graph500 Kronecker generator with scale of 23 [46]. Their statistic information is summarized in Table 5.1. In this work, we select the out-degree as the basis for graph partitioning.

5.8 Execution Time

Table 5.2 presents the execution time of PageRank following different implementing methodologies. We observe that HiPa consistently outperforms other methodologies on all graphs. It achieves the highest speedup against Polymer on graph *kron* by $10.65\times$. For p-PR, v-PR

Table 5.1: Graph Descriptions (K: Thousand, M: Million, B: Billion). Intra and inter stands for the intra-edges and the inter-edges per partition if size equals 1MB.

Graphs	Descriptions	#Vertices	#Edges	Intra	Inter
<i>journal</i>	Live Journal[48]	4.8M	68.5M	30.8K	7.9M
<i>pld</i>	Pay-Level-Domain[47]	42.9M	0.6B	7.2K	1.6M
<i>wiki</i>	Wiki Links[49]	18.3M	0.2B	74.9K	0.5M
<i>kron</i>	Synthetic Graph[46]	67M	2.1B	11.3K	2.8M
<i>twitter</i>	Twitter Follower[50]	41.7M	1.5B	10.5K	2.3M
<i>mpi</i>	Twitter Influence[51]	52.6M	2.0B	0.2M	1.6M

and GPOP, HiPa outperforms them by up to $1.45\times$, $4.56\times$, $3.62\times$ respectively.

On the majority of graphs, the hand-coded implementations (e.g., HiPa, p-PR and v-PR) provide better performance than the framework-based ones (e.g, GPOP and Polymer) due to lightweight designs. For example, graph frameworks typically employ a *frontier* for recording the active vertices in each iteration. It is used to avoid unnecessary accesses to the inactive vertices that do not participate in computation. However, the frontiers are completely redundant to PageRank since every vertex is processed in every iteration. In GPOP, this additional layer of can be disabled for PageRank. Hence, we only report the performance of simplified GPOP without frontier for evaluation in this chapter, such as in Table 5.2.

Table 5.2: Execution time (in seconds) of PageRank with various implementations.

	HiPa	p-PR	v-PR	GPOP	Polymer
<i>journal</i>	0.31	0.41	0.54	1.14	1.72
<i>pld</i>	2.43	3.37	8.44	4.18	22.27
<i>wiki</i>	1.74	1.80	1.96	3.90	4.63
<i>kron</i>	7.20	10.06	32.82	11.29	76.62
<i>twitter</i>	8.43	9.83	12.09	14.91	41.06
<i>mpi</i>	13.93	17.54	24.41	33.90	64.00

Also, on the same design basis (e.g., hand-coded or framework-based), the partition-centric paradigm substantially outperforms its vertex-centric counterpart. For instance, HiPa and p-PR are faster than v-PR; GPOP is faster than Polymer. The acceleration results from the aggressive memory reduction promoted by the partition-centric paradigm, which is to be discussed in the next section.

Lastly, the overhead introduced by HiPa consists of graph partitioning and NUMA-aware data binding, but excludes the loading of graphs. The overheads for graphs *journal*, *pld*, *wiki*, *kron*, *twitter*, *mpi* are 0.22s, 1.62s, 0.66s, 5.17s, 5.50s, 8.52s respectively. On average, they can be amortized by 12.7 iterations of PageRank in HiPa, while the normalized overheads for GPOP and p-PR are 9.61 and 12.44 iterations respectively. Plus the overheads, HiPa

still delivers better end-to-end performance than other implementations on most of graphs, except v-PR on graphs *wiki* and *twitter*.

5.9 Memory Utility

With the optimizations of in-core computing and edge compressing, we expect HiPa to incur insignificant memory consumption compared with the vertex-centric methodologies. Moreover, in favor of the NUMA architecture, the remote memory traffic generated by HiPa is to be minimized as well.

Fig. 5.5 illustrates the memory accesses normalized by the number of edges in graphs. We refer to the memory accesses per edge as MApE. The percentages of remote MApE are significantly reduced in NUMA-aware designs, including HiPa and Polymer, which are 13.80% and 10.11% respectively. As for the NUMA-oblivious designs, such as p-PR, v-PR, and GOP, the percentages of MApE are remarkably higher, which are 48.92%, 50.86%, and 53.00% respectively. Among all implementations, HiPa achieves the least number of *remote* memory accesses. The reduction in remote memory accesses allows HiPa to perform as the fastest methodology.

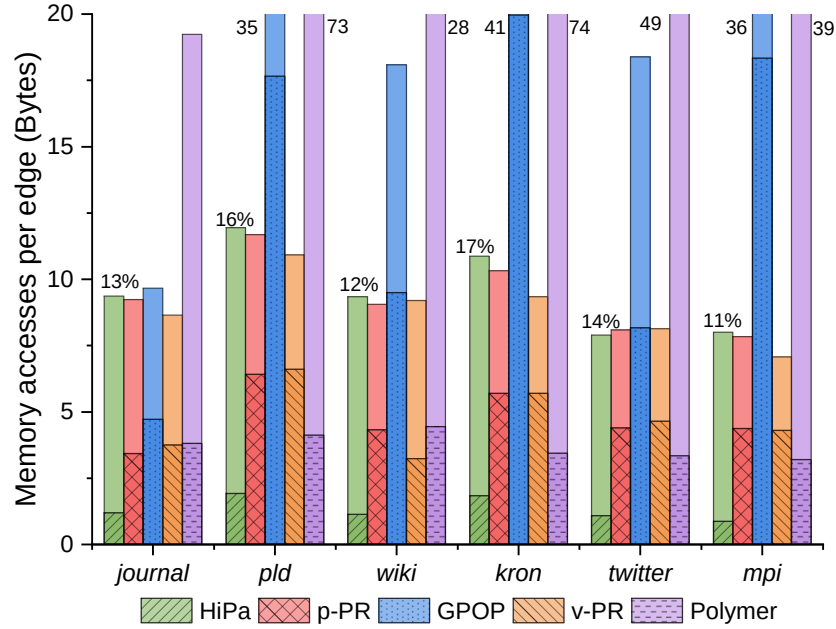


Figure 5.5: Memory accesses (normalized with graph edges) of PageRank in different implementing methodology. The total bar is the total accesses including remote and local memory accesses. The lower, shadowed bar segment is the remote memory accesses. The ratio of remote access for HiPa is denoted on top the total bar.

The partition-centric paradigm, deployed by HiPa, p-PR and GOP, effectively reduces the memory traffic during graph processing. On average of all graphs, HiPa, p-PR and

GPOP generate 9.57, 9.37 and 8.89 MApE respectively. The results are substantially lower than those of Polymer and v-PR, which are 26.66 and 47.31.

Among the partition-centric methodologies, GPOP produces the fewest total memory accesses, because of its largest partition size (i.e., 1MB) in comparison with HiPa and p-PR (i.e., both 256KB). The larger a partition, the better the compression (see Section 5.11). HiPa generates slightly higher memory accesses than p-PR, which are 9.57 vs. 9.37 MApE, due to the overhead of NUMA-aware optimizations. However, the remote memory accesses of p-PR account for 48.92%, which is notably higher than HiPa's 13.80%. As a result, HiPa delivers a considerably faster speed.

Polymer achieves the lowest *ratio* of remote memory accesses on the average of all graphs, which is 10.11% and even lower than HiPa's 14.17%. Nevertheless, the total memory accesses of Polymer is too high to justify the reduction in remote accesses. The inefficiency of Polymer (and also p-PR) results from the intrinsic disadvantages of vertex-centric paradigm. Suffering from atomic operations, low graph locality and irregular memory accesses [8], the performance of Polymer is severely limited.

5.10 Scalability

The scalability of HiPa as well as other methodologies is examined on graph *journal* by varying the number of threads for parallel processing. For each methodology, the results are normalized by its execution time using 40 threads to locate the peak-performance point (i.e., the lowest point). As depicted in Fig. 5.6, all plots finally converge to 1.

HiPa and other two vertex-centric implementations, Polymer and v-PR, exhibit high scalability. As the number of threads increase, the their performances show the tendency of improvement. When scaling from 2 to 20 threads, there is a sharp decrease in execution time. The scaling behavior become weak as more threads are employed, e.g., from 20 to 40. The best performance is achieved when all 40 threads are utilized.

By contrast, the two partition-centric methodologies, p-PR and GPOP, demonstrate relatively poor scalability. Their best performance is obtained when using 16 threads and 20 threads respectively. When 40 threads are fully utilized, the execution time will be nearly doubled. This is because the conventional partition-centric methodologies cause high consumption of memory bandwidth when the cores load and offload partitions [93]. The bandwidth is saturated with approximately half of total threads [8]. Any further addition of threads would only aggregate the contention on bus and cache resources, and thus deteriorates the performance.

Though HiPa is built under the partition-centric paradigm, it still scales well on the multicore system because it forces two-level constrains for a thread to access a partition. Firstly, the NUMA-awareness of HiPa limits about 85% of memory traffic to occur within

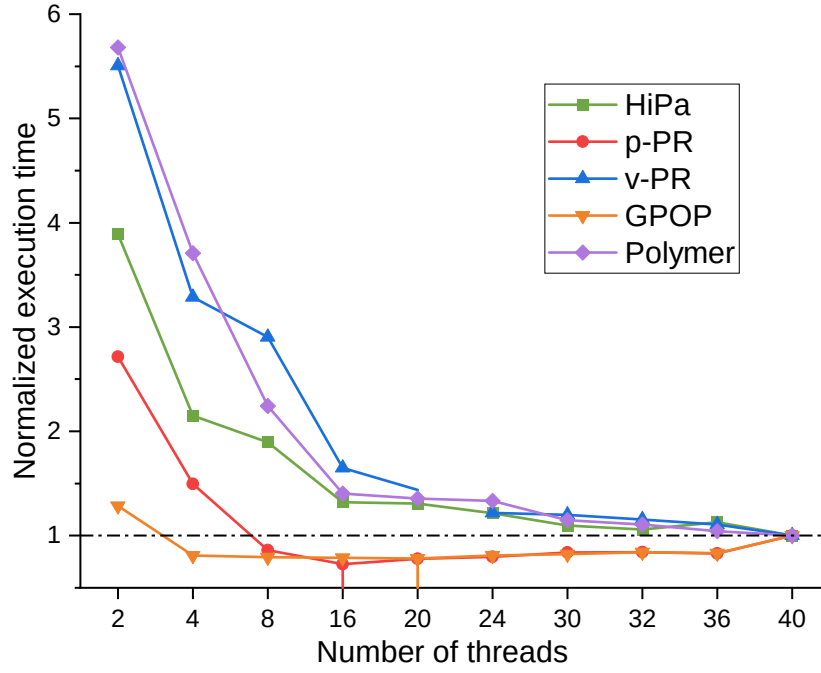


Figure 5.6: Execution time (normalized by the performance of 40 threads) with varied numbers of threads on *journal*.

the local memories. It effectively avoids cores contending for interconnect bus between different NUMA nodes. Inside each NUMA node, the data of every partition are further pinned to a thread, so the conflicts of threads on the same partition are eliminated. As a result, the limitation of conventional partition-centric methodology is overcome, and hence the scalability of HiPa is considerably enhanced.

5.11 Sensitivity Analysis of Partition Size

The partition size plays an important role in the effectiveness of partition-centric paradigm. Fig. 5.7 illustrates the impact of varied partition sizes on the execution time and LLC. In general, as the partition sizes become larger, the LLC hits and hit ratios continuously grow, which indicates the data accesses are increasingly concentrated in LLC. The best performance of HiPa is achieved when the partition size equals (or slightly less than) a quarter of L2 cache size, i.e., 256KB.

When deciding the proper size for graph partitioning, there exists a tradeoff between cache locality and inter-edge compression. If the partition size is small (e.g., size $\leq 256\text{KB}$), high spatial locality is ensured as random accesses are confined within the partitions in private caches. On the other hand, (too) small partitions, such as size=16KB, preserve

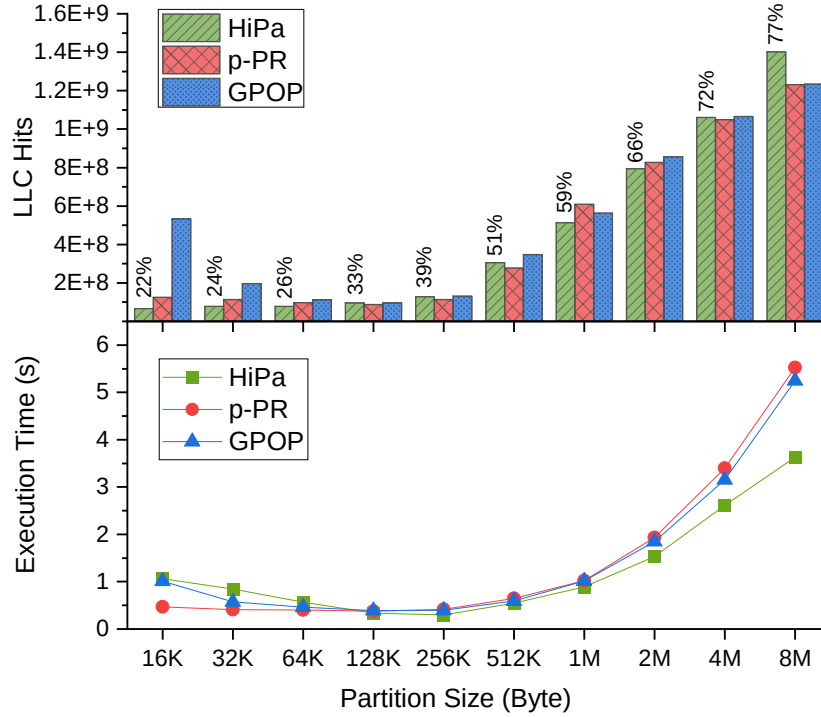


Figure 5.7: LLC cache hits (upper) and execution time (lower) with different partition sizes on graph *journal*. LLC hit ratios of HiPa are denoted.

a large number of inter-edges, which leads to high workloads and substantial memory traffic. Notice that GPOP incurs high LLC accesses, when the number of partitions is exceptional high with size = 16KB. This is because GPOP requires additional data fields for each partitions, such Flags, State [8].

As the partition size increases from 16KB to 256KB, inter-edges are continuously compressed, and workloads are accordingly reduced. Hence, the execution time keeps a steady decline. Nevertheless, once the size exceeds the threshold of 256KB, the intra-edge of a partition spill over from L2 cache to LLC. Cache spatial locality is not promised anymore. As Fig. 5.7 presents, there is a drastic surge in LLC hits (and hit ratios) for all partition-centric implementations when the partition size grows from 256KB to 8MB. Since LLC provides the highest latency in cache hierarchy, the execution is significantly decelerated.

Our experimental results show that the optimal partitioning size, or alternatively the threshold, is 256KB, which is a quarter of L2 size. This finding contrasts with the results in prior partition-centric works [52, 8]. In their settings, the partitions are usually set to fit into LLC or L2, which are 20MB or 1MB in our environment.

The conflict mainly originates from two factors. First of all, as mentioned beforehand, besides the vertex subset, a partition also contains an edge subset (including intra- and

Table 5.3: Normalized execution time using different partition sizes on Haswell and Skylake micro-architectures

Method.	Skylake				Haswell			
	64K	128K	256K	512K	64K	128K	256K	512K
HiPa	1.04	1.04	1.00	1.25	1.08	1.00	1.18	1.29
p-PR	0.85	0.89	1.00	1.30	1.00	1.00	1.08	1.25
GPOP	1.34	1.03	1.00	1.25	1.18	1.00	0.98	1.13
Average	1.08	0.99	1.00	1.27	1.09	1.00	1.08	1.22

inter-edges). Their sizes are unequal for each partition. Both the edge subset and the buffer require additional memory and cache space for writing and reading. Hence, to allow a partition to be privately accommodated by a core, the size of a vertex subset is supposed to be smaller than the L2 cache size, so that the edge subset and buffer are co-located with the vertex subset within the same L2 cache.

Secondly, *the micro-architectures of processors are different*. our experiments in this chapter are conducted on Intel 6th generation CPUs based on 14nm Skylake micro-architecture. All prior partition-centric methodologies were proposed according to the Intel 22nm Haswell micro-architecture, which is 2 generation older than Skylake. Architectural upgrades are deployed on Skylake, a major part of which relates to the cache hierarchy. For instance, in Haswell, the L2 cache is 256KB per core and the LLC is shared with 2.56MB per core. The LLC is inclusive, which guarantees every data exists in L2 cache must also exist in LLC. Skylake provides larger L2 cache of 1MB per core, and smaller shared LLC of 1.375MB per core. The LLC in Skylake is non-inclusive, which allows data in memory to be directly loaded to the L2 cache without through the LLC.

The new hardware characteristics lead to a variability in determining the size of cacheable partitions. To find the proper size on different micro-architectures, we extend the sensitivity analysis by examining the all partition-centric methodologies on Haswell and Skylake. All graphs except *kron* and *mpi* are tested, because these two exceed the memory capacity of our Haswell machine - Intel Xeon CPU E5-2667 with two NUMA nodes, 64GB main memory and 256KB L2 caches. To clearly present the trends of change around the threshold point, we normalize the execution time of each graph by the result of a particular size, for example, 128K on Haswell and 256K on Skylake. The normalized time of all tested graphs are averaged and then listed in Table 5.3.

As Table 5.3 shows, the optimal partition size on Skylake strikes at 256KB for HiPa and GPOP, which enables the fastest execution. For p-PR, the best performance is at 128KB. The performance difference between 256KB and 128KB is minute, since the average of them are 1.00 and 0.99 respectively. As for the Haswell, the optimal sizes for three methodologies

all hit at 128KB. When the size become larger than 256KB, both Skylake and Haswell are sharply decelerated due to the spillover of vertices into LLC.

In addition, the performance of HiPa deteriorates when deployed on single NUMA node with 20 threads, because all contentions are concentrated on one node. For instance, on graph *journal*, single-node HiPa costs 0.44s to finish 20 iterations of PageRank. With the same number of threads, 2-node HiPa, p-PR and GPOP cost 0.39s, 0.41s and 1.14s respectively to end the executions. Nevertheless, we expect the performance of HiPa to be further boosted in 4-node and 8-node machines.

5.12 Summary

In this chapter, we present HiPa, a hierarchical partitioning methodology that boosts the performance of PageRank on multicore systems. HiPa deploys two-level partitioning strategy based on the memory-cache architecture of the systems. Firstly, it subdivides the graph data into coarse-grained subsets according to the NUMA characteristics of the memory. Then, the graph subset in each NUMA node is refined into cache-able disjoint partitions to enable in-core computing. Furthermore, to co-locate computations and data, new data layout is designed and pinned to threads.

Extensive experiments are conducted to evaluate the performance of HiPa. In comparison with the state-of-the-art frameworks as well as hand-optimized implementations, it achieves the fast execution speed and the least remote memory accesses. Moreover, HiPa exhibits high scalability, because it effectively alleviates the thread contention on hardware resources, e.g., bus and caches. Also, we find that the optimal partitioning size for Skylake micro-architecture equals a quarter of L2 cache size, while for Haswell it is half of L2 cache size.

Albeit we use PageRank to demonstrate the effectiveness of HiPa, the methodology of HiPa can be deployed to more generic use scenarios. In the future work, we will explore the design space for the extension of HiPa to diverse algorithms, e.g., SpMV, PageRank Delta and BFS.

Chapter 6

Reformatting & Mixed Processing

Graph traversal algorithms can be recast as Sparse Matrix-Vector multiplication (SpMV). Thus, SpMV optimizations have significantly inspired the development of high-performance graph systems. However, conventional SpMV operations on graphs suffer from inevitable redundancy either in the row-major (i.e., pushing) or column-major (i.e., pulling) traversals, because not every vertex needs iterative execution to update its attributes. Worse, the irregular connectivity of graphs leads to highly random memory accesses during the execution of SpMV. To address the inefficiency of SpMV on graphs, we propose a coalescing pair of optimizations: a mixed graph format CSMix and a processing engine Mixen [43].

6.1 Background and Motivation

6.1.1 Compressed Sparse Format

Compressed Sparse Row (CSR) and Compressed Sparse Column (CSC) are the most widely used compression schemes for sparse matrices [81]. Here, we only examine CSR and its variants (see Fig. 6.1) due to limited paper space; CSC is symmetric and thus is omitted.

CSR is composed by two essential arrays (i.e., CSR offset and CSR index) and one optional array (e.g., property array of vertices or weight array of edges) depending on the graph tasks. The CSR offset array of size $n + 1$ registers the row pointers. The CSR index array of size nnz records the non-zero column indices.

Doubly Compressed Sparse Row (DCSR) extends CSR with the goal of removing the repeated elements in the offset array [113]. The matrix A is compressed, such that the non-zero rows of size n_{zr} and all columns are retained. As a result, the matrix is converted to $\bar{A}$ of size $n_{zr} \times n$.

However, DCSR requires an row auxiliary (RA) array of size n_{zr} to mark the non-zero rows, which adds a layer of indirection. The elements in the offset array must be accessed via

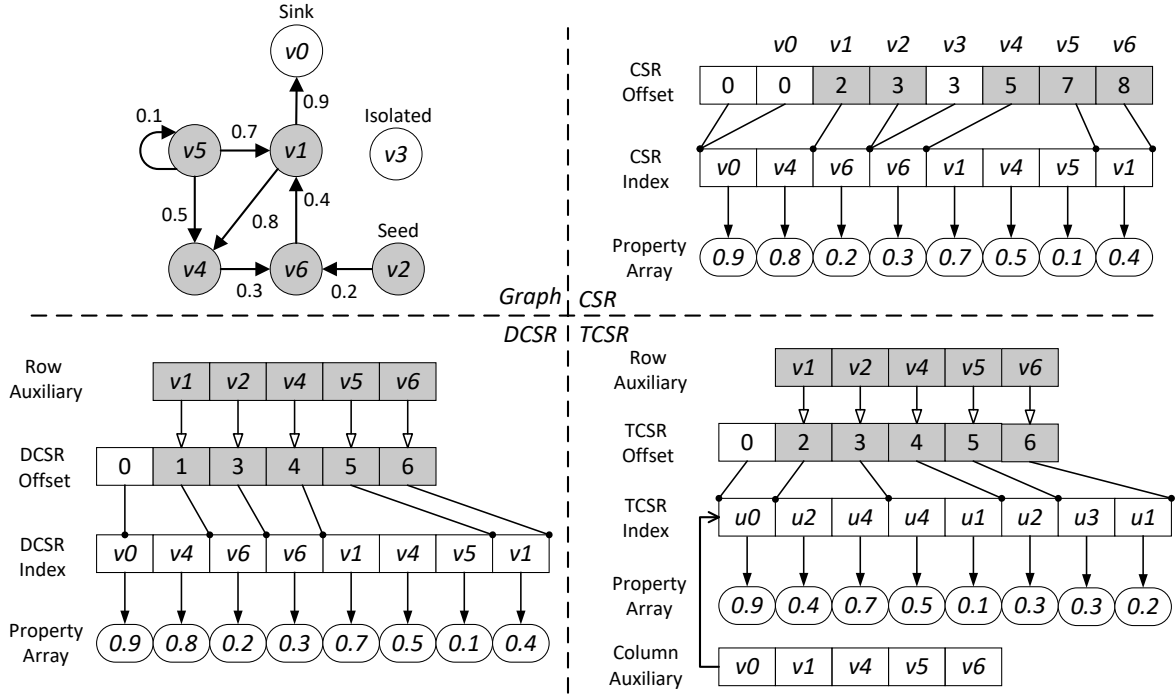


Figure 6.1: Directed graph in the compressed formats of CSR, DCSR and TCSR

the auxiliary: $pointer = [DCSR_Offset[Row_Aux[i]]]$. DCSC performs well for *hypersparse* graphs, where $nnz = O(n)$ [113]. When applied on denser graphs, the inefficiency of indirection may decelerate SpMV computation.

Triply Compressed Sparse Row (TCSR) further enhances DCSR by eliminating the empty column as well [114]. Like using RA to store non-zero rows, TCSR introduces a column auxiliary (CA) array pointing to the non-zero column, of which the size is nzc . To this end, the size of compressed matrix $\bar{\bar{A}}$ is $nzr \times nzc$.

To avert the indirections arising from auxiliary arrays, TCSR aligns the indices of input and output vectors x and y with the compressed matrix $\bar{\bar{A}}$. The elements of input x are filtered by CA, converting into vector $\bar{x}$ of size nzc . Similarly, the output vector y is compressed into $\bar{y}$ of size nzr according to RA.

After the compression of adjacency matrix, together with input and output vectors, SpMV operations are directly applied on the compressed primitives, e.g., $\bar{y} = \bar{\bar{A}} \otimes \bar{x}$. By co-designing the data structure for the matrix and vectors, TCSR tackles the issue of sparsity in SpMV (and graph processing), thereby offering memory and time efficiency [114].

6.1.2 Motivation

This section explains the motivation of our work. Firstly, the pros and cons of the blocking SpMV is discussed by comparing with the pulling SpMV. Then, we demonstrate the redundancy of SpMV caused by irregular connectivity of graphs. These inefficiency guides the designs of CSMix and Mixen, which offer a synergistic solution.

6.1.3 Pull vs Block

Since the performance SpMV is typically bottlenecked by memory activities instead of arithmetic operations [101], the behaviors of memory are modeled. Specifically, we investigate the memory traffic volume (i.e., writes and reads to the memory) and random memory accesses (i.e., non-sequential jumps in the memory address). The data types with respect to vertices, edges and updates are all assumed to be 1 Byte for simplicity.

Traffic Traffic Volume. The GAS paradigm of blocking SpMV is inherently inefficient. This is because, in every iteration, the GAS initiates data transmissions for twice. Firstly, the Scatter phase reads the CSR of size $n + nnz$ and the input array x of size n . Subsequently, it generates nnz data writing to the bins. Then, the Gather phase loads nnz pairs of data and destination for accumulation, and writes n sums. As a result, total traffic volume is $4nnz + 3n$ Bytes.

By contrast, the Pull traversal propagates the updates for only once. The CSC of size $n + nnz$ are scanned, meanwhile the elements in x are loaded for nnz times. Output array y of size n are written back to memory. Consequently, the memory traffic for Pull is $2nnz + 2n$ Bytes, which is substantially less than that for GAS.

Random Memory Access. Despite the reduction in memory traffic, the pulling SpMV suffers from highly randomized accesses. For the CSC-based Pull traversal, the requests to `csc_offset`, `csc_index` and y arrays are sequential, and therefore promise high cache locality. However, the reads to the x array are random, which is up to $O(nnz)$, thus resulting in poor cache utility.

The Blocking technique is developed upon the CSR-based Push traversal. It performs sequential accesses to the `csr_offset`, `csr_index` and x arrays, but invokes random writes to the y array. Fortunately, the random writes to y is substantially reduced by sorting the propagation [93]. The data accesses within bins are sequential, and only the switch of blocks will produce a random access. In this case, the number of random requests are $O((n/c)^2)$, where c is the cache size used to determine the size of blocks.

Use graph *wiki* [49], of which $n = 18.2$ M and $nnz = 172.2$ M, as an example. We further assume $c = 64$ KB, so the number of blocks are $(n/c)^2 \approx 285^2 = 80.9$ K. Considering the worst scenario where locality does not exist in *wiki*, so the access are completely randomized. The pulling SpMV causes 172.2 M random accesses, while the blocking SpMV only delivers

80.9 K. Nonetheless, the pulling SpMV generates 362.6 MB more memory traffic than the blocking one does.

In summary, the Blocking technique lessens the random memory accesses at the cost of raising more memory traffic. For graphs featuring high locality, the random accesses are intrinsically minimized, so the performance gain of Blocking is trivial, or even hurting [101]. Thereby, a graph processing engine is needed so as to take advantages of both blocking and pulling SpMV.

6.1.4 Computational Redundancy

The power-law degree distribution leads to two related results: (1) the maximum degree is considerable higher than the average, and consequently (2) a massive number of vertices are not connected (at least in one direction) for compensation. In this chapter, the zero-degree vertices are exploited.

Depending on the connected edge type, vertices can be classified into four types. (1) *Regular* vertices with both incoming and outgoing edges. (2) *Seed* vertices¹ with only outgoing edges. (3) *Sink* vertices with only incoming edges. (4) *Isolated* vertices with no edges. Table 6.1 lists a number of graphs with classification of vertices.

Table 6.1: Graph Decomposition and the maximum in- and out-degree.

Graphs	Vertices	In	Out	Reg.	Seed	Sink	Iso.
<i>weibo</i> [115]	5.8M	237.2K	4.0K	1%	99%	0%	0%
<i>track</i> [105]	12.8M	11.6M	1.1M	46%	54%	0%	0%
<i>wiki</i> [49]	12.2M	631.4K	9300	22%	33%	45%	0%
<i>pld</i> [47]	42.9M	1.8M	3.9M	56%	8%	28%	8%
<i>rmat</i> [116]	8.4M	21.3K	22.3K	7%	69%	0%	24%
<i>kron</i> [46]	16.8M	1.0M	1.0M	53%	0%	0%	47%

When translated to adjacency matrix, a seed vertex v_j corresponds to an empty column, that is, $A[:,j] = 0$. Likewise, a sink vertex v_i indicates an empty row, i.e., $A[i,:] = 0$. Conjointly, an isolated vertex suggests that both the row and the column are empty. The emptiness of matrices results in poor spatial locality.

CSR compresses the adjacency matrix by sequentially storing the non-zeros of each row. Unfortunately, the format of CSR is still inefficient for graph traversal. For instance, in the

¹Conventionally, they are named as *source* vertices. However, "source" vertices are also used in a pair with "destination" vertices to describe the message direction. Hence, to avoid confusion, the term of "seed" is employed.

graph *wiki*, 45 percent vertices (i.e., sink vertices) do not have outgoing edges, which means nearly half of elements in the CSR offset array are repeated if graph represented in CSR. They are unnecessarily scanned in every iteration. Similarly, if *wiki* is stored in CSC, there are still around one third redundant elements (i.e., seed vertices) in the CSC offset array.

Furthermore, the connectivity of vertices leads to redundant memory traffic. During the traversal, regular vertices play a central role as they are iteratively updated. Seed vertices maintain their data unchanged after initialization. The same data are repeatedly broadcast by them. Sink vertices do not affect other vertices, and their states solely depends on their in-neighbors. Hence, the propagation towards the sinks can be delayed until the finish of others in the last iteration. At last, isolated vertices are completely independent and thus not needed.

The GAS adopted by blocking SpMV exacerbates the redundancy, as the unnecessary traffic related to the seed and sink vertices are doubled by the Scatter and Gather phases. To this end, we are motivated to customize the graph representation for efficient graph traversal, and more importantly, to devise the graph processing engine for elimination of redundant memory traffic.

6.2 CSMix

In this section, a mixed format *CSMix* is presented, which unifies CSC, CSR and DCSR for graph representation. The design of *CSMix* is rooted in a simple idea – graph filtering. The details regarding graph filtering and formatting are elaborated in following subsections.

6.2.1 Graph Filtering

First of all, we compress the graphs using the advanced compressing scheme. As presented in Fig. 6.2a, a seed, sink or isolated vertex corresponds to an empty column or/and row respectively. We deploy TCSR on the original adjacency matrix of a graph, which deletes these empty contents. Consequently, a compressed matrix is generated as illustrated in Fig. 6.2b.

Subsequently, the filtering procedure is performed on the compressed result. As we discussed in Section 6.1, the iterative computation towards seed and sink vertices are needless. Hence, we move them to the preparation phase and the post phase respectively (detailed in Section 6.3.2). In order to do so, the non-zero rows of the seed vertices and the non-zero columns of the sink vertices are erased. As a result, we obtain a filtered matrix containing only regular vertices as in Fig. 6.2c.

The compressing and filtering procedures (e.g., Fig. 6.2a, 6.2b, 6.2c) can be summarized in one sentence: completely remove the columns and rows of the seed, sink and isolated

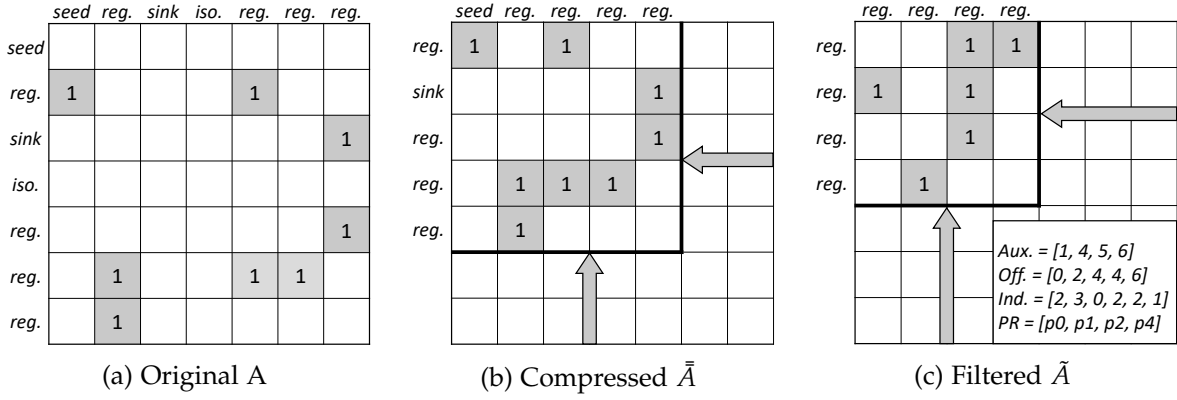


Figure 6.2: Compressing and filtering on the adjacency matrix of a graph. The DCSR of the filtered matrix $\tilde{A}$ is presented.

vertices. The remaining regular vertices and their internal connections compose a dense square matrix (e.g., Fig. 6.2c). This part of graph is the main source of workload, because it participates in every iteration of computation.

6.2.2 Graph Formatting

However, if the filtered graph is formatted in TCSR, the row auxiliary array and the column auxiliary array would be identical, incurring a repetition. To achieve the formatting efficiency, they merge into one single auxiliary array, which is the feature of DCSR. In other words, the utilization of DCSR is sufficient to express the filtered graph, though the filtering mechanism originates from TCSR.

As exemplified in Fig. 6.2c, the DCSR of $\tilde{A}$ consists of an auxiliary array (Aux.), an offset array (Off.), an index array (Ind.) and a property array (e.g., PageRank vector of vertices). The filtering DCSR exploits the advantages of both DCSR and TCSR. Firstly, it requires only one auxiliary array for encoding (like DCSR), since the rows and columns in the filtered graph point to the same set of vertices. Furthermore, it allows the direct application of SpMV operations on the filtered primitives (like TCSR), and thus avoids the indirection.

It should be emphasized that there are essential differences between the DCSR of $\tilde{A}$ (Fig. 6.2c) and the DCSR of A (Fig. 6.1). The former modifies the structure of graph and generates a sub-matrix. This idea is developed from TCSR but coincidentally complies with DCSR. Meanwhile, it requires a delicately designed scheduler for cooperation, which will be introduced later in Section 6.3.2. By contrast, DCSR and TCSR are universally usable, offering higher scalability. However, they hold substantial redundancies. The Row Auxiliary in DCSR forces indirect access to the vertices. TCSR requires an extra Column Auxiliary. Both of them includes seed and sink vertices that causes redundant computation

for PageRank.

Besides, the formatting of sink and sink vertices follows the basic formatting schemes. Since the seed vertices possess only outgoing edges, they are encoded in CSR. On the contrary, the sink vertices with only incoming edges are represented by CSC. As a result, a mixed formatting strategy is accomplished: DCSR for the regular, CSR for the seed, and CSC for the sink. We encapsulate the three into one format – CSMix.

Table 6.2: Space requirement for different formats.

Array	CSR	DCSR	TCSR	CSRMix
Row Aux		nzr	nzr	nzr
Offset	$n + 1$	$nzr + 1$	$nzr + 1$	$nzr + 1$
Index	nnz	nnz	nnz	nnz
Col Aux			nzc	

The space required for storing CSMix is the same as the conventional DCSR as listed in Table 6.2. In addition, metadata are needed to clarify the boundary of each part, so the loading of graphs can be correctly switched for different formats.

6.3 Mixen

Representing graphs in CSMix acts as an offline optimization. This section discusses the online optimization of *Mixen*. Mixen further partitions the filtered graph into cache-able blocks to improve memory efficiency. Also, Mixen discriminatively processes the graph parts based on their connectivity (i.e., formats) to remove redundant computations.

6.3.1 Graph Partitioning and Binning

We employ the 2-D blocking on the filtered matrix of graphs. The size of a block equals $c \times c$, where c is a cache-related indicator and the optimal value of it is tuned in Section 6.5.4. Assuming the number of regular vertices is $nreg$, then filtered matrix is divided into $\frac{nreg}{c} \times \frac{nreg}{c}$ blocks. We denote $\frac{nreg}{c}$ as nb . Conceptually, the matrix is converted to a $nb \times nb$ grid of blocks.

A block serves as the basic data unit for threads to process. It imposes constraints on the thread behavior in two dimensions. Firstly, the (horizontal) width of the block defines the vertex range for a thread to access. The horizontal blocking can be easily implemented by partitioning the DCSR of the filtered graph into multiple local DCSRs.

Fig. 6.3 provides an example that divides the filtered matrix from Fig. 6.2c into 4 blocks of size 2×2 . A vector specifying the vertex range (VR) of a block is added. Notice that only

the offset array (DO) and the index array (DI) are reshaped. They are allocated to threads with exclusive ownership. Meanwhile, the auxiliary array (RA) and the property array (PA) retain untouched, and are accessible depending on the VR of a block.

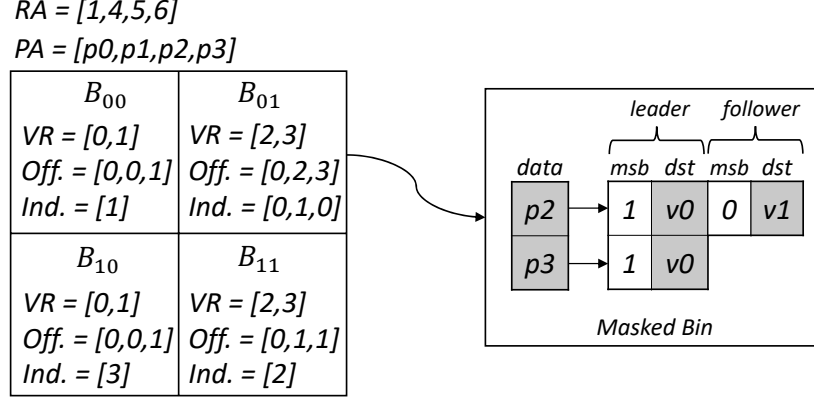


Figure 6.3: Local DCSRs and bins

Secondly, the (vertical) height of the block limits the propagation range of updates. To facilitate the vertical blocking, we adopt the *masked* bins [93] from previous graph processing works. The updates are buffered in the bins instead of writing to the dst vertices directly. A bin consists of two arrays, one for data and one for the dst vertices. In the dst array, the most significant bit (MSB) of vertex ID is masked.

An example is illustrated as in Fig. 6.3. The bit of 1 indicates the dst vertex (e.g., v_0) corresponds to the update (e.g., p_2) from the data array. The bit of 0 suggests that current vertex (e.g., v_1) can directly reuse the update (e.g., p_2) from its leader (e.g., v_0), so the repeated request to the data array is avoided. The vertices with the MSB of ID = 1 are regarded as the leader vertices, while those with MSB = 0 are the followers.

During the construction of local blocks, the masked bins are created. The vertex ID needs to be encoded for only once in the vertex array, since PageRank is a stationary algorithm. Then, they are reused in every iteration. The data array are iteratively updated, which is the root cause of memory traffic.

The exercise of masked bins optimize memory efficiency in several aspects. Firstly, it significantly reduces the write/read operations in memory, because the updates to multiple vertices in the same block are compressed into a single one (e.g., $p_2 \rightarrow v_0, v_1$). Secondly, the number of leader vertices nl , equivalent to the size of data array, will be no larger than the block's width: $nl \leq c$. Hence, the propagation of data is cache-able too. Thirdly, the bins organize the data-vertex lists in the order of sorted vertex ID, which promotes completely sequential access within the bins.

6.3.2 Graph Scheduling and Propagating

The iterative executions on the regular vertices contributes to the main workload of graph processing, which is referred as to the *Main-Phase*. Besides, there are two more phases that include seed and sink vertices and complete the overall workload: *Pre-Phase* and *Post-Phase*. The whole procedure is scheduled as in Algorithm 9.

Pre-Phase with Seed. In the phase of preparation, seed vertices are firstly initialized. Afterwards, the seeds push their data to dst vertices and turn inactive in following computation (line 2). Every receiving regular vertex accumulates the messages and caches the result, which is to be used in the Main-Phase. To remove the overhead of preparation, we integrate the Pre-Phase into the earlier stage – the construction of blocks (line 1).

Main-Phase with Regular. The processing of regular vertices can be generalized as an iterative Scatter-Cache-Gather-Apply (SCGA) model. This model operates on the $nb \times nb$ grid of blocks and produces the updates for regular vertices $v \in regular$.

The Scatter and Cache steps are grouped in one parallel regions (lines 4-8), which are parallelized column-wisely in the view of grid. In the Scatter step, the thread assigned with i is responsible for updating all the bins in the i^{th} column (i.e., $bin[i][:]$) of the grid. In each bin, only the leader vertices (ID with MSB set to 1) are read to index the transmission of data.

Thereafter, in the Cache step, the updates of vertices are reset to the cached values. This step is not needed in prior blocking SpMV works [101, 93]. It incurs additional costs for reading the cached array `y_cache`. Therefore, the graph processing is slightly burdened (further analyzed in Section 6.4).

The Gather and Apply steps are also grouped in one parallel regions (lines 9-13), but parallelized row-wisely. In the Gather step, the dst vertices accumulate all the buffered data from the bins in the i^{th} row (i.e., $bin[:,i]$). Both the leader vertices (MSB = 1) and follower ones (MSB = 0) are read to calculate the partial sum for every vertex. The data fetched by the leader are copied to its followers without moving from the bin again.

Subsequently, in the Apply step, user-defined functions are applied on the sum, which typically consists of varied arithmetic operations. The workload of them is relatively minor compared to that of cross-block data movement. Overall, the computational cost mainly stems from the Scatter and Gather steps, where the bins through a column/row are consecutively written and read.

In addition, in the end of the Main-Phase, the pointers to `x` and `y` are exchanged in order to start next iteration.

Post-Phase with Sink. Finally, after the Main-Phase is finished, either due to the convergence or the pre-defined number of iterations, the processing enters into the Post-Phase. The sink vertices pull the data from the regular and sink vertices along the incoming

Algorithm 9 SpMV performed by Mixen**Input:** seed, sink, regular**Output:** x, y, y_cache, bins

```

1: bins[nb][nb] = Block(regular)                                ▷ Blocking
2: y_cache = Push(seed)                                         ▷ Pre-Phase
3: while !converge & !max_iter do                               ▷ Main-Phase
4:   for all i ← 0 to nb-1 do in parallel
5:     for j ← 0 to nb-1, v ∈ regular do                         ▷ Scatter
6:       x[v] update bins[i][j]
7:       for v ∈ regular do                                       ▷ Cache
8:         y[v] = y_cache[v]
9:   for all i ← 0 to nb-1 do in parallel
10:    for j ← 0 to nb-1, v ∈ regular do                         ▷ Gather
11:      y[v] accumulate bins[j][i]
12:      for v ∈ regular do                                       ▷ Apply
13:        Function(y[v])
14:  Swap(x,y)
15: Pull(sink)                                                    ▷ Post-Phase

```

edges and then computes their own updates.

6.3.3 Benefits and Generalization

The optimizations proposed in this chapter build upon the tradeoff between blocking and pulling SpMV, and the observation on structural property of power-law graphs. Our solution is to perform different SpMV paradigms on graph parts. The solution not only take the advantage of each paradigm, but also avoid redundant memory traffic. Although initially designed to distinguish edge directions, the proposed solution is also effective on undirected graphs where isolated vertices are widely spread (see Section 6.5.2).

Mixen can be easily extended to more general computations. By employing hierarchic frontier that monitors the activity states of blocks and vertices [8], Mixen can process dynamic graph algorithms such as BFS. Moreover, the classification of vertices allows these graph algorithm to early stop by just examining the input vertex ID.

Also, the proposed optimizations can be applied to standard SpMV on non-square matrices by segmenting the columns and rows separately. In such case, the outer loop of Scatter and Gather phases parallelize the row- and column-wise blocks respectively.

6.4 Performance Analysis

We undertake theoretical analyses on the proposed SpMV implementation to better understand its improvements over existing works.

Here, we continue the assumption that the all data types are of size 1 Byte. Furthermore, we focus our discussion on the iterative computation of regular vertices, because other vertices are processed at most once. The portion of regular vertices over all vertices is notated as $\alpha = nreg/n$. The number of edges in the filtered graph $\tilde{A}$ is referred as to $nn\tilde{z}$, and their portion over all edges is $\beta = nn\tilde{z}/nnz$.

Also, we are interested in the effectiveness of applying the masked bins in reducing memory traffic. The numbers of leader vertices nl over all blocks are summed up: $nls = \sum_i^{nb} \sum_j^{nb} nl[i][j]$. The reduction rate r is obtained by: $r = nls/nn\tilde{z}$.

Memory Traffic Volume. In the Scatter step, a scan of regular vertices reads $nreg$ Bytes of updates and nls Bytes of dst vertices (i.e., leader vertices), and then writes nls Bytes in the data array of bins. In the Cache step, regular vertices are reset by the cached data, which causes $nreg$ Bytes of reads and writes respectively. The Gather step reads $nn\tilde{z}$ Bytes of dst vertices (i.e., leader and follower vertices) and nls Bytes of updates from the bins, followed by $nreg$ Bytes of sum writes. The data movement relating to Apply step is ignored as in previous discussion (Section 6.1.3). Overall, the memory traffic volume is:

$$\begin{aligned} mem &= 4nreg + 3nls + nn\tilde{z} \\ &= 4\alpha \cdot n + (3r + 1)\beta \cdot nnz \end{aligned} \tag{6.1}$$

The memory traffic incurred by Mixen is highly dependent on the regular vertices α and their connectivity β . The fewer the regular vertices are ($\alpha \rightarrow 0, \beta \rightarrow 0$), the lower the memory communication is. On the other hand, considering the worst case where an undirected graph contains few isolated vertices ($\alpha \approx 1, \beta = 1$), the impact of graph filtering is almost diminished.

Also, the compression of messages r plays an important role in improving memory efficiency. mem achieves the best result when all outgoing edges per vertex are compressed into a single one: $r = nreg/nn\tilde{z}$. Comparatively, the worst scenario is when there is no compressed edges: $r = 1$. Assuming $n \ll nnz$ and $nreg/nn\tilde{z} \approx n/nnz$, thereby, we obtain $mem \in [\beta \cdot nnz, 4\beta \cdot nnz]$. The traffic volume is bounded by the connections among the regular vertices.

Random Memory Access. Inside a block, the access to the data and dst arrays are completely sequential, thus facilitating high memory efficiency. The random access occurs when the thread switches a bin for updating or accumulation. Since there are total $nb \times nb$

bins (one bin for one block), the number of random memory accesses per iteration is:

$$rand = O(nb^2) = O\left(\left(\frac{\alpha \cdot n}{c}\right)^2\right) \quad (6.2)$$

The portion of regular vertices α controls the randomness by adjusting the number of blocks according to its square number α^2 . Hence, Mixen is expected to be of high effectiveness on graphs featuring a low α . Oppositely, when $\alpha = 1$, the performance degrades to conventional blocking SpMV, which is $(n/c)^2$.

6.5 Evaluation

6.5.1 Setup and Implementation Details

Experiments are conducted on two different machines, the characteristics are described in Table 6.3.

Table 6.3: Characteristics of machines.

	L1	L2	LLC	Mem	Cores	Threads
Skylake	64KB	1MB	27.5MB	256GB	20	40
Haswell	64KB	256KB	20MB	64GB	16	32

Our code is implemented in C++ and compiled using G++ 9.3.0 with O3 optimization. The parallelization is enabled by OpenMP [117]. During the graph processing, the dynamic scheduler is employed to facilitate workload balance. The data types of vertices, edges and properties are all of 32 bits, e.g., `unsigned int` or `float`. On Skylake machine, the size² of a block is set to 256 KB, which contains 64K vertices, though it is not always the optimal value (see Section 6.5.4). Meanwhile, 20 threads are employed for parallelization. On Haswell, the block size and utilized threads are 128 KB and 16 respectively.

To obtain a convincing result, we run an experiment for 5 rounds of 20 iterations and report the average. The cache performance is measure by Linux' `perf` [95], and the memory activities are monitored by Liwid [96].

The programs are tested over a variety of graph datasets, including real-world graphs as well as synthetic ones. Graphs *weibo* [115], *track* [105], *wiki* [49] and *pld* [47] are crawled from social network or hyperlinks in webpages. For these graphs, the isolated vertices are usually discarded before being open-sourced. Graphs *rmat* [116] with $a = 0.6, b = 0.1, c =$

²since a block is a square matrix, we only mention its side length s in one dimension to represent its actual size $s \times s$ for convenience.

0.1, $d = 0.1$ and *kron* [46] are manufactured using specific graph generators. Their attributes are detailed in Table 6.4.

Table 6.4: Graph datasets and their attributes (M: Million).

Graphs	n	nnz	Type	Directed	α	β
<i>weibo</i> [115]	5.8M	261.3M	Real	Yes	0.01	0.06
<i>track</i> [105]	12.8M	140.6M	Real	Yes	0.46	0.60
<i>wiki</i> [49]	18.2M	172.2M	Real	Yes	0.22	0.78
<i>pld</i> [47]	42.9M	623.1M	Real	Yes	0.56	0.84
<i>rmat</i> [116]	8.4M	134.2M	Syn	Yes	0.07	0.59
<i>kron</i> [46]	16.8M	520.8M	Syn	No	0.53	1

We use PageRank [85] as the representative graph-based SpMV for performance evaluation. It ranks the importance of vertices based on their connections. The rank score of vertex V is acquired by iterative computation: $new_rank[v] = 0.15/n + 0.85 \sum_{u \in N_v^-} old_rank[u] / |N_u^+|$, where N^- and N^+ means in- and out-neighbors of a given vertex. The equation can be represented in linear algebraic form: $y = (1 - \theta)\mathbb{A}^T x + \theta\Omega$, where $\mathbb{A}$ is the row normalized adjacency matrix, and θ and Ω are the constant scalar and vector respectively.

The performance of Mixen is compared with three hand-optimized versions and two framework-based implementations. Unless otherwise stated, the number of working thread T equals the number of available threads, e.g., 40 for Skylake and 32 for Haswell. When necessary, the optimal blocking size (S in KB) is carefully examined. The selected works are:

Pulling PageRank (PuPR) is used as the baseline. *Propagation Blocking (PB)* [101] enhances the basic Blocking technique by blocking the propagation of updates. For PB, (S, T) are (512, 20) on Skylake, and (256, 16) on Haswell respectively. *Partition-centric Processing Methodology (PCPM)* [93] proposes the masked bins to compress inter-subgraphs edges to reduce memory traffic. PCPM achieves the optimality when (S, T) are (256, 20) on Skylake, and (128, 16) on Haswell respectively. *Graph Based Benchmark Suite (GBBS)* [7] implements PageRank also by pulling the updates with graphs formatted in adjacency lists. *GraphMat* [82] represent graphs in DCSC and partitioned along rows.

6.5.2 Execution Time

Foremost, we evaluate the effectiveness of Mixen in boosting the speed of PageRank. Table 6.5 presents the execution time of various PageRank implementations. Running on Skylake and on average of all graphs, Mixen outperforms PCPM, PB, PuPR, GBBS, GraphMat by $4.13\times$, $9.56\times$, $9.40\times$, $10.70\times$, $10.09\times$ respectively. Excluding the odd point of the unconventional graph *weibo*, Mixen still yields speedup over the second fastest PCPM by

$1.56\times$. Similarly, on the platform of Haswell, Mixen achieves accelerations over the best of the competitor by $4.51\times$, and $1.43\times$ without *weibo*.

The frameworks GBBS and GraphMat suffer from implementation overheads, such as the support to dynamic algorithms. Also, their graph representations are less efficient than CSR when running pulling PageRank. The adjacency list used by GBBS prohibits the consecutive access to the edge set. The DCSC in GraphMat causes an indirection because of the Row Auxiliary array. Hence, they are inherently suboptimal compared to the hand-coded counterparts.

In particular, Mixen delivers a dominating speedup over the second fastest implementation on the graph *weibo*, e.g., $17.0\times$ on Skylake and $19.9\times$ on Haswell. This graph is composed by 99 percent of seed vertices. Other graphs, being more "regular", thus contribute to lower speedups.

Moreover, though initially designed for directed graphs, Mixen boosts the performance on the undirected graph *kron* too. This is because, in *kron*, isolated vertices account for 47% of total vertices. These vertices are accessed (e.g., in the *csc_offset* array) but never computed, which causes a considerable waste of memory traffic. By confining the access range to the regular vertices, Mixen avoids isolated vertices and therefore results in better performance. In addition, the factor $\alpha = 0.53$. According to Eq. 6.2, it lessens random accesses by $\frac{1}{\alpha^2} = 3.56\times$ compared with PCPM.

In comparison with PuPR, the basic blocking approach (i.e., PB) causes slowdowns on graphs that enjoy high locality [101], which in our case are the graphs *wiki* and *rmat*. For instance, graph *rmat* is built with parameter $a = 0.6$. In other words, 60 percent of non-zeros fall into partition A of the R-MAT model [116], leading to high spatial locality.

The slowdowns result from an inherent drawback of Gather-Apply-Scatter propagation model, which is inherited by PB as well as PCPM and Mixen. In the GAS model and its derivatives, graphs are traversed twice in every iteration: once for Scatter and once for Gather. By contrast, in the pushing and pulling paradigms (e.g., PuPR), traversal occurs only once. When the gains from the improved cache-memory efficiency is outweighed by the loss from the repeated graph traversal, the performance is undermined, and vice versa. Mixen alleviates the drawback of GAS by moving a part of computations to the efficient pushing and pulling paradigms.

PCPM is the state-of-the-art block-oriented method for high-performance graph analytics, which is referred to as partition-centric work by the authors. Viewed from the perspective of SpMV, it enhances prior cache blocking methods via the use of masked bins. This substantially shrinks the size of data array and thus reduces memory traffic. As a result, it yields the best result amongst the selected benchmarks.

Mixen extends its predecessors with more efficient graph representation and the removal of redundant computations. The memory traffic is further decreased. In following Section

Table 6.5: Execution time (in seconds) of PageRank with various implementations. Symbol * stands for running out of memory, thus obtaining no result.

Skylake						
	Mixen	PCPM	PB	PuPR	GBBS	GraphMat
<i>weibo</i>	0.185	3.144	7.618	8.077	8.105	8.319
<i>track</i>	1.310	2.911	4.571	2.316	4.091	4.593
<i>wiki</i>	1.335	1.634	3.726	2.248	3.081	3.461
<i>pld</i>	2.426	3.254	7.075	9.743	13.524	12.737
<i>rmat</i>	1.239	2.051	3.991	1.847	1.416	3.925
<i>kron</i>	1.571	1.983	5.883	6.490	7.106	7.312
Haswell						
	Mixen	PCPM	PB	PuPR	GBBS	GraphMat
<i>weibo</i>	0.190	3.770	18.995	7.461	6.844	10.284
<i>track</i>	1.594	3.126	6.464	5.255	3.594	6.058
<i>wiki</i>	1.450	1.699	3.955	4.049	3.050	4.310
<i>pld</i>	5.579	5.911	20.643	8.666	21.661	*
<i>rmat</i>	1.076	1.989	2.366	3.320	1.354	5.224
<i>kron</i>	3.541	4.048	11.421	6.597	12.579	*

6.5.3, we validate our analysis on the memory behaviors of different methods. Also, the discussion relating to Haswell is omitted because similar results can be acquired.

6.5.3 Memory Traffic

We only report the memory performance of the hand-coded implementations running on Skylake processor. The frameworks tend to involve with more design factors and complicate the discussion. Hence, they are neglected. Fig. 6.4 depicts the memory traffic volume normalized by the number of graph edges during 20-iteration PageRank. The traffic is broken down into reads (the lower shadowed bars) and writes (the upper bars).

It can be observed that Mixen produces the least amount of traffic, which explains why it achieves the shortest execution time. On average, Mixen generates $2.90\times$ and $4.26\times$ and $7.56\times$ less than PCPM, PB and PuPR respectively. The memory traffics of Mixen, PB and PCPM show a steady uprising, which matches the tendency of their execution time.

PuPR oftentimes produces drastically higher memory traffic than other methods, which leads to slower processing speed. The exceptions are the high-locality graphs *wiki* and *rmat*, of which the accesses to source vertices are highly likely to remain in cache. On them, PuPR invokes lightweight memory traffic and yields better performance than PB.

Furthermore, compared with these cache blocking methods, PuPR incurs substantially

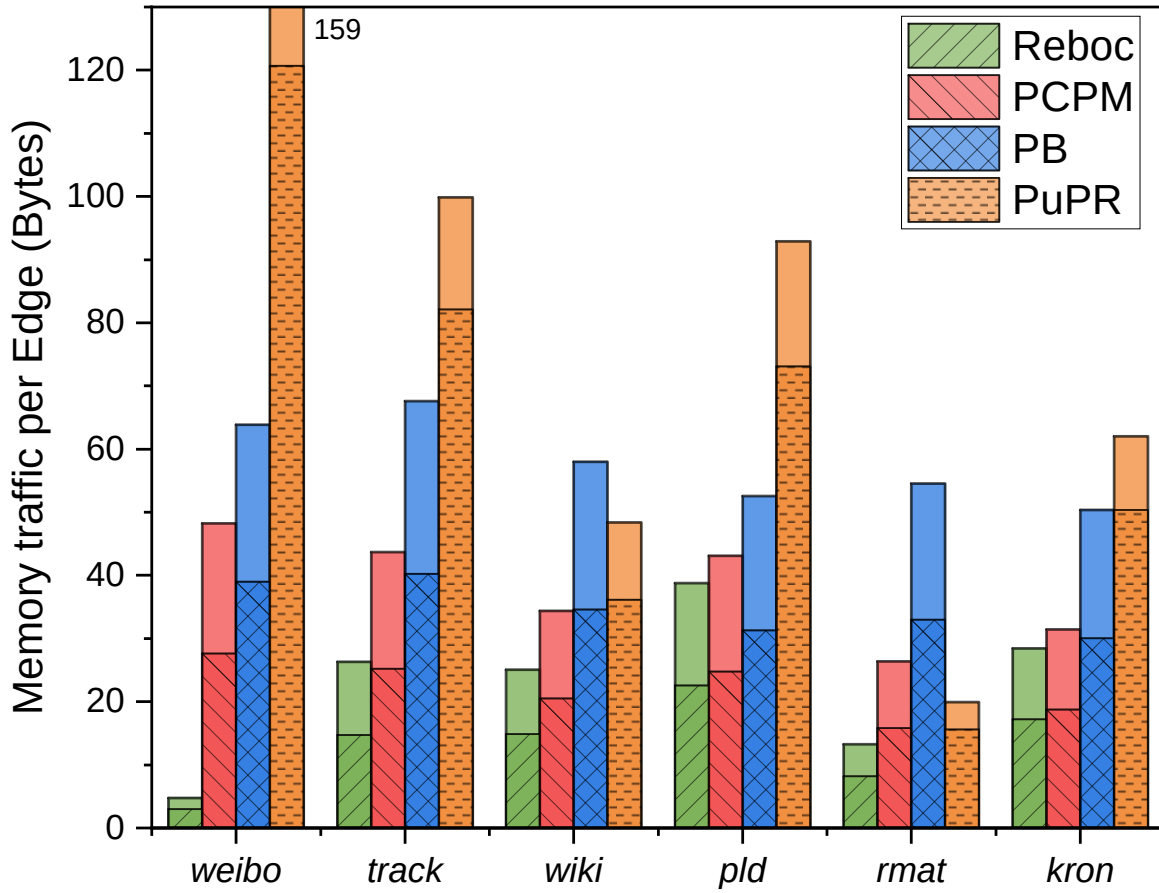


Figure 6.4: Normalized memory traffic. The lower shadowed bars are the reads and the upper bars are the writes.

more read operations, which compose 79 percent of memory traffic on average. This is because the reading (i.e., pulling) of the source vertex rank score is arbitrary and unpredictable. The randomness leads to poor cache locality and high memory traffic.

On the contrary, for the block-based methods, the reading range of each thread is confined within a cache-sized block. This effectively reduces random jumps on memory space, and therefore retain operations within cache. Thereby, for Mixen, PCPM and PB, the read operations account for 60, 59 and 60 percent of overall traffic respectively, far lower than PuPR.

However, the GAS-like propagation model, adopted by the cache blocking methodology, induces the writes not only to the updates of vertices, but also to the data buffer of blocks. Hence, it tend to invokes more writes than pulling SpMV, which would degrade the processing speed.

6.5.4 Block Size

The block size plays a important role in determining the performance of Mixen. The choice of block size symbolizes a sophisticated tradeoff between workload distribution, cache locality and memory traffic. In Fig. 6.5, experimental results demonstrate that the *side length* of a block best fits within either L2 cache (for *track pld* and *kron*) or L1 cache (for *weibo*, *wiki* and *rmat*).

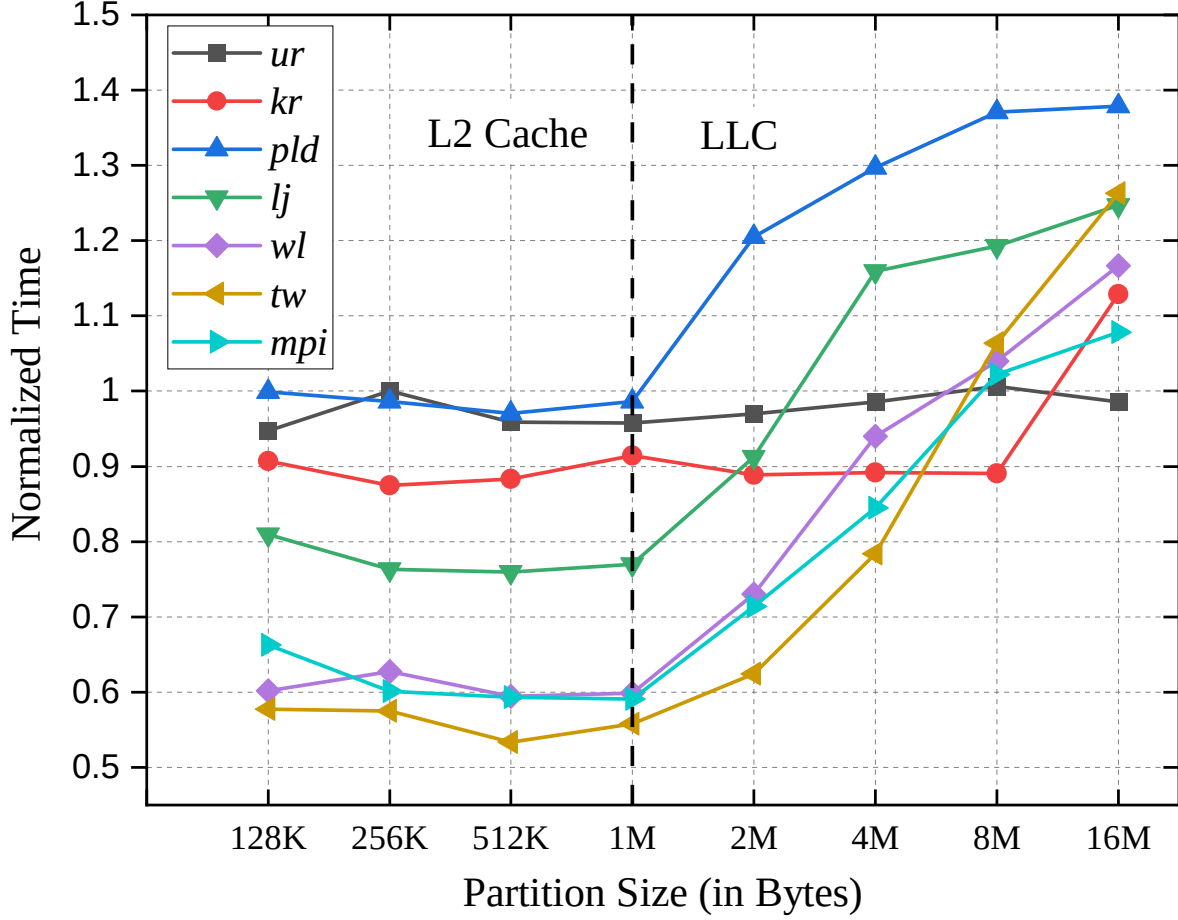


Figure 6.5: The execution time with varied block size. The results are normalized by the longest execution time.

The larger the block, the better the compression of propagation, and hence the fewer the data movement. At the extreme, if the whole graph is contained by one block, all the outgoing edges of a vertex can be represented by a single message to its leader dst vertex. In this scenario, $nb = 1, r = nreg / nnz$.

Nevertheless, when the bins are not containable by L2 cache anymore, the performance starts to decline. The access to the data is spilled over onto L3 cache and main memory, thus

increasing the latency. For instance, in Fig. 6.5, the increases of LLC hits (starting from 256 KB) and the memory traffic (starting from 1 MB) correspond to the growing execution time of *pld* (starting from 256 KB) in Fig. 6.5. In addition, the workload per thread is incremented and the number of parallel tasks is reduced, which is harder for the dynamic scheduler of OpenMP to balance.

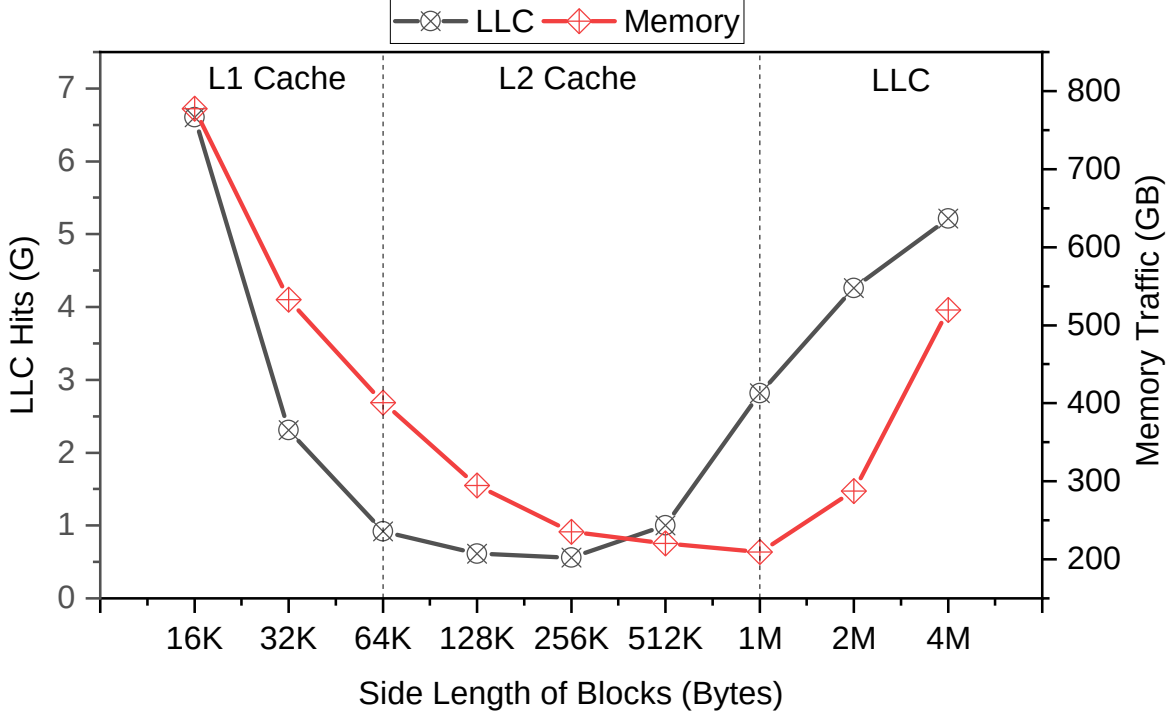


Figure 6.6: LLC hits and memory traffic with different block size on graph *pld*

On the opposite, the decrease of block size promotes high cache locality by confining data access inside cache. Also, smaller blocks yield a fine-grained workload allocation amongst threads, thus enhancing parallelism of the graph task. However, this also hurts the effectiveness of compression, surging up the memory traffic again. For example, in Fig. 6.6, both LLC hits and memory traffic are elevated as the block size shrinks from 256 KB to 16 KB. A corner case is, when every block contains exactly one edge, messages will never be compressed; that is, $nb = nreg, r = 1$.

For graphs with a large number of regular vertices (e.g., *track pld* and *kron*), the side length of 256 KB is the preferred configuration of blocks. As depicted in Fig. 6.6, it fits into L2 cache and reaches a compromise between traffic and cache locality. We set it as the default parameter for Mixen.

However, for graphs with insufficient regular vertices, the block size of 256 KB results in too few blocks. This prevents multiple threads from being effectively utilized. For example,

only 7 blocks are built after the filtering of *weibo*, while there are totally 40 threads available in our multicore systems. In this case, 33 out of 40 threads are waiting idly, causing a waste of resources. To address this issue, smaller size of blocks is favored, because it generates more blocks and promotes fine-grained parallelism. For example, fixing the block size to 32 KB, there are 53 blocks for *weibo*.

6.5.5 Thread Behavior

In this section, we dig into the thread behavior and related memory activities, which reflects the scalability of Mixen on multicore systems. The exploration is exemplified on graph *pld* as presented in Fig. 6.7. By adjusting the number of threads, the time improvement over the 2-thread baseline is collected, together with the sustained memory bandwidth.

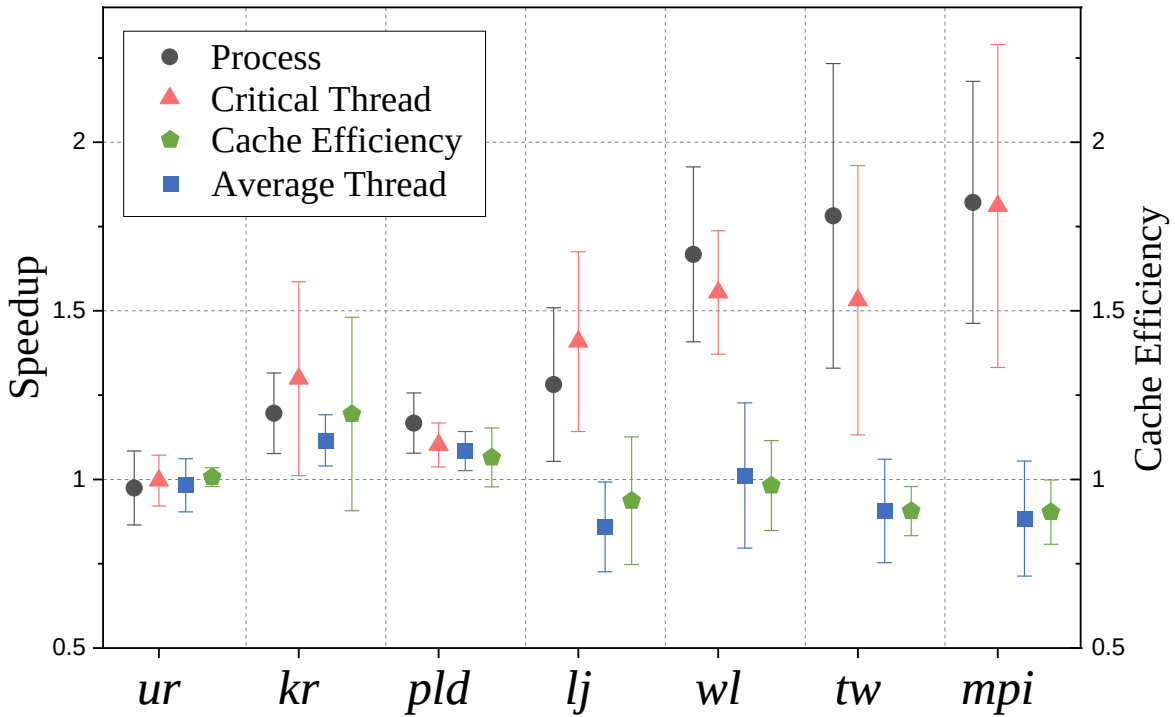


Figure 6.7: Sustained memory bandwidth and speedup (over the 2-threaded result) with different number of threads on graph *pld*.

Mixen affects the memory bandwidth at two sides. On one hand, it effectively reduces random memory accesses (previously discussed in Section 6.4), which benefits memory throughput performance. On the other hand, the cache-sized bins lead to high-volume memory transactions, thus draining the memory bandwidth.

As displayed in Fig. 6.7, Mixen undergoes steady improvement as thread number grows from 2 to around 20 (i.e., the number of physical cores). In this region, sharp

increases are observed in both the speedup and the memory bandwidth usage. However, the memory bandwidth is projected to cope with the demands of physical cores rather than the logical ones (i.e., threads) [103]. Hence, as the thread number continues to grow, that is $\#thread > 20$, the utility of bandwidth is saturated. Thus, the addition of threads provides trivial performance improvement or even deterioration.

6.6 Related Work

SpMV. SpMV has been drawing enormous research attention over the past decades for diverse computing architectures, including CPUs [67, 68, 69, 70] and GPUs [71, 72, 73, 74]. The essential challenge of SpMV arises from the massive number of zeros in the sparse matrix, which are not engaged in the computation. Hence, the matrices are commonly represented in compressed formats that contain only the non-zeros [75, 76].

However, there is no single format universally suitable for all SpMV algorithms on all architectures. The selection of matrix representation depends on the specific tasks. For instance, when the non-zeros are well structured and form a few matrix diagonals, Diagonal (DIA) is a favored choice [77]. Ellpack (ELL), allowing a general pattern of matrices, suits for regular SIMD programming [78]. For the irregular matrices, Coordinate List (COO) copes with the problem of load imbalance [79]. Furthermore, Hybrid (HYB) approach is devised by decomposing the matrix into two parts of regular ELL and irregular COO [80], which expresses a similar idea to CSMix.

The duality between graphs and matrices remarkably stimulates the advancement in both domains. Graph algorithms, such as PageRank and BFS, serve as the representative SpMV applications in SpMV papers [84]. In reverse, optimized SpMV operations also inspire the high performance of graph processing [82], e.g., our work.

Graph Processing. Mixen belongs to the category of shared-memory graph processing. In this case [4, 8, 35, 101, 6, 7], a modern server is deployed, equipped with sufficiently large memory and multiple cores, for example, 256GB and 20 cores. Thus, graphs can be fitted into the memory and globally shared by all threads.

When the memory volume of multicore systems is not able to accommodate large-scale graphs, the graph processing can be scaled up to distributed systems [19, 118, 119]. The distributed-memory processing requires explicit graph partitioning, so that subgraphs are locally handled by each node. However, the synchronization among subgraphs incurs expensive communication overheads across the network. Thus, the distributed work sometimes deliver suboptimal performance in comparison with the shared-memory counterpart [19] or even a single-threaded implementation [23].

Besides, the graph processing can also be offloaded to disks when the dataset can not fit into the memory [12, 13, 14], which is defined as the external-memory system. The disk

stores partial graph data and intermediate results during the execution of programs. A main design concern for such system is the efficient utilization of disk (e.g., improving disk I/O bandwidth) for data streaming [11].

6.7 Summary

In this chapter, we propose a pair of offline formatting and online processing optimizations to accelerate SpMV on graphs. A mixed format, named *CSMix*, is firstly introduced, which combines CSC, CSR and DCSR formats to represent a graph. Graph components with different connectivity pattern are encoded in distinct formats. The deployment of *CSMix* naturally produces a filtered graph containing only regular vertices and enjoying high locality.

Then, a processing engine *Mixen*, is explicitly designed for the efficient utilization of *CSMix*. It schedules the processing procedure into three phases to avoid unnecessary computation and data movement. Also, the filtered graph is decomposed into cache-sized blocks. These blocks are iteratively processed in the model of Scatter-Cache-Gather-Apply (SCGA).

Mixen can be extended as an add-on to contemporary graph systems, such as GraphMat [82] and Graphit [30], for performance improvement. Also, it broadens the optimization space of SpMV. In the future, it is worth to explore the Blocking technique and SCGA paradigm to optimize SpMV on GPUs. The caching, delaying and ignoring of seed, sink and isolated vertices are expected to an effective and efficient approach to utilize GPU memory, which is a scarce resource. Considering the distinctive memory hierarchy between GPUs and CPUs, the partitioning strategy for the graph may rely on an alternative cache level, e.g., L1 cache.

Chapter 7

Locality Extraction & Blocking

Inspired by graph reordering and matrix blocking techniques, we propose *Locality Extraction & Blocking (Lebo)*, a novel optimization system that accelerates SpMV on graphs [44]. Lebo extracts locality from power-law graphs by lightweight reordering, and apply blocking on the high-locality dense portion. In following sections, we firstly introduce the background knowledge (a brief rewind for Chapter 2), and then offers details in the design of Lebo.

7.1 Background

Graph Reordering. To enhance the access locality of graphs, graph reordering methodology is proposed, which rearranges the order of computations by relabeling vertex IDs. Graph reordering functions as an preprocessing step to boost downstream graph tasks. For practicality, the reordering time should be sufficiently short to justify its contribution.

The reordering algorithms can be categorized into two types: *degree-based* and *structure-based*. Degree-based algorithms solely rely the values of vertex degrees to redistribute vertices into different segments, for instance, Hub Clustering (HC) [35], Frequency-Based Clustering (HBC) [52] and Degree-Based Grouping (DBG) [53]. Structure-based reordering conducts in-depth investigation into the internal structure of graphs. Rabbit Order explores the hierarchical communities in a graph [37]. Gorder extensively calculates the locality of vertex pairs [38]. Because of the additional analysis of structural knowledge, the structure-based methods tend to cost more time than the degree-based ones.

Matrix Blocking. We refer to Matrix Blocking as an encapsulation of SpMV optimizations built on Blocking. Blocking is a long used optimization technique leveraging memory hierarchy [120, 121]. Blocked operations target on cache-able submatrices instead of an entire row or column. Therefore, cached data are more likely to be reused.

Then, Binning creates (*contribution, destination*) pairs to buffer the updates of each block [104, 101]. As a result, the random accesses to the destinations become sequential and

consecutive, enjoying high spatial locality. Edge Compression wraps edges from a source to multiple destinations belonging to the same block into a single edge [93]. It is worthy noting that standard SpMV operation $y = A \cdot x$ is executed in row-major order with CSR.

7.2 Design of Lebo

To utilize the locality of a graph, Lebo distinguishes the dense and sparse submatrices within the adjacency matrix by classifying vertices into three groups. As illustrated in Fig. 7.1, submatrices are executed under different paradigms. For the high-locality dense portion, Lebo performs block-based SpMV operations and propagates the updates in pushing flow instead of the intended pulling flow. For the low-locality sparse portion, Lebo treats it as an ordinary graph and execute in pulling flow.

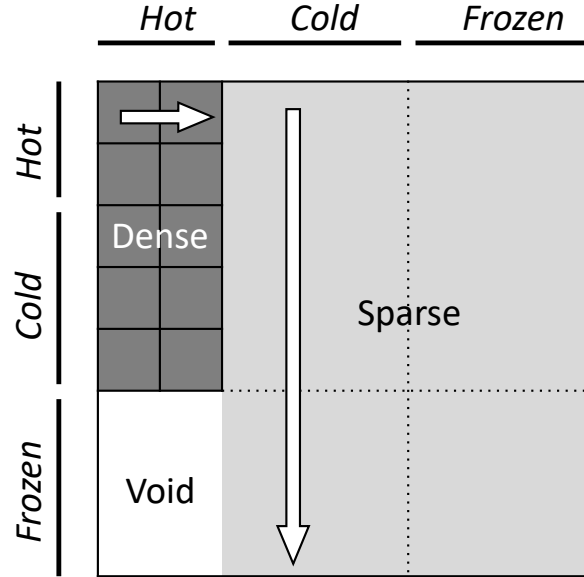


Figure 7.1: Concept of Lebo. Lebo distinguishes three types of vertices: hot, cold and frozen. Vertex order is reorganized for the generation of different submatrices: dense, sparse and frozen. The dense submatrix is divided into cache-able blocks that are outlined as boxes. During SpMV of graphs, dense submatrix enjoying high-locality is processed in row-major order, whilst sparse matrix is executed in column-major order.

Submatrix generation. The submatrices are generated by a lightweight reordering method, which categorizes vertices into three types: hot, cold and frozen. The re-organization of different vertex types produces dense, sparse and void submatrices.

High-locality dense submatrix for Blocking. The dense submatrix encompasses a vast majority of non-zeros in matrix A , e.g., 87% for graph *wiki*. The most crowded columns in the original matrix is selected to form the dense portion. For power-law graphs, the selected

columns only accounts for a small fraction of the original columns, e.g., 11% for *wiki*.

Furthermore, some empty rows inside the crowded columns (e.g., 51% for *wiki*) are moved to the bottom for compactness. They frame a void submatrix that is not engaged in computation. The isolation of void submatrix effectively shrinks the size of dense submatrix and avoids redundant loading of vertices. Afterwards, the dense submatrix is segmented into numerous cache-able blocks of *varying* sizes, which is to be described in Section 7.4.

Low-locality spare submatrix for pulling. The remaining columns of matrix **A** construct the sparse submatrix. This portion is characterized by low locality, leaving little opportunity for cache reuse. The optimization overhead could easily outweigh the performance gain. Hence, Lebo employs standard CSC-based pulling traversal.

7.3 Locality Extraction by Reordering

The building of dense and sparse submatrices depends on the classification of graph vertices. Three types of vertices are identified:

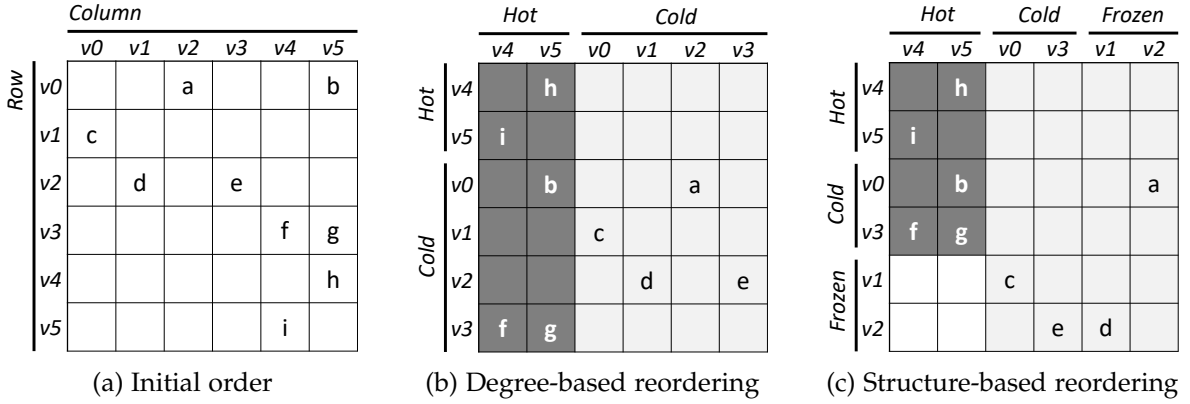


Figure 7.2: The reordering method of Lebo. Firstly hot vertices and cold ones are separated via degree-based reordering. Next, frozen vertices are identified from cold vertices via structure-based reordering. In (b) and (c), the dense, sparse and void submatrices are colored in dark grey, light grey, and white respectively.

- Hot vertices: with in-degree larger than the average degree of the graph.
- Cold vertices: with in-degree no larger than the average degree, and comprising at least one outgoing-edge to the hot vertices.
- Frozen vertices: with in-degree no larger than the average degree, but holding *no* outgoing-edges to the hot vertices.

Lebo applies a mix of *degree-based* and *structure-based* reordering approaches. At first, the degree-based reordering compares vertices' in-degrees over the average degree of the graph to locate hot vertices (i.e., the most crowded columns). Hot vertices are relabeled with consecutive IDs starting from index 0. Meanwhile, the relative order of hot vertices are preserved to retain original locality. The rest of vertices are marked as cold temporarily.

Fig. 7.2 shows the example of Lebo's reordering method. Fig. 7.2a is the adjacency matrix of a graph. The average degree of the graph is $nnz/n = 9/6 = 1.5$. The number of non-zeros in a column means the in-degree of the respective vertex. Thus, v_4 and v_5 are hot vertices, because their in-degree are 2 and 3. Using the degree-based reordering, they are shifted to the beginning of the vertex array as in Fig. 7.2b. Other vertices are tagged as cold.

Next, the structure-based reordering examines the incoming edges of hot vertices. If a cold vertex is *not* an in-neighbor of any hot vertex, then it turns frozen. The cold vertices are assigned with IDs larger than the hot but smaller than the frozen ones. Also, the relative order between vertex of cold as well as frozen stays the same as in the original graph. This leads to minimum disruption to initial graph order.

Continuing to the example of Fig. 7.2, it can be observed that cold vertices v_1 and v_2 carry no outgoing edge to the hot vertices v_4 and v_5 . Thereby, they are altered as frozen vertices. Finally, by the structure-based reordering, the frozen vertices v_1 and v_2 are relocated to the end of the vertex array, forming a void submatrix as delineated in Fig. 7.2c.

After classifying different vertex types, the dense and sparse matrices are ready for installation. Denote the numbers of hot, cold and frozen vertices as nh , nc and nf respectively, and $nh + nc + nf = n$. The row of the dense matrix enfolds the hot and cold vertices (as source), and the column matches with the hot vertices (as destination). Thereby, the size of the dense submatrix is $(nh + nc) \times nh$. For the sparse matrix, the row covers all vertices (as source) whilst the column includes the cold and frozen vertices. The size of the sparse matrix is $n \times (nc + nf)$.

7.4 Blocking on The Dense Submatrix

Leveraging the high locality, Lebo subdivides the dense submatrix into numerous 2-D blocks fitting into the caches. Unlike prior Blocking techniques that keep the same size for all blocks [104, 101], Lebo provides an elastic resizing method for workload balance. The subdivision procedure of Lebo consists of two steps:

(1) *Square the blocks*. Initially, Lebo follows the convention of Blocking that performs an equal-sized partitioning method. The row height and column width of every block (denoted as rb and cb respectively) are equal to cache size c ; that is, $rb = cb = c$. Each block serves as the basic data unit passed to a thread for processing. Considering a block as an element, the dense submatrix of size $(nh + nc) \times (nh)$ is converted to a grid of $((nh + nc)/c) \times (nh/c)$

blocks. The optimal value of c is to be explored in Section 7.7.4.

An example is offered in Fig. 7.3 on the left side. Assume a dense submatrix of size 8×6 is acquired after Lebo's reordering, and the cache can accommodate 2 vertices. This submatrix can be divided into 12 blocks of size 2×2 . Conceptually, the blocks constitute a grid of size 4×3 .

(2) *Refine the columns.* The blocks obtained in Step (1) are all squares of the same size. However, during the formation of the dense submatrix, the most crowded columns (i.e., vertices with the largest in-degrees) tend to be concentrated in a few blocks. Consequentially, the writing operations (e.g., line 3 in Algorithm 1) performed by different blocks are highly disproportionate. For instance, in Fig. 7.3 Col. 3 of the grid containing 16 non-zeros, while Col. 1 and Col. 2 only have 2 and 3 non-zeros respectively.

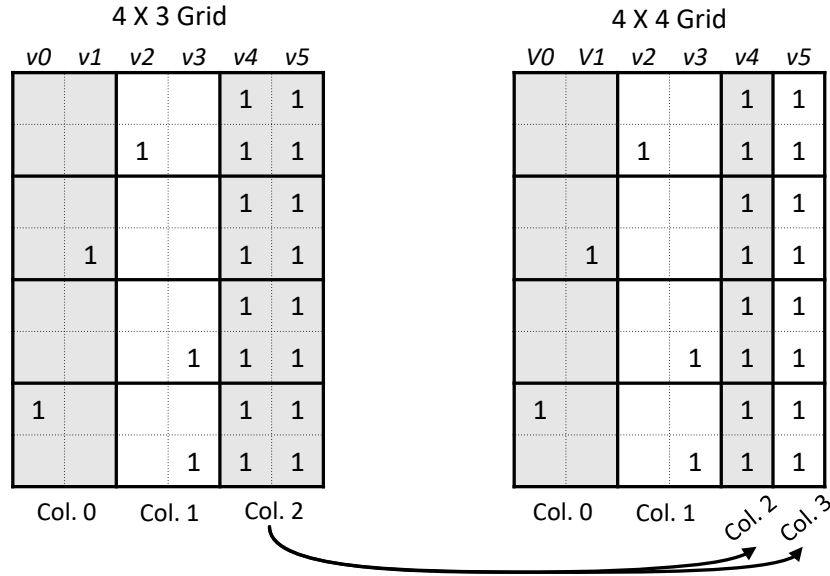


Figure 7.3: The refinement of columns by Lebo. The dense submatrix can be conceptually converted to a grid of blocks. The columns of the grid is painted with interleaved colors for readability. In initial grid on the left side, Col.2 is over-populated, so it is split into two narrower columns as Col. 2 and Col. 3 in the new grid on the right side.

To alleviate imbalanced workloads, Lebo adjusts the column widths of blocks by viewing the dense submatrix from the perspective of a grid. Inside the grid, an entire column of blocks is further subdivided into multiple sub-columns when it is over-populated. The condition of "over-populated" is whether the number of non-zeros in the k^{th} column of blocks $nnz(block[:,k])$ is strictly larger than 2 times of the average over all columns $\sum_{j=0}^{nc} nnz(block[:,j])$; that is, $nnz(block[:,k]) > 2 \cdot \sum_{j=0}^{nc/c} nnz(block[:,j])$. When the condition is met, the column width of these blocks is iteratively divided by 2 until the condition is violated.

In the given example of Fig. 7.3, the non-zeros for Col. 3 is $nnz(block[:,3]) = 16$, and

the average non-zeros over three columns is 7. Since $16 > 2 \cdot 7$, Col. 2 is subdivided into two sub-columns. As on the right side of Fig. 7.3, new Col. 3 is presented. Also, the size of blocks in Col. 2 and Col. 3 becomes 2×1 . It should be emphasized that the physical layout of the dense matrix is not modified. Hence, the refinement of block columns incurs minimum overhead.

Furthermore, Lebo inherits two existing optimizations to facilitate the execution of SpMV:

Binning [104, 101]. Each block is allocated with bins recording the destinations and contributions from the source vertices in this block. The bins are sorted, so the reading and writing to bins are sequential and consecutive. More importantly, the buffering of propagation eliminates the need for atomics in the pushing direction, e.g., Algorithm 2.

Edge Compression [93]. To reduce memory traffic, messages from a source vertex to multiple destinations in another block are compressed into a single edge. This kind of edges, which point from a source vertex to a destination block, are called *inter-edges*. Correspondingly, *intra-edges* are devised, which transfer the message from current block to local destination vertices.

The Binning and Edge Compression are integrated in one single procedure utilizing CSR of the reordered graph. Three bins are configured for each block during the first iteration of dense submatrix traversing: *inter*, *intra* and *data*. They are responsible for buffering the inter-edges, intra-edges and updates respectively. In subsequent iterations, *inter* and *intra* stay unchanged, and only *data* will be written for the propagation of updates.

7.5 SpMV with Lebo

7.5.1 Generic Execution Model

Lebo selectively executes SpMV on different matrix portions in distinctive paradigms. The dense submatrix is firstly processed as a grid of blocks in pushing direction. Thereafter, the sparse submatrix follows the convention of pulling traversal.

Algorithm 10 SpMV execution with Lebo

Data: gridWdt, gridHgt $\rightarrow$ grid width and height,
 colRng[] $\rightarrow$ vertex range of every column in the grid,
 inter[], intra[], data[] $\rightarrow$ bins,
 colPtr[], rowNdx[] $\rightarrow$ CSC representation of graph

```

1: /* Dense Submatrix */
2: for all i=0 to gridHgt-1 do in parallel
3:   for j=0 to gridWdt-1 do
4:     Scatter(scatterF, inter[i][j], data[i][j])
5: for all i=0 to gridWdt-1 do in parallel
6:   for v=colRng[i].first to colRng[i].last do
7:     Reset(resetF, v)
8:   for j=0 to gridHgt-1 do
9:     Gather(gatherF, intra[j][i], data[j][i])
10:  for v=colRng[i].first to colRng[i].last do
11:    Apply(applyF, v)
12: /* Sparse Submatrix */
13: for all i=colRng[gridWdt-1].last to n-1 do in parallel
14:   Reset(resetF, i)
15:   for j=colPtr[i] to colPtr[i+1]-1 do
16:     Pull(pullF, i, rowNdx[j])
17:   Apply(applyF, i)

```

Algorithm 10 describes the execution steps within one iteration of SpMV with Lebo. gridWdt and gridHgt are the constants specifying the size of the grid in the unit of blocks. colRng[i] encapsulates the first and the last vertices (denoted by vertex ID) of the i^{th} column in the grid. For the instance of the grid on the left side in Fig. 7.3, gridWdt = 3 and gridHgt = 4; colRng[1].first = v_2 and colRng[1].last = v_3

The processing paradigm for the dense submatrix can be modeled as Scatter-Reset-Gather-Apply (SRGA). Each phase of the model corresponds to an interface: Scatter, reset, Gather and Apply. User-defined functions are called by the interfaces, which is to be explained in the next section. In the first parallel region (line 2-4), the updates of source vertices (i.e., hot and cold vertices) are pushed to the buffer spaces of destination blocks via inter-edges by Scatter. In the second parallel region (line 5-11), the destination vertices (i.e., hot vertices) are reset by Reset. Next, the buffered data are aggregated to destination vertices via intra-edges by Gather. Finally, Apply updates the result of aggregation.

The third parallel region continues to execute the sparse submatrix (line 13-17). The

paradigm for this submatrix adheres to the pulling SpMV of Algorithm 1. Interfaces `Reset` and `Apply` are deployed again for computing updates. Besides, an interface `Pull` is added, which supports the CSC-based pulling traversal. Thereby, we generalize the overall processing model as SRGAP.

7.5.2 Programmability of Lebo

The interfaces of Lebo allows an intuitive expression of various SpMV-based algorithms. According to the five different interfaces of the Lebo execution model, respective functions are required too. Functions `scatterF` and `gatherF` handle data propagation, whereas `resetF` and `applyF` undertake data computation. Function `pullF` plays a co-joint role.

Algorithm 11 PageRank in Lebo

```

1: Structure PageRank {
2:   prSrc[] = {1/n}
3:   prDst[] = {0}
4:   Function scatterF(v)
5:     return prSrc[v]/Deg[v]
6:   Function resetF(v)
7:     prDst[v] = 0
8:   Function gatherF(v, update)
9:     prDst[v] += update
10:  Function applyF(v)
11:    prDst[v] = (1-d)/n + d·prDst[v]
12:  Function pullF(v, u)
13:    prDst[v] += prSrc[u]
14: }
15: while !converged and !max_iter do
16:   SRGAP(PageRank)
17:   swap(prSrc, prDst)

```

Here, we take PageRank [85] as an example. PageRank iteratively computes the rank scores of all vertices. Its linear algebra form is $\mathbf{y} = \frac{1-d}{n}\mathbf{1} + d\mathbf{A}^T\mathbf{x}$. The score per vertex is obtained by accumulating the scaled scores (by out-degree) of all in-neighbors. Subsequently, the accumulated sum is normalized by the damping factor d .

Lebo translates PageRank into the code of Algorithm 11. Inside the structure, Function `scatterF` distributes the scaled rank scores of source vertices to bins. `resetF` resets the scores of destination vertices to 0 for preparation. `gatherF` sums the updates from bins and writes to the destination vertex. `applyF` normalizes the sum by d and adds a constant. `pullF` directly fetches the update from source vertex to destination vertex.

The user-defined functions are passed to the execution model SRGAP (line 16). In the end of every iteration, the pointers to the score arrays `prSrc` and `prDst` are swapped to

reutilize allocated memory.

7.6 Design Rationale for Lebo

Reordering based on the in-degree rather than the out-degree. Many power-law graphs exhibit asymmetric distributions in terms of in-degree and out-degree; that is, the highest in-degree is significantly larger than the highest out-degree. This reflects real-world phenomena widely observed in web graphs and social graphs. For example, in social networks, a few celebrities attract over tens of millions of followers worldwide, while there is no human user following the same number of other users.

Hence, if the graph is reordered based on the out-degree, the number of hot vertices will be increased to compensate for the decreased degree values. As a result, the dense submatrix will be "diluted", and turn into a larger-sized submatrix. In the case of graph *wiki*, the highest in-degree and out-degree are 631.4K and 9.3K respectively. After in-degree and out-degree reordering, the number of hot vertices are 2.0M and 3.5M respectively. To keep a compact layout for the dense portion, the in-degree is decided as the basis for Lebo's reordering method.

Refining the columns rather than rows. For the grid (i.e., dense submatrix), the column width is smaller than the row height. Hence, the measurement and subdivision on the columns is substantially simpler. Moreover, it is the columns, representing the high in-degree vertices, that are ordered at the first place, and therefore cause the workload imbalance. Hence, vertical refinement on the columns becomes a natural choice.

7.7 Evaluation

The performance evaluations are conducted on a server-class multicore system running on Ubuntu 20.04. It contains two Intel Xeon Silver 4210, providing 20 physical cores and 40 logical cores. The volumes of RAM, LLC, L2 and L1 caches are 256 GB, 20 MB, 1 MB, and 64 KB respectively.

The code of Lebo implemented in C++17, compiled using G++ 10.3 with optimization level 3. The parallelization is enabled by OpenMP library [117]. The hardware performances are profiled by Perf [95] and Likwid [96].

Lebo is compared with three other graph frameworks by performing PageRank as the SpMV-based graph traversal. GraphMat [82] optimizes graph processing based on a set of matrix operations. GPOP [8] incorporates the optimizations of Blocking, Binning and Edge Compression on the whole graph adjacency matrix. Ligra allows graph traversal in both pushing and pulling direction. Additionally, an hand-optimized baseline (Pull) is

implemented by following the processing paradigm of pulling SpMV in Algorithm 1. For consistency, all tests run for 5 rounds of 20 iterations.

The effectiveness of Lebo is examined over seven large-scale graph datasets. Graphs *live* [106], *twitter* [50] and *mpi* [51] derive from online social networks. Graphs *wiki* [107], *track* [105], *sd* [47] and *uk* [49] are crawled from hyperlinks of websites. Graph *kron* [46] is a synthetic graph. Graphs are represented in both CSR and CSC. The number of vertices, edges and the maximum in-degree and out-degree are listed in Table 7.1.

Table 7.1: Graph attributes. The number of vertices and edges, and the highest in-degree and out-degree (K: Thousand, M: Million, B: Billion.)

Graph	Vertices	Edges	Max In.	Max Out.
<i>live</i> [106]	7.5M	112.3M	1.1M	300
<i>wiki</i> [107]	18.3M	172.1M	631.4K	9300
<i>twitter</i> [50]	41.7M	1.5B	3.0M	770.1k
<i>mpi</i> [51]	52.6M	2.0B	3.5M	780.0K
<i>kron</i> [46]	67.1M	2.1B	1.0M	1.0M
<i>host</i> [47]	101.7M	2.0B	2.3M	1.3M
<i>uk</i> [49]	105.2M	3.3B	728.3K	262.8K

7.7.1 Execution Time

Table 7.2 presents the execution time for various implementations of PageRank, which performs typical SpMV calculations. Lebo exhibits considerable performance advantage, delivering average speedups over Ligra, GraphMat¹, GPOP and Pull by $6.80\times$, $3.05\times$, $1.43\times$ and $1.89\times$ respectively.

Table 7.2: PageRank execution time per round (in Seconds)

Graph	Ligra	GraphMat	GPOP	Pull	Lebo
<i>live</i>	16.20	1.73	2.04	0.75	1.87
<i>wiki</i>	6.80	3.05	1.43	1.89	0.97
<i>twitter</i>	39.75	17.38	8.53	11.47	6.24
<i>mpi</i>	92.62	29.96	15.41	22.22	9.16
<i>kron</i>	110.20	30.72	9.83	29.28	8.11
<i>host</i>	99.28	43.23	12.53	27.27	11.70
<i>uk</i>	54.57	/	8.38	5.38	7.78

¹the size of graph *uk* exceeds the capacity of GraphMat

Ligra shows unsatisfactory performance, because it stores graphs as adjacency lists in discrete memory space. When traversing the graph, this leads to inefficient memory accesses. In other works, graphs are formatted in CSC/CSR. When SpMV operations are performed, the accesses to the `colPtr/rowPtr` and `rowNdx/colNdx` arrays are completely sequential and consecutive, thus achieving higher memory efficiency.

GraphMat internally relies on the pulling SpMV (Pull) for the processing of graphs. Pull eliminates the complex designs of frameworks, such as interfaces for supporting multiple algorithms. Hence, Pull is always faster than GraphMat.

GPOP optimizes the SpMV operations by integrating the techniques of Blocking, Binning and Edge Compression, which enhance cache locality, promote sequential accesses and reduce memory traffic. Consequently, it yields better results over the baseline Pull on the majority of graphs.

Lebo improves existing SpMV optimizations by employing a selective processing mechanism on the graph adjacency matrix. The sparse portion of the adjacency matrix acquires the simplicity and efficiency of pulling SpMV. Meanwhile, the dense portion benefits from the refined Blocking method. Together, the dual paradigms remarkably boost Lebo.

7.7.2 Decomposition of Lebo

The execution of Lebo is decomposed to differentially process the dense and the sparse submatrices. Table 7.3 presents the size ratio of the dense submatrices to their belonging graph adjacency matrices. The column widths of the dense submatrices constitute a tiny ratio of the adjacency matrices, ranging from 4.3% to 16.7% (on average 10.2%). By contrast, the row heights frame a significant ratio from 41.6% to 92.9% (on average 63.5%). Due to the narrow columns, the areas of the hot portions are rather small, only accounting for 1.8% to 12.3% (on average 6.8%) of the adjacency matrices.

Table 7.3: Size ratio (%) of dense submatrix to the adjacency matrices

Graph	<i>live</i>	<i>wiki</i>	<i>twitter</i>	<i>mpi</i>	<i>kron</i>	<i>sd</i>	<i>uk</i>
Column	4.3	10.9	11.4	9.5	8.4	10.2	16.7
Row	41.6	49.1	92.9	75.3	48.2	64.3	73.4
Area	1.8	5.4	10.6	7.2	4.0	6.6	12.3

Given the small areas (e.g., on average 10.2%), the dense portions contain a massive number of non-zeros, covering from 79.0% to 91.9% (on average 85.1%) elements. Fig. 7.4 plots the contributions of the non-zero elements and of the execution time by Lebo's dense and sparse submatrices. Accordingly, the dense portions consume a major fraction of execution time, from 53.8% to 91.5% (on average 69.6%).

It should be noticed that the numbers of non-zeros inside the dense and sparse submatrices are disproportional to their corresponding execution time. The sparse portions cost up to 46.2% time while they occupy up to 21% non-zeros. This is because, on the one hand, the optimizations applied on the dense portions effectively reduce the processing time for it; on the other hand, the sparse portions suffer from low locality and therefore invoke more memory accesses and longer latency.

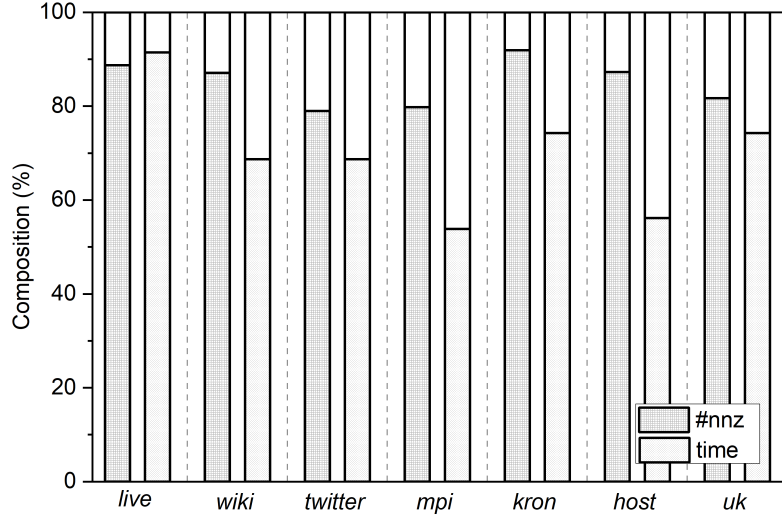


Figure 7.4: The contributions of #nnz and of the execution time by the dense and sparse submatrices. Upper empty bars: sparse submatrices. Lower shadowed bars: dense submatrices.

7.7.3 Preprocessing Overhead

The pulling SpMV (Pull) is free of preprocessing overhead because it directly runs on the CSC representation of graphs. On the contrary, the framework-based implementations oftentimes require substantial preparation time in order to achieve their optimization goals.

Table 7.4 lists the number of iterations performed by the pulling SpMV implementation to amortize the overheads of different frameworks, ignoring the data loading from disk. The overheads produced by Ligra, GPOP and Lebo are acceptable, which can be amortized by 10+ iterations. On the contrary, GraphMat imposes a staggeringly high cost, making itself impractical to deploy.

The preprocessing of Lebo includes two stages: graph reordering (Section 7.3) and dense submatrix blocking (Section 7.4). As in Table 7.4, the reordering stage consumes more time than the blocking stage does. This is because the former modifies the whole graph, while the later only access a subset (i.e., dense submatrix) of it.

Table 7.4: The preprocessing overhead of frameworks amortized by the number of the pulling SpMV iterations

Graph	Ligra	GraphMat	GPOP	Lebo		
				Reordering	Blocking	Overall
<i>live</i>	13.4	363.9	4.1	14.0	4.1	18.1
<i>wiki</i>	11.6	240.6	7.7	11.8	3.3	15.1
<i>twitter</i>	15.6	382.7	14.4	10.5	6.0	16.5
<i>mpi</i>	8.7	239.8	9.1	7.9	3.6	11.5
<i>kron</i>	6.9	195.6	16.3	6.4	2.1	8.5
<i>host</i>	7.3	211.2	24.8	6.6	2.4	9.0
<i>uk</i>	67.0	/	20.3	20.9	20.0	40.9
Average	18.6	272.3	13.8	11.2	5.9	17.1

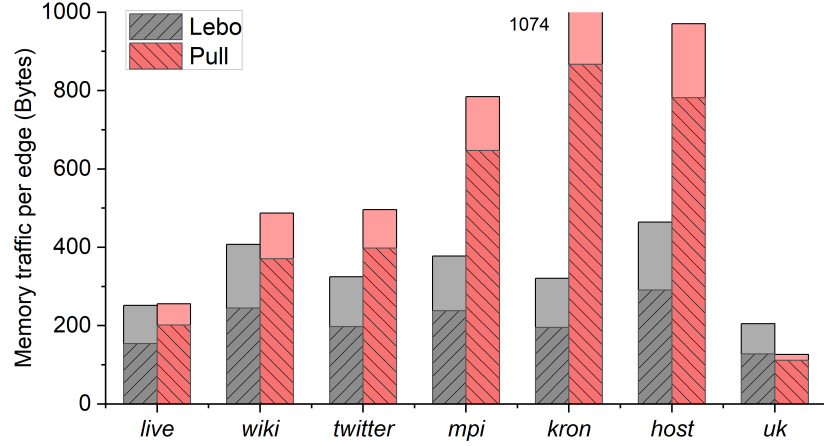
7.7.4 Memory & Cache Utility

Lebo poses non-trivial impact on the memory activity of the multicore system. Fig. 7.5a illustrates the normalized memory traffic volume generated by Lebo and Pull. Compared with the baseline Pull, Lebo aggressively decreases the memory traffic on many graphs except for *lj* and *uk*, which explains why it is slower on the two graphs.

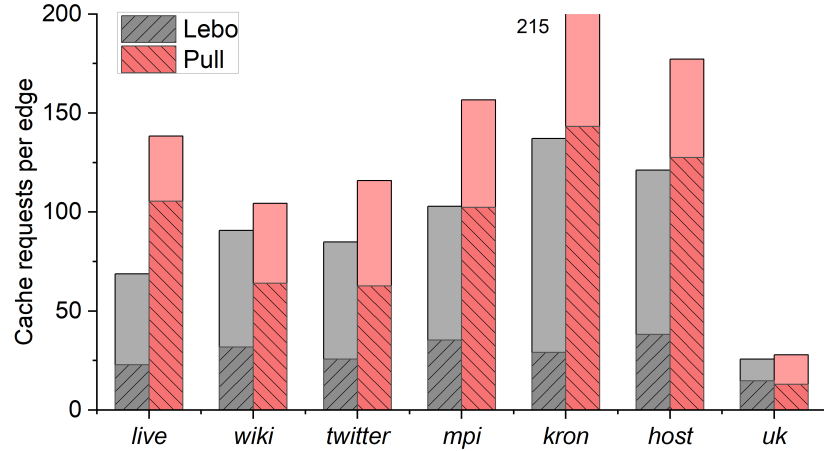
On average, Lebo generates $1.88\times$ less traffic than Pull does. As previously discussed, the pulling flow suffers from random memory reads, so the main traffic loads are the reads. Lebo propagates the dense submatrix that contains the majority of non-zeros in pushing direction, and thus invokes less (random) reads. Moreover, adopting the Binning and Compression optimizations, Lebo leads to a slight increase in writing operations instead of the expected drastic upsurge.

Lebo partitions the dense portion of a adjacency matrix into numerous cache-sized blocks. The cache-awareness effectively improves the temporal locality since the accesses of threads are constrained within caches. As depicted in Fig. 7.5b, the requests (hits + misses) to L2 cache are substantially decreased. The overall requests are reduced by $1.45\times$ on average of all graphs.

Furthermore, the total number of cache misses are declined by $3.01\times$. For the Pull implementation, cache misses account for 63% cache requests. The high miss ratio reflects the irregularity of graph structure. In comparison, Lebo decreases the cache miss ratio to 35%, which demonstrates Lebo's cache efficiency.



(a) Normalized memory traffic. Upper empty bars: writes. Lower shadowed bars: reads.



(b) Normalized cache requests. Upper empty bars: cache hits. Lower shadowed bar: cache misses.

Figure 7.5: Memory traffic volume and L2 cache requests, normalized by the number of edges.

7.7.5 Size Tuning

Lebo shapes the dense submatrix into 2-D blocks of differing sizes in two steps, as described in Section 7.4. Initially, the initial blocks are square-sized by $c \times c$, dependent on the cache size. In the following refinement, the row heights of blocks keep unchanged, while the column widths are divided into smaller units for workload balance, resulting in refined blocks of size $c \times \frac{c}{2^z}$, where $z = 1, 2, 4, \dots$ and obtained according to the density of column-wise blocks. Here, the optimal value of initial block size is of interest.

To determine the initial block size, sensitivity analysis is undertaken by tuning the size from 16 KB to 4 MB as exemplified in Fig. 7.6. For convenience, we use one-dimension length to represent the size of the initial block, since it is a square matrix. The execution time and cache hits to L2 and LLC are recorded as initial block size on the graph *wiki* are modified.

The performance of each indicator is normalized by its smallest result, so the minimum value of performance is always 1. When block size is 256 KB, Lebo achieves the shortest time and the highest number of L2 cache hits. As the block size expands, data accesses spill over from L2 into LLC (and memory), thus causing less hits to L2 cache but more hits to LLC. Thereby, the execution of Lebo is slowed down. Contrariwise, if the block size shrinks, cached data in L2 and LLC are unlikely to be reused. This causes fewer cache hits and more cache misses, therefore poorer performance.

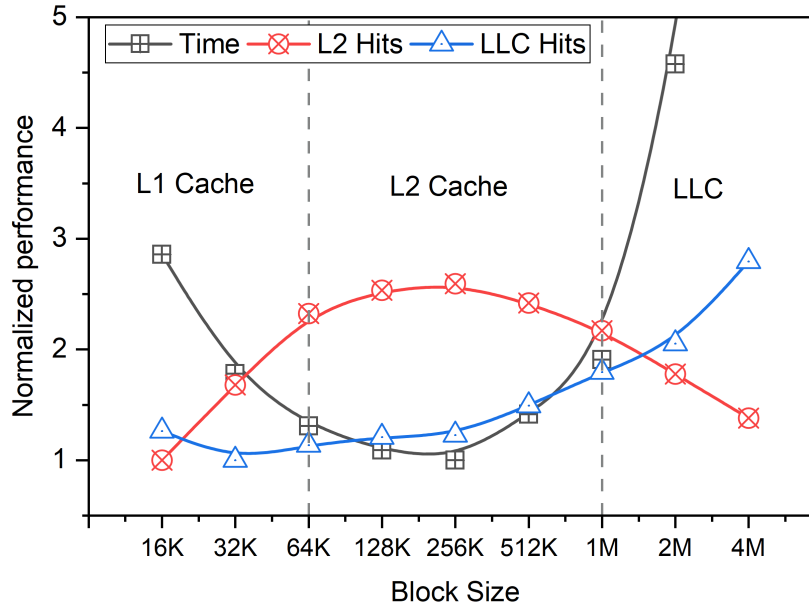


Figure 7.6: The execution time, L2 cache hits and LLC cache hits on graph *wiki*, normalized by the smallest results respectively.

7.7.6 Scalability

Lebo assigns cache-able blocks to threads with private ownership for execution. On the one hand, this allows better cache efficiency as discussed in prior sections. On the other hand, the cache-sized data transferred between memory and thread intensify memory throughput and thus thread contention. To attain the best performance, the multithreading behavior of Lebo is explored.

In Fig. 7.7, the performance of Lebo (normalized by the 1-threaded version) with varied number of threads are examined on graph *wiki*, in accordance with the memory bandwidth. The behavior indicates the scalability of Lebo on the multicore system. In this system, 40 logical cores (i.e., threads), or equivalently 20 physical cores, are available. As the number of deployed threads is smaller than 20, higher degree of parallelism promises faster speed.

Lebo reaches the peak performance when there are 20 working threads. Therefore, the proper number of threads for parallelization is 20, which equals to the number of physical cores in the multicore system. In this case, one thread is able to occupy a complete L2 cache without competing against another thread on the same physical core.

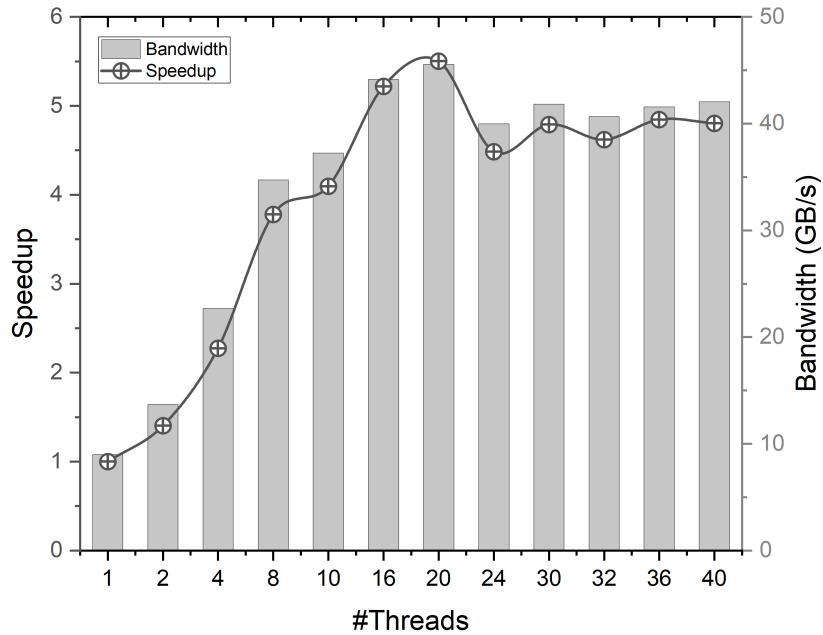


Figure 7.7: The speedup of Lebo with different number of threads over the 1-threaded version on graph *wiki*, and the corresponding memory bandwidth.

However, after the point of 20 threads, the bandwidth becomes saturated. Further enlargement of thread pool would only intensify the thread contention hardware resources, for example, burdening memory bandwidth, thus deteriorating the performance.

7.8 Summary

On modern multicore systems, a simple stack-up of graph reordering and matrix blocking optimizations fails to boost the SpMV on large-scale graphs following power-law degree distribution. To cope with this issue, this chapter proposes *Locality Extraction & Blocking* (Lebo), which extracts the locality out of the power-law graphs and applies refined blocking

on the graph adjacency matrix.

Lebo categorizes graph vertices into three types: hot, cold and frozen, based on the degree and structural knowledge of the graph. Then, the adjacency matrix are divided into dense and sparse submatrices, and operated in different paradigms. The dense portion is further partitioned into numerous blocks of varied sizes and processed in pushing flow. The sparse portion follows the convention of pulling flow for simplicity and efficiency. Compared with the best of state-of-the-art, Lebo achieves speedup by $1.43\times$ on average.

Lebo is extendable to graph frameworks, such as GPOP [8], GraphMat [82] and Graphit [30], for supporting the dense pulling traversal. Furthermore, Lebo's refinement on block size can be adopted by generic cache blocked SpMV to balance the workloads among submatrices.

Moreover, with the adaption of memory hierarchy, it is possible to apply Lebo to GPUs. Since the L2 cache of a GPU is globally shared [122], the L1 cache could be considered as the guideline for deciding the block size. Also, the effectiveness of graph reordering on GPUs is to be explored in the future.

Chapter 8

Conclusion and Future Work

8.1 Conclusion

The growing capacity and computing power of multicore machines allow for efficient and fast solutions to a wide range of large-scale problems. This thesis systematically studies the behavior of multicores for processing irregular graph workloads by characterizing graphs' intrinsic properties and their impacts on multicore activities. Unlike many prior works simply focusing on improving the cache locality and memory efficiency, this thesis demonstrates that thread balance and multicore parallelism also play non-negligible roles in affecting the efficiency of graph parallel computing. With insights into the fundamental factors determining the performance of graph processing, tradeoffs are weighed between design choices for delivering optimal solutions. The essential contributions of this thesis are outlined as follows:

At first, Corder investigates the workload imbalance amongst threads caused by the gathering of densely connected vertices (Chapter 3). To cope with this issue, Corder distributes computation-demanding graph vertices into cache-sized subsets, and inside each subset, those vertices are concentrated to enhance cache locality. Corder acts as a preprocessing operation applicable to a wide variety of graph analytics frameworks.

Secondly, Syze balances the workloads of threads via a different approach from Corder (Chapter 4). Firstly, Syze actively caches graph data by dividing the graph into subgraph of equal sizes. Afterwards, the workloads per subgraph are estimated, and the subgraphs suffering high loads are iteratively subdivided. To facilitate the communication across subgraphs and vertices, vertex ID are encoded so the propagation can be performed by simple bitwise instructions. Differing from Corder, Syze is a framework.

Thirdly, HiPa exploits the memory-cache hierarchy of modern multicore systems (Chapter 5). At the memory level, HiPa partitions a graph according to NUMA characteristics, so that the remote memory accesses are minimized. At the cache level, HiPa further segments

the graph components into cache-sized subsets to enhance cache locality. The cache-able subgraphs are pinned to the threads to co-locate computations and data. HiPa is designed with the goal of improving memory- and cache-efficiency. Though it is exemplified on PageRank, but can be promoted to generic SpMV as well as graph frameworks.

Fourthly, Mixen perceives graphs as matrices and analyzes how sparse matrix representation interacts with the graph traversing scheme (e.g., pushing or pulling) and thus leads to performance variability (Chapter 6). Based on the connectivity of vertices, a graph is reformatted to a new encoding format CSMix, which is specifically designed for Mixen to deploy a mixed graph traversing plan. Mixen is aimed to reduce computational redundancy.

Lastly, Lebo combines the techniques of graph reordering and matrix blocking to boost SpMVs on graphs (Chapter 7). Graph locality is extracted by constructing a dense submatrix from graph adjacency matrix via graph reordering. Then, the dense portion is segmented into 2-D cache-sized blocks for the improvement of cache locality and reduction of memory traffic. The sparse portion follows the classic CSC-based pulling flow to avoid over-optimization.

The five works are tightly coupled and mutually inspiring. Optimization techniques are commonly shared by them, including graph reordering, graph partitioning, matrix blocking, etc. Lebo integrates the ideas of its predecessors. It recasts a graph into a matrix (as Mixen), applies graph reordering to enhance locality (as Corder), deploys unequal-sized cache blocking to alleviate workload imbalance (as Syze).

Throughout this thesis, two central themes are repeatedly stressed: *access locality* and *workload balance*. The enhancement towards to memory and cache access locality is an obvious solution for accelerating shared-memory graph processing, and therefore has been extensively researched. Nevertheless, the issue of workload imbalance is rarely considered in multicore systems, which results in suboptimal performance for many frameworks. Across the proposed works in this thesis, the importance of load balance is raised along with access locality to ensure a smooth cooperation between the two factors.

8.2 Future Work

Promising future work could be built on this thesis.

Dynamic Graph Algorithms. The majority of discussed topics in this thesis (e.g., HiPa, Mixen, Lebo) are tested over stationary graph algorithms (e.g., PageRank). Meanwhile, many graph algorithms are dynamic where the active vertices are constantly changing in every iteration, such as BFS and SSSP. In Syze, dynamic algorithms are implemented with the support of hierarchic frontier, so that the inactive vertices will not be accessed. Hence, it is possible to merge Syze with other works for the construction of a more powerful and general framework.

Generic SpMV and SpMM. The SpMV optimizations described in Chapter 6 and 7 could be extended to standard SpMV as well as SpMM by adjusting the execution direction from column-major to row-major and adding one more layer of loop. Moreover, since both Mixen and Lebo generate memory-friendly dense submatrix, they are potentially collaborative with a typical SpMV optimization – vectorization.

Irregular Workloads on GPUs. A core pillar of this thesis is the exploitation of memory-cache architecture of CPUs on multicore systems. In comparison with CPUs, GPUs show remarkable difference in their nested memory hierarchy and massive thread parallelism. The difference challenges the effectiveness of graph reordering, which only tested on CPUs, and poses a critical question regarding the proper size of graph partitioning/blocking. A comprehensive evaluation is needed to find out the answers.

Distributed Graph Processing. Distributed computing oftentimes suffers from communication inefficiency and cross-node synchronization. Irregular graph workloads might further deteriorate the performance of the cluster, because many graph algorithms are communication-bound. It is not clear whether the computation efficiency obtained in the thesis can outweigh the communication inefficiency in the distributed environment. To optimize the distributed graph processing, it is necessary to conduct detailed investigations into network with respect to traffic volume, sustained bandwidth, bursty flow, etc.

References

- [1] P. J. Carrington, J. Scott, and S. Wasserman, *Models and methods in social network analysis*, vol. 28. Cambridge university press, 2005.
- [2] N. Kannan and S. Vishveshwara, “Identification of side-chain clusters in protein structures by a graph spectral method,” *Journal of molecular biology*, vol. 292, no. 2, pp. 441–464, 1999.
- [3] C. Barrett, K. Bisset, M. Holzer, G. Konjevod, M. Marathe, and D. Wagner, “Implementations of routing algorithms for transportation networks,” in *DIMACS Workshop on Shortest-Path Challenge*, Citeseer, 2006.
- [4] J. Shun and G. E. Blelloch, “Ligra: a lightweight graph processing framework for shared memory,” in *Proceedings of the 18th ACM SIGPLAN symposium on Principles and practice of parallel programming*, pp. 135–146, 2013.
- [5] K. Zhang, R. Chen, and H. Chen, “Numa-aware graph-structured analytics,” in *Proceedings of the 20th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pp. 183–193, 2015.
- [6] J. Malicevic, B. Lepers, and W. Zwaenepoel, “Everything you always wanted to know about multicore graph processing but were afraid to ask,” in *2017 {USENIX} Annual Technical Conference ({USENIX}{ATC} 17)*, pp. 631–643, 2017.
- [7] L. Dhulipala, G. E. Blelloch, and J. Shun, “Theoretically efficient parallel graph algorithms can be fast and scalable,” *ACM Transactions on Parallel Computing (TOPC)*, vol. 8, no. 1, pp. 1–70, 2021.
- [8] K. Lakhotia, R. Kannan, S. Pati, and V. Prasanna, “Gpop: A scalable cache-and memory-efficient framework for graph processing over parts,” *ACM Transactions on Parallel Computing (TOPC)*, vol. 7, no. 1, pp. 1–24, 2020.
- [9] S. Grossman, H. Litz, and C. Kozyrakis, “Making pull-based graph processing performant,” *ACM SIGPLAN Notices*, vol. 53, no. 1, pp. 246–260, 2018.
- [10] J. Sun, H. Vandierendonck, and D. S. Nikolopoulos, “Graphgrind: addressing load imbalance of graph partitioning,” in *Proceedings of the International Conference on Supercomputing*, pp. 1–10, 2017.

- [11] X. Zhu, W. Han, and W. Chen, “Gridgraph: Large-scale graph processing on a single machine using 2-level hierarchical partitioning,” in *2015 {USENIX} Annual Technical Conference ({USENIX}{ATC} 15)*, pp. 375–386, 2015.
- [12] A. Roy, I. Mihailovic, and W. Zwaenepoel, “X-stream: Edge-centric graph processing using streaming partitions,” in *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, pp. 472–488, 2013.
- [13] A. Kyrola, G. Blelloch, and C. Guestrin, “Graphchi: Large-scale graph computation on just a {PC},” in *10th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 12)*, pp. 31–46, 2012.
- [14] S. Maass, C. Min, S. Kashyap, W. Kang, M. Kumar, and T. Kim, “Mosaic: Processing a trillion-edge graph on a single machine,” in *Proceedings of the Twelfth European Conference on Computer Systems*, pp. 527–543, 2017.
- [15] D. Zheng, D. Mhembere, R. Burns, J. Vogelstein, C. E. Priebe, and A. S. Szalay, “{FlashGraph}: Processing {Billion-Node} graphs on an array of commodity {SSDs},” in *13th USENIX Conference on File and Storage Technologies (FAST 15)*, pp. 45–58, 2015.
- [16] C. Wu, G. Zhang, Y. Wang, X. Jiang, and W. Zheng, “Redio: Accelerating disk-based graph processing by reducing disk i/os,” *IEEE Transactions on Computers*, vol. 68, no. 3, pp. 414–425, 2018.
- [17] G. Malewicz, M. H. Austern, A. J. Bik, J. C. Dehnert, I. Horn, N. Leiser, and G. Czajkowski, “Pregel: a system for large-scale graph processing,” in *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data*, pp. 135–146, 2010.
- [18] R. Chen, J. Shi, Y. Chen, B. Zang, H. Guan, and H. Chen, “Powerlyra: Differentiated graph computation and partitioning on skewed graphs,” *ACM Transactions on Parallel Computing (TOPC)*, vol. 5, no. 3, pp. 1–39, 2019.
- [19] X. Zhu, W. Chen, W. Zheng, and X. Ma, “Gemini: A computation-centric distributed graph processing system,” in *12th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 16)*, pp. 301–316, 2016.
- [20] M. Zhang, Y. Wu, K. Chen, X. Qian, X. Li, and W. Zheng, “Exploring the hidden dimension in graph processing,” in *12th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 16)*, pp. 285–300, 2016.
- [21] C. Martella, R. Shaposhnik, D. Logothetis, and S. Harenberg, *Practical graph analytics with apache giraph*, vol. 1. Springer, 2015.
- [22] A. Ching, S. Edunov, M. Kabiljo, D. Logothetis, and S. Muthukrishnan, “One trillion edges: Graph processing at facebook-scale,” *Proceedings of the VLDB Endowment*, vol. 8, no. 12, pp. 1804–1815, 2015.
- [23] F. McSherry, M. Isard, and D. G. Murray, “Scalability! but at what {COST}?,” in *15th Workshop on Hot Topics in Operating Systems (HotOS {XV})*, 2015.

- [24] T. J. Ham, L. Wu, N. Sundaram, N. Satish, and M. Martonosi, "Graphicionado: A high-performance and energy-efficient accelerator for graph analytics," in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 1–13, IEEE, 2016.
- [25] A. Mukkara, N. Beckmann, M. Abeydeera, X. Ma, and D. Sanchez, "Exploiting locality in graph analytics through hardware-accelerated traversal scheduling," in *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 1–14, IEEE, 2018.
- [26] M. Yan, X. Hu, S. Li, A. Basak, H. Li, X. Ma, I. Akgun, Y. Feng, P. Gu, L. Deng, *et al.*, "Alleviating irregularity in graph analytics acceleration: A hardware/software co-design approach," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, pp. 615–628, 2019.
- [27] Y. Wang, A. Davidson, Y. Pan, Y. Wu, A. Riffel, and J. D. Owens, "Gunrock: A high-performance graph processing library on the gpu," in *Proceedings of the 21st ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pp. 1–12, 2016.
- [28] X. Shi, Z. Zheng, Y. Zhou, H. Jin, L. He, B. Liu, and Q.-S. Hua, "Graph processing on gpus: A survey," *ACM Computing Surveys (CSUR)*, vol. 50, no. 6, pp. 1–35, 2018.
- [29] S. Hong, H. Chafi, E. Sedlar, and K. Olukotun, "Green-marl: a dsl for easy and efficient graph analysis," in *Proceedings of the seventeenth international conference on Architectural Support for Programming Languages and Operating Systems*, pp. 349–362, 2012.
- [30] Y. Zhang, M. Yang, R. Baghdadi, S. Kamil, J. Shun, and S. Amarasinghe, "Graphit: A high-performance graph dsl," *Proceedings of the ACM on Programming Languages*, vol. 2, no. OOPSLA, pp. 1–30, 2018.
- [31] P. Faldu, J. Diamond, and B. Grot, "Domain-specialized cache management for graph analytics," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pp. 234–248, IEEE, 2020.
- [32] J. E. Gonzalez, Y. Low, H. Gu, D. Bickson, and C. Guestrin, "Powergraph: Distributed graph-parallel computation on natural graphs," in *10th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 12)*, pp. 17–30, 2012.
- [33] J. E. Gonzalez, R. S. Xin, A. Dave, D. Crankshaw, M. J. Franklin, and I. Stoica, "Graphx: Graph processing in a distributed dataflow framework," in *11th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 14)*, pp. 599–613, 2014.
- [34] M. Faloutsos, P. Faloutsos, and C. Faloutsos, "On power-law relationships of the internet topology," *ACM SIGCOMM computer communication review*, vol. 29, no. 4, pp. 251–262, 1999.
- [35] V. Balaji and B. Lucia, "When is graph reordering an optimization? studying the effect of lightweight graph reordering across applications and input graphs," in *2018 IEEE International Symposium on Workload Characterization (IISWC)*, pp. 203–214, 2018.

- [36] S. Beamer, K. Asanovic, and D. Patterson, "Locality exists in graph processing: Workload characterization on an ivy bridge server," in *2015 IEEE International Symposium on Workload Characterization*, pp. 56–65, IEEE, 2015.
- [37] J. Arai, H. Shiokawa, T. Yamamuro, M. Onizuka, and S. Iwamura, "Rabbit order: Just-in-time parallel reordering for fast graph analysis," in *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 22–31, IEEE, 2016.
- [38] H. Wei, J. X. Yu, C. Lu, and X. Lin, "Speedup graph processing by graph ordering," in *Proceedings of the 2016 International Conference on Management of Data*, pp. 1813–1828, 2016.
- [39] Y. Chen and Y.-C. Chung, "Corder: cache-aware reordering for optimizing graph analytics," in *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pp. 472–473, 2021.
- [40] Y. Chen and Y.-C. Chung, "Workload balancing via graph reordering on multicore systems," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 5, pp. 1231–1245, 2021.
- [41] Y. Chen and Y.-C. Chung, "An unequal caching strategy for shared-memory graph analytics," Tech. Rep. No. 1, System Software Lab., CUHK-SZ, 2021.
- [42] Y. Chen and Y.-C. Chung, "Hipa: Hierarchical partitioning for fast pagerank on numa multicore systems," in *50th International Conference on Parallel Processing*, pp. 1–10, 2021.
- [43] Y. Chen and Y.-C. Chung, "Co-design of mixed matrix format and processing engine for efficient spmv on graphs," Tech. Rep. No. 2, System Software Lab., CUHK-SZ, 2022.
- [44] Y. Chen and Y.-C. Chung, "Locality extraction & blocking for graph adjacency matrix multiplications," Tech. Rep. No. 3, System Software Lab., CUHK-SZ, 2022.
- [45] A.-L. Barabási and R. Albert, "Emergence of scaling in random networks," *science*, vol. 286, no. 5439, pp. 509–512, 1999.
- [46] S. Beamer, K. Asanović, and D. Patterson, "The gap benchmark suite," *arXiv preprint arXiv:1508.03619*, 2015.
- [47] R. Meusel, S. Vigna, O. Lehmberg, and C. Bizer, "The graph structure in the web—analyzed on different aggregation levels," *The Journal of Web Science*, vol. 1, 2015.
- [48] J. Leskovec and A. Krevl, "Snap datasets: Stanford large network dataset collection," 2014.
- [49] J. Kunegis, "Konekt: the koblenz network collection," in *Proceedings of the 22nd International Conference on World Wide Web*, pp. 1343–1350, 2013.

- [50] H. Kwak, C. Lee, H. Park, and S. Moon, "What is twitter, a social network or a news media?," in *Proceedings of the 19th international conference on World wide web*, pp. 591–600, 2010.
- [51] M. Cha, H. Haddadi, F. Benevenuto, and K. P. Gummadi, "Measuring user influence in twitter: The million follower fallacy," in *fourth international AAAI conference on weblogs and social media*, 2010.
- [52] Y. Zhang, V. Kiriansky, C. Mendis, S. Amarasinghe, and M. Zaharia, "Making caches work for graph analytics," in *2017 IEEE International Conference on Big Data (Big Data)*, pp. 293–302, IEEE, 2017.
- [53] P. Faldu, J. Diamond, and B. Grot, "A closer look at lightweight graph reordering," in *2019 IEEE International Symposium on Workload Characterization (IISWC)*, pp. 1–13, IEEE, 2019.
- [54] Y. Lim, U. Kang, and C. Faloutsos, "Slashburn: Graph compression and mining beyond caveman communities," *IEEE Transactions on Knowledge and Data Engineering*, vol. 26, no. 12, pp. 3077–3089, 2014.
- [55] P. Boldi, M. Rosa, M. Santini, and S. Vigna, "Layered label propagation: A multiresolution coordinate-free ordering for compressing social networks," in *Proceedings of the 20th international conference on World wide web*, pp. 587–596, 2011.
- [56] D. LaSalle and G. Karypis, "Multi-threaded graph partitioning," in *2013 IEEE 27th International Symposium on Parallel and Distributed Processing*, pp. 225–236, IEEE, 2013.
- [57] Y. Low, J. Gonzalez, A. Kyrola, D. Bickson, C. Guestrin, and J. M. Hellerstein, "Distributed graphlab: A framework for machine learning and data mining in the cloud," *Proceedings of the VLDB Endowment*, vol. 5, no. 8, 2012.
- [58] R. Chen, J. Shi, Y. Chen, and H. Chen, "Powerlyra: differentiated graph computation and partitioning on skewed graphs," in *Proceedings of the Tenth European Conference on Computer Systems*, pp. 1–15, 2015.
- [59] G. Karypis and V. Kumar, "Metis: A software package for partitioning unstructured graphs, partitioning meshes, and computing fill-reducing orderings of sparse matrices," 1997.
- [60] G. Karypis and V. Kumar, "A fast and high quality multilevel scheme for partitioning irregular graphs," *SIAM Journal on scientific Computing*, vol. 20, no. 1, pp. 359–392, 1998.
- [61] H. Meyerhenke, P. Sanders, and C. Schulz, "Partitioning complex networks via size-constrained clustering," in *International Symposium on Experimental Algorithms*, pp. 351–363, Springer, 2014.
- [62] G. M. Slota, K. Madduri, and S. Rajamanickam, "Pulp: Scalable multi-objective multi-constraint partitioning for small-world networks," in *2014 IEEE International Conference on Big Data (Big Data)*, pp. 481–490, IEEE, 2014.

- [63] G. M. Slota, S. Rajamanickam, K. Devine, and K. Madduri, "Partitioning trillion-edge graphs in minutes," in *2017 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 646–655, IEEE, 2017.
- [64] G. Karypis and V. Kumar, "Parallel multilevel series k-way partitioning scheme for irregular graphs," *Siam Review*, vol. 41, no. 2, pp. 278–300, 1999.
- [65] H. Meyerhenke, P. Sanders, and C. Schulz, "Parallel graph partitioning for complex networks," *IEEE Transactions on Parallel and Distributed Systems*, vol. 28, no. 9, pp. 2625–2638, 2017.
- [66] G. M. Slota, C. Root, K. Devine, K. Madduri, and S. Rajamanickam, "Scalable, multi-constraint, complex-objective graph partitioning," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 12, pp. 2789–2801, 2020.
- [67] A. Pinar and M. T. Heath, "Improving performance of sparse matrix-vector multiplication," in *SC'99: Proceedings of the 1999 ACM/IEEE Conference on Supercomputing*, pp. 30–30, IEEE, 1999.
- [68] A. Buluç, J. T. Fineman, M. Frigo, J. R. Gilbert, and C. E. Leiserson, "Parallel sparse matrix-vector and matrix-transpose-vector multiplication using compressed sparse blocks," in *Proceedings of the twenty-first annual symposium on Parallelism in algorithms and architectures*, pp. 233–244, 2009.
- [69] K. Kourtis, G. Goumas, and N. Koziris, "Exploiting compression opportunities to improve spm_{xv} performance on shared memory systems," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 7, no. 3, pp. 1–31, 2010.
- [70] B. Xie, J. Zhan, X. Liu, W. Gao, Z. Jia, X. He, and L. Zhang, "Cvr: Efficient vectorization of spmv on x86 processors," in *Proceedings of the 2018 International Symposium on Code Generation and Optimization*, pp. 149–162, 2018.
- [71] N. Bell and M. Garland, "Efficient sparse matrix-vector multiplication on cuda," tech. rep., Citeseer, 2008.
- [72] M. M. Baskaran and R. Bordawekar, "Optimizing sparse matrix-vector multiplication on gpus," *IBM Research Report RC24704*, no. W0812–047, 2009.
- [73] J. L. Greathouse and M. Daga, "Efficient sparse matrix-vector multiplication on gpus using the csr storage format," in *SC'14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 769–780, IEEE, 2014.
- [74] M. Steinberger, A. Derlery, R. Zayer, and H.-P. Seidel, "How naive is naive spmv on the gpu?," in *2016 IEEE High Performance Extreme Computing Conference (HPEC)*, pp. 1–8, IEEE, 2016.
- [75] F. G. Gustavson, "Some basic techniques for solving sparse systems of linear equations," in *Sparse matrices and their applications*, pp. 41–52, Springer, 1972.

- [76] R. F. Boisvert, R. F. Boisvert, and K. A. Remington, *The matrix market exchange formats: Initial design*, vol. 5935. US Department of Commerce, National Institute of Standards and Technology, 1996.
- [77] Y. Saad, *Iterative methods for sparse linear systems*. SIAM, 2003.
- [78] R. G. Grimes, D. R. Kincaid, and D. M. Young, *ITPACK 2.0 user's guide*. Center for Numerical Analysis, Univ., 1979.
- [79] G. Flegar and H. Anzt, "Overcoming load imbalance for irregular sparse matrices," in *Proceedings of the Seventh Workshop on Irregular Applications: Architectures and Algorithms*, pp. 1–8, 2017.
- [80] H. Anzt, T. Cojean, C. Yen-Chen, J. Dongarra, G. Flegar, P. Nayak, S. Tomov, Y. M. Tsai, and W. Wang, "Load-balancing sparse matrix vector product kernels on gpus," *ACM Transactions on Parallel Computing (TOPC)*, vol. 7, no. 1, pp. 1–26, 2020.
- [81] J. Kepner and J. Gilbert, *Graph algorithms in the language of linear algebra*. SIAM, 2011.
- [82] N. Sundaram, N. R. Satish, M. M. A. Patwary, S. R. Dulloor, S. G. Vadlamudi, D. Das, and P. Dubey, "Graphmat: High performance graph analytics made productive," *arXiv preprint arXiv:1503.07241*, 2015.
- [83] J. Kepner, P. Aaltonen, D. Bader, A. Buluç, F. Franchetti, J. Gilbert, D. Hutchison, M. Kumar, A. Lumsdaine, H. Meyerhenke, *et al.*, "Mathematical foundations of the graphblas," in *2016 IEEE High Performance Extreme Computing Conference (HPEC)*, pp. 1–9, IEEE, 2016.
- [84] M. Li, Y. Ao, and C. Yang, "Adaptive spmv/spmspv on gpus for input vectors of varied sparsity," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 7, pp. 1842–1853, 2020.
- [85] L. Page, S. Brin, R. Motwani, and T. Winograd, "The pagerank citation ranking: Bringing order to the web.," tech. rep., Stanford InfoLab, 1999.
- [86] U. Kang, D. H. Chau, and C. Faloutsos, "Mining large graphs: Algorithms, inference, and discoveries," in *2011 IEEE 27th International Conference on Data Engineering*, pp. 243–254, IEEE, 2011.
- [87] X. Zhu and Z. Ghahramani, "Learning from labeled and unlabeled data with label propagation," 2002.
- [88] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *arXiv preprint arXiv:1609.02907*, 2016.
- [89] M. Wang, D. Zheng, Z. Ye, Q. Gan, M. Li, X. Song, J. Zhou, C. Ma, L. Yu, Y. Gai, *et al.*, "Deep graph library: A graph-centric, highly-performant package for graph neural networks," *arXiv preprint arXiv:1909.01315*, 2019.
- [90] A. Buluç, H. Meyerhenke, I. Safro, P. Sanders, and C. Schulz, "Recent advances in graph partitioning," in *Algorithm Engineering*, pp. 117–158, Springer, 2016.

- [91] C. Jiang, F. Coenen, and M. Zito, "A survey of frequent subgraph mining algorithms," *Knowledge Engineering Review*, vol. 28, no. 1, pp. 75–105, 2013.
- [92] S. Harenberg, G. Bello, L. Gjeltema, S. Ranshous, J. Harlalka, R. Seay, K. Padmanabhan, and N. Samatova, "Community detection in large-scale networks: a survey and empirical evaluation," *Wiley Interdisciplinary Reviews: Computational Statistics*, vol. 6, no. 6, pp. 426–439, 2014.
- [93] K. Lakhotia, R. Kannan, and V. Prasanna, "Accelerating pagerank using partition-centric processing," in *2018 {USENIX} Annual Technical Conference ({USENIX}{ATC} 18)*, pp. 427–440, 2018.
- [94] S. Zhou, K. Lakhotia, S. G. Singapura, H. Zeng, R. Kannan, V. K. Prasanna, J. Fox, E. Kim, O. Green, and D. A. Bader, "Design and implementation of parallel pagerank on multicore platforms," in *2017 IEEE High Performance Extreme Computing Conference (HPEC)*, pp. 1–6, IEEE, 2017.
- [95] V. M. Weaver, "Linux perf_event features and overhead," in *The 2nd International Workshop on Performance Analysis of Workload Optimized Systems, FastPath*, vol. 13, p. 5, 2013.
- [96] J. Treibig, G. Hager, and G. Wellein, "Likwid: A lightweight performance-oriented tool suite for x86 multicore environments," in *2010 39th International Conference on Parallel Processing Workshops*, pp. 207–216, IEEE, 2010.
- [97] D. A. Spielman and S.-H. Teng, "A local clustering algorithm for massive graphs and its application to nearly linear time graph partitioning," *SIAM Journal on computing*, vol. 42, no. 1, pp. 1–26, 2013.
- [98] J. Shun, F. Roosta-Khorasani, K. Fountoulakis, and M. W. Mahoney, "Parallel local graph clustering," *Proceedings of the VLDB Endowment*, vol. 9, no. 12, 2016.
- [99] K.-Y. Liang, S. L. Zeger, and B. Qaqish, "Multivariate regression analyses for categorical data," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 54, no. 1, pp. 3–24, 1992.
- [100] A. Kleen, "A numa api for linux," *Novel Inc*, 2005.
- [101] S. Beamer, K. Asanović, and D. Patterson, "Reducing pagerank communication via propagation blocking," in *2017 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 820–831, IEEE, 2017.
- [102] "Intel® VTune™ Profiler Performance Analysis Cookbook ." <https://software.intel.com/content/www/us/en/develop/documentation/vtune-cookbook/top/tuning-recipes/os-thread-migration.html>, 2011. [Online; accessed 22-April-2021].
- [103] R. Intel, "Intel 64 and ia-32 architectures optimization reference manual," *Intel Corporation*, Sept, 2014.

- [104] D. Buono, F. Petrini, F. Checconi, X. Liu, X. Que, C. Long, and T.-C. Tuan, "Optimizing sparse matrix-vector multiplication for large-scale data analytics," in *Proceedings of the 2016 International Conference on Supercomputing*, pp. 1–12, 2016.
- [105] S. Schelter and J. Kunegis, "Tracking the trackers: A large-scale analysis of embedded web trackers," in *Tenth International AAAI Conference on Web and Social Media*, 2016.
- [106] A. Mislove, M. Marcon, K. P. Gummadi, P. Druschel, and B. Bhattacharjee, "Measurement and analysis of online social networks," in *Proceedings of the 7th ACM SIGCOMM conference on Internet measurement*, pp. 29–42, 2007.
- [107] S. Auer, C. Bizer, G. Kobilarov, J. Lehmann, R. Cyganiak, and Z. Ives, "Dbpedia: A nucleus for a web of open data," in *The semantic web*, pp. 722–735, Springer, 2007.
- [108] M. Frigo, C. E. Leiserson, and K. H. Randall, "The implementation of the cilk-5 multithreaded language," in *Proceedings of the ACM SIGPLAN 1998 conference on Programming language design and implementation*, pp. 212–223, 1998.
- [109] "OpenCilk." <https://cilkplus.github.io/>, 2011. [Online; accessed 21-Sept-2021].
- [110] D. Nguyen, A. Lenharth, and K. Pingali, "A lightweight infrastructure for graph analytics," in *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, pp. 456–471, 2013.
- [111] "Optimizing Applications for NUMA." <https://software.intel.com/content/www/us/en/develop/articles/optimizing-applications-for-numa.html>, 2011. [Online; accessed 21-April-2021].
- [112] N. Satish, N. Sundaram, M. M. A. Patwary, J. Seo, J. Park, M. A. Hassaan, S. Sengupta, Z. Yin, and P. Dubey, "Navigating the maze of graph analytics frameworks using massive graph datasets," in *Proceedings of the 2014 ACM SIGMOD international conference on Management of data*, pp. 979–990, 2014.
- [113] A. Buluc and J. R. Gilbert, "On the representation and multiplication of hypersparse matrices," in *2008 IEEE International Symposium on Parallel and Distributed Processing*, pp. 1–11, IEEE, 2008.
- [114] M. H. Mofrad, R. Melhem, Y. Ahmad, and M. Hammoud, "Efficient distributed graph analytics using triply compressed sparse format," in *2019 IEEE International Conference on Cluster Computing (CLUSTER)*, pp. 1–11, IEEE, 2019.
- [115] R. A. Rossi and N. K. Ahmed, "An interactive data repository with visual analytics," *ACM SIGKDD Explorations Newsletter*, vol. 17, no. 2, pp. 37–41, 2016.
- [116] D. Chakrabarti, Y. Zhan, and C. Faloutsos, "R-mat: A recursive model for graph mining," in *Proceedings of the 2004 SIAM International Conference on Data Mining*, pp. 442–446, SIAM, 2004.
- [117] L. Dagum and R. Menon, "Openmp: an industry standard api for shared-memory programming," *IEEE computational science and engineering*, vol. 5, no. 1, pp. 46–55, 1998.

- [118] M. H. Mofrad, R. Melhem, Y. Ahmad, and M. Hammoud, "Graphite: a numa-aware hpc system for graph analytics based on a new mpi* x parallelism model," *Proceedings of the VLDB Endowment*, vol. 13, no. 6, pp. 783–797, 2020.
- [119] Y. Ahmad, O. Khattab, A. Malik, A. Musleh, M. Hammoud, M. Kutlu, M. Shehata, and T. Elsayed, "La3: A scalable link-and locality-aware linear algebra-based graph analytics system," *Proceedings of the VLDB Endowment*, vol. 11, no. 8, pp. 920–933, 2018.
- [120] M. D. Lam, E. E. Rothberg, and M. E. Wolf, "The cache performance and optimizations of blocked algorithms," *ACM SIGOPS Operating Systems Review*, vol. 25, no. Special Issue, pp. 63–74, 1991.
- [121] E.-J. Im, K. Yelick, and R. Vuduc, "Sparsity: Optimization framework for sparse matrix kernels," *The International Journal of High Performance Computing Applications*, vol. 18, no. 1, pp. 135–158, 2004.
- [122] "cuda memory hierarchy." <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html>, 2011. [Online; accessed 01-May-2022].